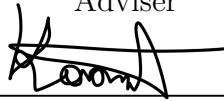


RECOMBINANT: A COMPOSABLE, EXTENSIBLE, INTELLIGENT I/O
PLATFORM FOR INTEGRATED HIGH-PERFORMANCE I/O MANAGEMENT

BY
LUKE LOGAN

Submitted in partial fulfillment of the
requirements for the degree of
Doctor of Philosophy in Computer Science
at the Illinois Institute of Technology

Approved _____
Adviser
Approved _____
Co-Adviser

Chicago, Illinois
December 2025

© Copyright by
LUKE LOGAN
2025

ACKNOWLEDGMENT

I am deeply grateful to my adviser, Prof. Xian-He Sun, and my co-adviser, Prof. Anthony Kougkas, who both contributed significantly to the conceptualization and progress of the research project throughout my doctoral studies. Prof. Sun's wisdom and guidance have been instrumental in shaping my research direction and methodology, while Prof. Kougkas has been an exceptional mentor, providing continuous feedback, guidance, and support that enabled me to develop both technically and professionally. Dr. Jay Lofstead generously shared his expertise and resources, offering valuable guidance that significantly contributed to the research presented in this thesis.

I also want to extend my sincere thanks to my friends and colleagues at the Gnosis Research Center. Our engaging discussions and collaborative spirit were instrumental in shaping the ideas presented in this thesis, and your intellectual partnership made this endeavor both challenging and rewarding.

Lastly, I am deeply grateful to my family for their unwavering love, encouragement, and understanding throughout this demanding journey. Your support has been my constant source of strength.

AUTHORSHIP STATEMENT

I, Luke Logan, attest that the work in this thesis is substantially my own.

In accordance with the disciplinary norm of Computer Science (see IIT Faculty Handbook, Appendix S), the following collaborations occurred in the thesis:

This thesis incorporates material from two of my prior publications, which I led as the primary author. Chapter 2 builds upon my work published as [1]¹, where I was responsible for the design, implementation, and evaluation of the system. My advisor Xian-He Sun provided essential research direction that enhanced the overall quality of this work. My co-advisor Anthony Kougkas offered critical feedback that strengthened the contribution. Jaime Cernuda Garcia, a trusted colleague and friend, contributed valuable insights and technical discussions that enhanced the work. Jay Lofstead provided essential guidance on system design and access to resources.

Chapter 4 extends my work published as [2]², where I served as the lead researcher responsible for the system design, implementation, experimental evaluation, and manuscript preparation. My advisor Xian-He Sun provided essential research direction and critical feedback that strengthened this work. My co-advisor Anthony Kougkas offered valuable technical guidance that enhanced the quality and presentation of this work.

All figures and experimental results presented in this thesis are my own work, unless otherwise cited.

¹© 2022 IEEE. Reprinted, with permission, from Luke Logan; Jaime Cernuda Garcia; Jay Lofstead; Xian-He Sun; Anthony Kougkas, LabStor: A Modular and Extensible Platform for Developing High-Performance, Customized I/O Stacks in Userspace, 13-18 November 2022

²© 2024 IEEE. Reprinted, with permission, from Luke Logan; Anthony Kougkas; Xian-He Sun, MegaMmap: Blurring the Boundary Between Memory and Storage for Data-Intensive Workloads, 17-22 November 2024

TABLE OF CONTENTS

	Page
ACKNOWLEDGMENT	iii
AUTHORSHIP STATEMENT	iv
LIST OF TABLES	vii
LIST OF FIGURES	viii
ABSTRACT	xi
CHAPTER	
1. INTRODUCTION	1
1.1. Research Questions	3
1.2. The Recombinant System for Composing Custom I/O Stacks over Heterogeneous Hardware	5
1.3. Design of Recombinant	9
1.4. Contributions	11
2. LABSTOR: A MODULAR AND EXTENSIBLE PLATFORM FOR DEVELOPING HIGH-PERFORMANCE, CUSTOMIZED I/O STACKS IN USERSPACE	13
2.1. Introduction	13
2.2. Background and Related Work	16
2.3. The LabStor Platform	22
2.4. Evaluations	42
2.5. Conclusion	54
3. CHIMAERA: A DISTRIBUTED SEMI-MICROKERNEL FOR COMPOSING HETEROGENEOUS DATA SERVICES	55
3.1. Introduction	55
3.2. Background and Related Work	59
3.3. Design	64
3.4. Evaluations	81
3.5. Conclusion	91
4. MEGAMMAP: BLURRING THE BOUNDARY BETWEEN MEM- ORY AND TIERED STORAGE FOR DATA-INTENSIVE WORK- LOADS	92
4.1. Introduction	92
4.2. Background and Related Work	96

4.3. Design	100
4.4. Evaluations	115
4.5. Discussion & Limitations	128
4.6. Conclusion	129
5. ETERNIA: AN EFFECTIVELY INFINITE DISTRIBUTED SHARED MEMORY FOR GPU WORKLOADS	130
5.1. Introduction	130
5.2. Background and Related Work	134
5.3. Design	138
5.4. Evaluations	155
5.5. Conclusion	164
6. RELATED WORKS	165
6.1. I/O Development Platforms	165
6.2. Heterogeneous Storage and Memory Management	168
6.3. Mediating Contention on Storage and Processors	172
7. CONCLUSION	176
7.1. Summary	176
7.2. Future Work	180
BIBLIOGRAPHY	182

LIST OF TABLES

Table		Page
2.1	LabMod live upgrade performance in seconds	44
2.2	P99 Latency of Linux and LabStor schedulers	50
5.1	Comparison between Eternia and Alternatives	134

LIST OF FIGURES

Figure	Page
1.1 High-level architecture of the Recombinant platform.	10
2.1 Comparison of different I/O stack design philosophies	17
2.2 Component Diagram	30
2.3 LabStack Example	38
2.4 I/O stack anatomy	42
2.5 Dynamic Work Orchestration	45
2.6 Storage API performance. IOPS are normalized.	47
2.7 Metadata throughput	48
2.8 Performance of different I/O Schedulers	49
2.9 LabStor under different storage systems	51
3.1 Depiction of Chimaera for scheduling an application operation (OP) on heterogeneous hardware	55
3.2 Comparison of heterogeneous I/O stacks. Horizontal means moving state + computations across processors or nodes. Vertical means moving computations across processors within a single machine de- pending on current load.	59
3.3 High-level architecture of Chimaera.	64
3.4 Basic ChiMod API	66
3.5 Effect of sampling frequency on runtime and monitoring accuracy. Low frequency (1/10) has high performance but takes 8x longer to reach maximum accuracy. High frequency gets to maximum accu- racy fastest, but incurs an overhead of 27% compared to the 4/6 fre- quency. 4/6 obtains best performance since the model is more accu- rate than 1/10 for most of the program, leading to better scheduling decisions.	83
3.6 Effect of upgrade on runtime performance for a 4KB I/O benchmark on 4 nodes. The upgrade is sent at 6 seconds, resulting in a brief dip in performance for 8ms in total.	84

3.7	Comparison of two I/O stacks for two different workloads. Compression gives VPIC a 40% performance improvement by reducing I/O volume. Compression, however, gives significant overhead to the latency-sensitive web server.	86
3.8	Comparison of I/O stack designs for running a transparent KMeans clustering. Chimaera, unlike all other solutions, utilizes all available processors, yielding the highest performance and resource utilization.	87
3.9	Performance and utilization of co-locating compute- and bandwidth-heavy Gray Scott and latency-sensitive varmail for different scheduling policies.	90
4.1	Depiction of Coherent, Distributed, and Tiered MegaMmap Memory	94
4.2	MegaMmap Architecture	100
4.3	Overview of coherence policies. Read (R) / write (W) to non-overlapping regions will first place data locally on the node that produced it without synchronization. Read-only will allow for data to be replicated in the shared cache to improve availability. Read, write, append global will hash page faults and evictions to the same worker.	110
4.4	MegaMmap code 45% - 2x smaller. In each case, all I/O partitioning, I/O compatability, and most messaging is removed.	117
4.5	Weak scaling study of MegaMmap vs alternative application designs, where datasets fit entirely in memory. MegaMmap performs competitively to MPI and as much as 2x faster than Spark.	121
4.6	Increasing the resolution of Gray-Scott through tiering. After $L = 2688$, MPI-based Gray-Scott crashes due to memory overutilization. MegaMmap is unbounded, and can run simulations as large as $L = 3456$, producing 2x the simulation data of $L = 2688$. It's also at least 20% faster than other tiered I/O systems due to effective asynchronous data movement.	123
4.7	Tiering study of MegaMmap for 768-process Gray-Scott. D=DRAM, H=HDD, S=SATA SSD, N=NVMe. The number indicates the amount of storage per-node given to the program (e.g., 48D means 48GB DRAM per node). MegaMmap improves performance as much as 1.8x by using NVMe. However, performance is related closely to cost.	125

4.8	Memory scaling study of MegaMmap. Each program has 1536 processes over 32 nodes. Through intelligent tiering, DRAM can be lowered as much as 2.6x while maintaining competitive (within 10%) performance of full DRAM capacity.	127
5.1	Depiction of Eternia for Tiered Storage	131
5.2	High-level Architecture of Eternia	143
5.3	Comparison against state-of-the art solutions. In this case where the data fits entirely in memory, Eternia has minimal overhead to applications.	157
5.4	Comparison of different page size granularities on performance. . .	158
5.5	Comparison of performance and resource utilization of mapping different data formats with Eternia.	159
5.6	Increase resolution of GPU Gray Scott through tiered memory. After 128GB, cuFile and DDN Gray Scott crash due to memory overutilization. Eternia is unbounded and can produce 2x the simulation data. It's also at least 18% faster than other tiered I/O systems due to effective asynchronous data movement.	160
5.7	Tiering study of Eternia for 4-node KMeans. G=GPU (HBM), D=DRAM, H=HDD, N=NVMe. The number indicates the amount of storage per-node given to the program (e.g., 32D means 32GB DRAM per node). Aggregation improves performance as much as 8x by reducing number of I/O ops and increasing I/O sizes. Prefetching improves performance as much as 30% by informing data placement. Introducing tiers always improves performance, but benefits related to cost.	162
5.8	Memory reduction study of Eternia. 4096 threads per node, 4 nodes. Through performance matching, DRAM can be lowered as much as 4x while maintaining competitive (within 10%) performance of full DRAM capacity.	163

ABSTRACT

Modern workloads spanning data analytics, artificial intelligence (AI), scientific simulation, and business intelligence are becoming increasingly data-intensive. As computing centers move towards exascale deployments, storage systems struggle to meet the extreme computing requirements of these diverse workloads with their dynamic and conflicting I/O behaviors, resulting in severe I/O bottlenecks on application runtimes. While machine architectures are equipping many new storage and processing technologies simultaneously, this explosion of storage and processing hardware and specialized I/O stacks is not accompanied by an abstraction to mask its complexity from everyday users. Existing storage solutions were rigidly designed for particular applications and hardware, without sustainable methods to expand to future advances, making the development of correct, trustworthy I/O stacks notoriously difficult. In this work, we introduce Recombinant: a composable I/O platform for diverse data-intensive applications running on complex machine architectures. Recombinant re-envision the methodology by which I/O stacks are developed, deployed, and scheduled across all integrated resources through a loosely-coupled development platform enabling custom I/O features to be built and uploaded with minimal service interruption and evolved over time. We develop and compose several custom I/O stacks from these I/O features to address workload-specific I/O requirements while automatically scheduling their work across all integrated processing hardware. Contention across co-located I/O stacks and applications is mediated through intelligent orchestration policies that dynamically learn and factor computational and I/O behavior. We demonstrate this composable design through various evaluations: a modular I/O stack design achieving up to 10 $\times$ fewer lines of code and 60% performance improvements, intelligent hardware matching reducing peak memory utilization by up to 4 $\times$, and multi-tenant orchestration achieving up to 80 $\times$ lower p99 latency through intelligent I/O operation scheduling.

CHAPTER 1

INTRODUCTION

Modern workloads spanning various fields such as data analytics, data processing, business intelligence, artificial intelligence (AI), and simulation are becoming increasingly data-intensive. Yet, large-scale computing clusters are increasingly suffering from an I/O bottleneck. As computing centers move towards exascale deployments, storage systems struggle to meet the extreme computing requirements of these various workloads, resulting in severe I/O penalties on application runtimes [3]. Outside of sheer data volume, applications semantics are also becoming increasingly diverse, which leads to dynamic and conflicting I/O behaviors over storage. For example, deep learning workloads often result in many small, random I/O operations for training, while simulations tend to result in large, periodic bursts of data. This can result in conflict on caching, data organization, and metadata systems. This reality continues even with the rise of Autonomous AI agents, which are expected to become the main drivers of data exploration and software deployment in the near future. The IOWarp project [4], for example, proposes a natural-language interface for scientists to explore data and easily execute diverse scientific simulations and analytics software through custom AI agents and MCPs. In this system, a scientist could theoretically ask the IOWarp agent, “Will there be a tornado next week?”, and it could respond to the question by launching a weather simulation and processing its output.

While work is being done to improve the performance of I/O systems through a combination of hardware and software techniques, there is still significant manual effort to leverage the existing work. Various storage hardware (e.g., NVMe) technologies have been developed with different capacity and performance characteristics to act as intermediate storage to improve I/O times [5–8]. Hardware-specific I/O systems [9–11] and near-data processors [12, 13] have been proposed to further improve

performance, reduce CPU utilization, and reduce chances of failure. To optimize for a variety of workloads, machine architectures are equipping many of the aforementioned technologies simultaneously [6]. However, this explosion of storage/processing hardware and specialized I/O stacks is not accompanied by an abstraction to mask its complexity from everyday users. There is a difficult matching problem of where to place data and computations, which leads to either suboptimal manual solutions or simply lack of use. Existing storage solutions were rigidly designed for particular applications and hardware, without sustainable methods to expand to future advances [14]. Developing correct, trustworthy I/O stacks to accomplish this is notoriously difficult [15], especially when the hardware and application landscape is constantly evolving.

In this work, our main research question is: **How can we develop I/O systems to satisfy diverse application I/O requirements while integrating all available storage and processing hardware?** We argue that a *Composable I/O system* can provide the flexibility required for this. *Composable* is defined as the ability for users (e.g., people or AI) to create custom I/O paths and features for their specific circumstance while allowing the I/O system to intelligently schedule its I/O stack work across all integrated hardware to improve resource utilization and mediate contention across conflicting I/O patterns.

1.1 Research Questions

To build a composable I/O system, we need to address the following research questions.

Question 1: *How do we improve I/O stack development methodologies to account for rapid evolution in hardware and application I/O requirements without drastic code changes?*

I/O stacks are currently developed with a rigid set of tightly coupled features, leading to costly redevelopment efforts and lack of extensibility [1, 14]. Many I/O stack solutions do not provide the ability to upgrade code and release features dynamically [16–18], requiring significant service interruption and potentially years of manual development and debugging efforts to evolve [15]. For these reasons, I/O code is either infrequently or never upgraded after being deployed in many computing environments [1, 15]. However, applications and machine architectures are diverse and rapidly evolving, requiring flexibility and extensibility from I/O stacks to meet performance and resource utilization objectives. A scientific AI processing user inquiries, for example, may need complex, multi-variate, workload-specific operators such as partial differential equations [19, 20] to execute transparently in the I/O stack to reduce data movements, which requires the ability to customly develop and dynamically deploy in-transit data operators. Live fraud detection algorithms require specific clustering and classification algorithms transparently operating on incoming banking transactions to quickly detect malicious behavior [21]. Alternatively, CXL devices are emerging and will expose new storage interfaces, which will require code changes in the I/O stack to account for.

Question 2: *How do we mask the growing diversification of processing and storage hardware to improve resource utilization for applications?*

I/O Stacks have traditionally been designed for a specific machine architecture and problem which leads to unportable/hardware-dependent designs, suboptimal resource

utilization, and contention. For example, SmartNICs contain wimpy processors that can be used to offload filesystem operations such as compression. However, compression is an expensive operation. For a data-intensive application running a highly parallel, coordinated scientific workflow where I/O sizes are large, always offloading in-transit compression operations can result in overutilization when the accelerator has lower core frequencies, core counts, and memory than the CPU [10]. Alternatively, never offloading compression can result in contention with application threads, also resulting in performance degradation due to phenomena such as the straggler effect [22]. Alternatively, parallel filesystems were designed to provide high capacity for highly parallel scientific simulations, but are traditionally built for only a single tier of storage (mainly HDDs), ignoring high-performance intermediate storage such as node-local NVMe while also resulting in suboptimal performance for other problem types, such as latency-sensitive databases [1].

Question 3: *How do we manage contention across all integrated storage and processing hardware in diverse, multi-tenant computing environments?* In multi-tenant situations where several applications are processed simultaneously, I/O stacks designed for specific workloads result in suboptimal resource utilization and contention when executing simultaneously [23, 24]. For example, a latency-sensitive application (e.g., AurorGPT’s web server for querying scientific publications) executing on the same machine as a bandwidth-intense application (e.g., an inquiry that spawns a scientific simulation) can result in severe I/O and network contention [10, 25], causing high latencies, poor quality-of-service, and bandwidth starvation without careful mediation. Alternatively, two I/O stacks that both perform in-transit computations can result in contention and poor resource utilization when scheduled on the same processor [26].

1.2 The Recombinant System for Composing Custom I/O Stacks over Heterogeneous Hardware

To answer the aforementioned questions, we introduce Recombinant, which facilitates the development and execution of customly-composed I/O stacks for diverse applications running on complex machine architectures. Recombinant addresses Question 1 by providing modular development tools that enable I/O stacks to be composed from reusable, single-responsibility modules that can be independently developed, debugged, and upgraded with minimal service interruption, allowing rapid adaptation to evolving hardware and application requirements. Question 2 is addressed through intelligent hardware matching algorithms that present diverse storage and processing hardware under intuitive interfaces, including infinite memory abstractions and key-value stores. Question 3 is addressed through contention mediation policies that use reinforcement learning to automatically mediate resource contention across heterogeneous processors and storage by globally considering the highly dynamic I/O and computational characteristics of co-located I/O stacks. **The ultimate goal of this work is to foster the rapid development of efficient, future-proof I/O stacks that integrate all storage and processing hardware on the machine to maximize resource utilization.**

1.2.1 Challenges under Recombinant to achieve Composable Storage

To achieve these goals, we identify the following main challenges:

- C1 *I/O stacks must switch to a modular design.* Traditionally, I/O systems have been developed within the confines of a centralized OS kernel. This led to monolithic and rigid storage systems that are limited by low development speed, expressiveness, and performance. Various assumptions are imposed including reliance on the UNIX-file abstraction, the POSIX standard, and a narrow set of I/O policies. This monolithic design philosophy makes it difficult to develop

and deploy new I/O approaches to satisfy the rapidly-evolving I/O requirements of modern applications. However, switching to a modular design is difficult because applications have a variety of I/O requirements (e.g., I/O interface, data operators) and hardware designs are highly diverse. Finding suitable abstraction to encompass this variety and enable code reuse while also respecting the need for workload and hardware-specific optimization and future innovations is non-trivial.

- C2 *New reactive algorithms must be created to automatically match I/O and computations to heterogeneous hardware and execute them.* Large-scale, data-intensive applications spanning HPC and Cloud have highly diverse I/O and computational requirements, resulting in the adoption of heterogeneous storage hardware and the development of specialized data-centric accelerators, such as SmartNICs, Processing In Memory (PIM), and computational storage. This has led to the development of many one-off storage solutions that are developed in the confines of a particular hardware configuration and workload, resulting in costly redevelopment efforts, limited portability, lack of customization, and suboptimal resource utilization. However, automatically matching I/O and computations to hardware is difficult because each new storage and processor have special characteristics that require expertise to manage. With each new technology, data movement and processing becomes far more complex.
- C3 *New scheduling policies must be created that understand diverse, multi-tenant application behavior and hardware characteristics.* Data-intensive, large-scale computing centers are expected to run a variety of applications simultaneously. Each of these applications have varied expectations of the I/O stack, and contend with each other in different ways over memory, storage, and processing elements. However, automatically mediating contention is difficult because ap-

plications are highly diverse and conflict with each other differently, resulting in an exponentially-complex problem to address each combination of programs. The effects of conflict also vary depending on the storage and processing hardware.

1.2.2 Solutions under Recombinant

we have developed four software systems to address the above challenges as given below. These four components form the foundations of Recombinant

S1 LabStor: We first present a composable, evolutionary system for building single-node I/O stacks over heterogeneous storage that can be refined by AI, people or applications. To rapidly and comprehensively address workload-specific I/O requirements and incorporate new innovations in hardware (C1), LabStor introduces a novel code module development kit and API that allows I/O features to be independently developed, debugged, and upgraded by experienced developers. End users or AI can compose these plugins into specialized I/O stacks. Experimental results demonstrate that customized LabStor I/O stacks can yield performance gains of up to 60% over alternatives for various workloads and storage devices while allowing I/O code to be upgraded live within milliseconds, enabling rapid adoption and utilization of new hardware and workload-specific optimizations. Experimental results demonstrate that customized LabStor I/O stacks can yield performance gains of up to 60% over alternatives under various workloads and storage devices while allowing I/O code to be upgraded in milliseconds. (Chapter 2).

S2 Chimaera: We then evolved this system to the general case through a composable, evolutionary system for building reliable, distributed I/O stacks over heterogeneous storage and processors. In this work, we extend the abstrac-

tions in LabStor to be distributed and encompass both heterogeneous processors and storage to allow distributed, portable I/O stacks to be rapidly developed (Challenge C1). Chimaera introduces a novel algorithm that matches computations to heterogeneous processing hardware based on live application monitoring and specific processor characteristics (Challenge C2). Chimaera also introduces a novel intelligent process scheduling algorithm that automatically mediates resource contention across heterogeneous processors and storage by globally considering the highly dynamic I/O and computational characteristics across co-located I/O stacks when matching them to hardware to ensure high performance and resource utilization (Challenge C3). This policy uses reinforcement learning techniques to automatically adapt to diverse data-intensive application behaviors. Experimental results demonstrate that I/O paths composed in Chimaera can yield performance gains of up to 46% over alternatives under various workloads by utilizing heterogeneous processors. Chimaera is currently used as the foundations for the implementation of several funded projects, including Hermes, IoWarp, and COEUS grants, totalling nearly \$10 million of funding. (Chapter 3).

S3 MegaMmap: Using these development tools, we build an effectively infinite multi-tiered shared memory abstraction for memory-intensive applications. MegaMmap, which demonstrates a specific I/O stack built in Chimaera for memory-constrained data-intensive workloads, provides a new, intuitive memory abstraction that allows massive datasets to be presented as if they were in memory while automatically matching memory pages across heterogeneous memory and storage through novel, intelligent prefetching and data movement policies that factor dynamic access pattern and storage characteristics (Challenge C2). Applications inform MegaMmap of its access pattern intent through a novel transactional memory API, allowing MegaMmap to generally to a di-

versity of applications. Evaluations showcase that MegaMmap reduces peak memory utilization by as much as $2.6\times$ compared to leading solutions while accomplishing similar or better performance. Evaluations showcase that developing memory-intensive applications with MegaMmap reduces peak memory utilization by as much as $2.6\times$ compared to leading solutions while accomplishing similar or better performance. (Chapter 4).

S4 **Eternia**: Building upon the foundations established by MegaMmap, Eternia extends the memory abstraction paradigm to address the unique challenges of GPU-centric applications (Challenge C2). Eternia provides a distributed shared memory abstraction that enlarges effective GPU memory capacity through intelligent tiered DRAM and storage management, allowing massive datasets to be presented as if in GPU memory. Through a parallel transactional memory API, GPU kernels can inform data placement, prefetching, and metadata caching algorithms to ensure critical data is efficiently accessible. Eternia addresses the full lifecycle of GPU data, enabling direct storage access when possible while ensuring persistence across diverse storage tiers. Evaluations demonstrate that Eternia can reduce HBM consumption for various workloads up to $4\times$ **without sacrificing performance through effective storage tiering**. (Chapter 5).

1.3 Design of Recombinant

The Recombinant platform is shown in Figure 1.1. Recombinant is divided into three main components:

1. **Development Tools**: Software to develop new, modular I/O stacks (Challenge C1). LabMods (Solution S1) provide the abstractions for building single-node I/O stacks, while ChiMods (Solution S2) are their direct extension to distributed I/O stacks. They are used to build single-responsibility I/O mod-

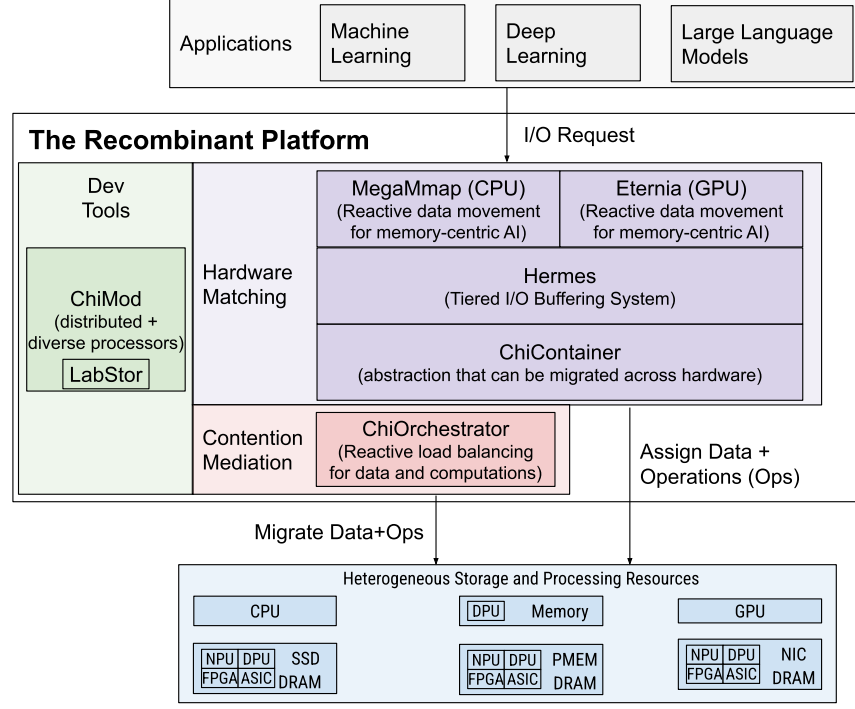


Figure 1.1. High-level architecture of the Recombinant platform.

ules that can be reused across I/O stacks and stiched together to form complex I/O systems. A ChiMod, for example, could represent a filesystem or an encryption module. When combined, they form an I/O stack with transparent encryption.

2. **Hardware Matching**: New reactive I/O systems developed using the Development Tools component to automatically match I/O and computations to heterogeneous hardware (Challenge C2). MegaMmap (Solution S3) and Eternia (Solution S4) provide infinite memory abstractions that transparently spill to storage when memory becomes overutilized. MegaMmap is for CPU-driven workloads, while Eternia is for GPU-first workloads. Hermes is a multi-tiered I/O system with extensible data placement and reorganization policies used by MegaMmap and Eternia for data placement. Hermes is an NSF project engineered on top of Chimaera, though not directly proposed by this dissertation.

ChiContainers (Solution S2) are the fundamental abstraction that MegaMmap, Eternia, and Hermes inherit from to facilitate contention mediation by providing the basic callbacks and methods necessary to migrate data and computations across nodes and processors.

3. **Contention Mediation:** New policies that understand diverse, multi-tenant application behavior and hardware characteristics (Challenge C3). The ChiOrchestrator (Solution S2) presents a novel process scheduling algorithm that automatically mediates resource contention across heterogeneous processors and storage by globally considering the highly dynamic I/O and computational characteristics across co-located I/O stacks when matching them to hardware to ensure high performance and resource utilization.

1.4 Contributions

This work contributes to the existing body of research as follows:

1. The traditional, individualistic methodology for developing I/O stacks must be re-envisioned to address diverse data-intensive application I/O requirements. Switching to a modular I/O stack design allows workload-specific and hardware-optimized I/O paths to be constructed rapidly and easily maintained without sacrificing performance. We demonstrate up to 10x fewer lines of code due to efficient code reuse in addition to up to 60% performance improvements from using specially-composed I/O services.
2. Methods for utilizing the increasing storage and processor heterogeneity of modern computing environments must be improved to ensure the sustainability and performance of data-intensive workloads. Automatically matching I/O and computations to hardware allows improvements to both energy-efficiency

and performance while leaving applications unburdened by these decisions. We demonstrate up to 4x peak memory utilization reduction by efficient hardware matching while maintaining comparable performance.

3. With the rise of diverse applications and the multi-tenant nature of modern computing environments, methods for sharing resources must be improved to minimize cross-application interference. Scheduling policies that factor dynamic and diverse application behavior and hardware characteristics improve performance and resource utilization in multi-tenant environments. We demonstrate that multi-tenant workloads benefit up to 80x lower p99 latency by intelligent orchestration of I/O operations.

CHAPTER 2

LABSTOR: A MODULAR AND EXTENSIBLE PLATFORM FOR DEVELOPING HIGH-PERFORMANCE, CUSTOMIZED I/O STACKS IN USERSPACE

2.1 Introduction

HPC systems are increasingly suffering from a storage bandwidth bottleneck. Compute performance is increasing at a much faster rate than storage bandwidth. In this environment, new storage technologies are being deployed that offer much higher bandwidth with low latency, but the I/O stack through the Linux kernel imposes serious overheads. Approaches such as demonstrated in pMEMCPY [27] show it is possible to adapt to the native interfaces and dramatically increase bandwidth. Unfortunately, these are one-off solutions for each different technology. To truly achieve all available bandwidth, the design philosophy of the Linux I/O stack must be reconsidered.

Parallel filesystems (PFS), such as Lustre [28], and other distributed storage systems (e.g., key-value stores, (No)SQL databases) [29] rely on native Linux filesystems (e.g., ext4) as their backend on each storage server. This is because they provide a well-tested, secure, and familiar interface to a diverse set of storage hardware devices [30, 31]. However, this reliance imposes limitations [29, 32, 33] on all I/O operations. Specifically, Linux filesystems suffer from three main issues regarding I/O system development and performance: a) Severe performance degradation caused by workload-agnostic policies (e.g., I/O scheduling, caching); b) Significant software overhead for low-latency devices [12, 30, 34–36] caused by frequent context switching and data copying [31, 37]; c) Limited expressiveness caused by the UNIX file and the strong POSIX-compliance (e.g., transactions are difficult to program using the UNIX file representation [29]) forcing other data representations to be translated into a file. Since PFS offer a higher-level view of the data on top of multiple local

storage servers, the I/O stacks used locally on each storage node must be optimized to improve performance and customizability of the the distributed layer.

This work seeks to re-envision the way that I/O stacks are developed and deployed in order to provide per-workload and per-hardware optimizations. To achieve this, we have to overcome several obstacles. First, the methodologies used to implement I/O systems must be substantially improved to provide high-velocity development. Developing within the kernel should be avoided since it is limited in functionality, does not support upgrades without service interruptions, and is generally complex and time consuming [15,38–40]. Monolithic design patterns [41] should also be avoided since it slows development speed, testing is harder, and code-reuse is limited [42,43]. All layers of the I/O stack, including I/O interfaces, must be modular and hence easily programmable [44–50]. Second, wide support for both traditional and modern storage hardware must be maintained. Development platforms which support only one type of hardware should be avoided since they increase assumptions, enforce dependencies, limit portability, and lower adoption rate [51,52]. It is unrealistic to expect re-development of device drivers for all storage mediums current or future. Third, flexible and tunable access control mechanisms must be provided. Strict protocols enforced globally and holistically should be avoided since they have limited guarantees [53–55], are not always required [56–58], and impose significant performance costs [35,59]. Fourth, high-performance must be ensured. Restricting device access behind long I/O paths should be avoided as they cause software overheads [60–62], limit expressiveness [50,63,64], and dismiss hardware-specific optimizations [65–67]. The ability to directly access hardware using its native APIs should be provided [56–58,68,69]. Fifth, end-users should find it easy to integrate newly-developed I/O stacks. Requiring the individual deployment of multiple, independently developed I/O systems through interception should be avoided, as this requires expertise to resolve conflicts in symbol resolution, hinders productivity, and invites erroneous outcomes [42,70,71].

Modular and programmable stacks hold promise to address all these challenges and have tremendous impact on both the developers and the end users of I/O systems.

To address the limitations of rigid, monolithic designs, we propose LabStor: a modular, extensible, and high-performance I/O platform that can be customized and extended to meet the needs of different environments. LabStor introduces the following contributions:

1. **LabMods:** single-purpose, self-contained code objects which can be developed in the comfort of userspace and released as plug-ins. LabMods can represent any functionality, such as I/O scheduling, caching, access control, or filesystems, and can provide any interface, including POSIX, key-value store, SQL, etc.
2. **LabStacks:** A combination of LabMods made by end-users to form optimized, situation-specific I/O stacks. They are defined in a human-readable schema file.
3. **LabStor Runtime:** The system for deploying, querying, debugging, upgrading and executing LabStacks. It provides various utility scripts and a userspace process/thread scheduling framework for balancing LabStack work with application work.
4. **Implementation:** LabStor comes equipped with a variety of LabMods, including Driver LabMods, which enable out-of-the-box support for developing new I/O stacks for different hardware types.

Currently, no framework exists that provides high-velocity development, hardware diversity, and manageability, while also providing I/O interface diversity and performance. LabStor’s I/O stacks and interfaces can be uniquely customized and rapidly developed to support a wide variety of workloads in both HPC and Cloud environments, all with the improved I/O performance and manageability required by

modern systems. While this approach may appear to limit itself to a single node or an exclusive storage system, consider burst buffers or other fast storage tiers. In many cases, such as in ORNL’s Summit machine and LLNL’s upcoming El Capitan, node-local storage system extensions could take advantage of these designs to maximize overall storage performance. Gestalt machine designs [72] may use persistent memory as either extended DRAM or node-local storage on a job-by-job, node-by-node basis. These hardware deployments could take strong advantage of customized LabStacks for managing the node-local portion of the storage hierarchy. In that context, we explore LabStor as an approach to take best advantage of storage devices in a supercomputer in this tier. Through the use of a new filesystem and key-value store, LabStor showcases performance improvements of up to 60% under different workloads in different environments when compared with existing approaches. *LabStor is not a single I/O stack, rather it is a comprehensive platform that facilitates the rapid development and composition of new workload- and hardware-specific I/O stacks.*

2.2 Background and Related Work

In recent years, there has been a growing interest in exploring changes and improvements to the traditional UNIX I/O stack motivated by: a) hardware innovation in low-latency storage (e.g., NVMe [37] and Persistent Memory (PMEM) [73]) which demand leaner I/O stacks, and b) the increasingly diverse I/O requirements of modern workloads which necessitates high-velocity development and customization of the I/O stack.

	Heterogeneous	Interface	High-Velocity	High-Perf	Secure	Upgradable		Manageable
	Storage types supported	Modules that can be developed	Development Location	Support for kernel-bypass	Access control approach	Live upgrade support	Deployment Approach	Support for multiplexing I/O
VFS	HDD, SSD, NVMe, PMEM	FS	Kernel	No	Every I/O	No	Kernel	Yes
FUSE	HDD, SSD, NVMe, PMEM	FS	Userspace	No	Every I/O	No	Plugin	Yes
Bento	HDD, SSD, NVMe, PMEM	FS	Userspace	No	Every I/O	Yes	Plugin	Yes
Demikernel	NVMe, PMEM	Any, but IOSched	Userspace	Yes	Every I/O	No	Plugin	No
Microkernel	HDD, SSD, NVMe, PMEM	Any (FS,SQL,etc.)	Userspace	Yes	Every I/O	Yes	New kernel type	Yes
LabStor	HDD, SSD, NVMe, PMEM	Any (FS,SQL,etc.)	Userspace	Yes	Tunable	Yes	Plugin	Yes

Figure 2.1. Comparison of different I/O stack design philosophies

2.2.1 Shifting I/O from Kernel to Userspace

Traditionally, storage devices are exposed to the users via the OS kernel, which acts as an authoritative source of trust. Applications interact indirectly with storage devices using I/O system calls which involves context switching between user processes and the kernel. However, this approach generates a rigid software stack [42, 43, 50, 63, 64] that has been shown to demonstrate significant overheads [30, 35, 37, 57, 61, 69]. To ease some of the burden from the kernel, alternative approaches offer shorter paths to the hardware.

Hardware APIs: The Storage Performance Development Kit (SPDK) [57] is an ©Intel open-source project that provides a set of tools and libraries for writing userspace storage applications for NVMe devices. SPDK leverages the NVMe specification to map the PCI Base Address Register (BAR) into user applications. Similarly, LightNVM [58] is a userspace driver for OpenChannel SSDs that enables the programming of the Flash Translation Layer (FTL) and submission of I/O requests to OpenChannel SSDs entirely in userspace. DAX [56] is an I/O mechanism that allows development of PMEM I/O systems in userspace. To get the lowest latency, DAX treats PMEM devices as byte-addressable memory avoiding normal filesystem block I/O conventions. By mapping the PMEM devices into the application address space, DAX allows direct interaction with PMEM via CPU load/store operations.

2.2.2 Developing I/O Stacks in Userspace

The Linux I/O stack is a constantly evolving entity. With every new update of the kernel new changes and optimizations get included [15]. Yet, many works have discussed the limitations of the Linux I/O stack and have proposed modifications to it. For example, many I/O schedulers have been proposed to better handle specific workloads [44–48]. Some examples include Blk-switch [44] and Bulk I/O Dispatch (BID) [49]. Typically, developing the Linux I/O stack requires direct kernel modification and recompilation. This approach limits the pace of development since custom enhancements or modifications are hard to install, manage, and share. To mitigate this, the kernel allows the development of kernel modules (KM), pieces of code that can be loaded and executed inside the kernel. There are several types of KMs that allow customization of the I/O stack such as I/O schedulers, page fault handlers, filesystems, and char/block device drivers. Filesystems are implemented through the *Virtual Filesystem (VFS) Layer* and are then inserted into the kernel. Yet, kernel programming is complex, difficult to debug and limited by its complexity [15, 38, 71]. New I/O system development methodologies have been proposed that leverage the Linux I/O stack while enabling its development and customization in userspace.

Filesystems in Userspace (FUSE): FUSE [71] enables users to implement filesystems in userspace through the use of *upcalls*. When a user executes an I/O system call, the request is passed to the FUSE VFS kernel module, which then forwards the I/O request to the userspace filesystem implementation. This approach enables high-velocity filesystem development, but incurs a significant performance sacrifice due to the addition of multiple context switches [15, 38]. Works such as Direct FUSE [40] and XFUSE [38] have proposed improvements to FUSE to reduce the number of context switches. However, none of these works allow for development of alternative representations to files, alternative interfaces to POSIX, and still inherit the overheads of

the kernel I/O stack.

Injecting User Code into the Kernel: Bento [15] is a platform which enables filesystems to be developed in userspace and interpreted in the kernel. This enables the use of well-established kernel functionality such as permission checking and device drivers while also avoiding FUSE overheads. It also improves manageability (e.g., support for live upgrades). Bento makes use of a VFS kernel module that is capable of running pre-compiled Rust-based filesystem implementations submitted by the user and to which I/O operations are directed. Thus, the Bento framework drastically improves the velocity of filesystem development. However, Bento suffers the same limitations as FUSE, but with less context switching cost.

Microkernels: A microkernel, such as MINIX3 [74] or SeL4 [75], is a minimalistic kernel architecture where the majority of OS services are provided as libraries developed in userspace, including device drivers and I/O systems. This provides enhanced development speed when compared to monolithic kernels (e.g., Linux), as most programming is done outside the kernel. However, microkernels are not used in production computing centers. As of November 2021, all Top500 supercomputing centers run Linux [76]. This is because Linux has a massive community of developers and is easily installed on any machine architecture. To utilize microkernels, HPC centers would have to uproot their infrastructure and manually determine the set of modules required for their systems to run, including drivers, which is both difficult and expensive. For these reasons, maintaining compatibility with Linux is a must.

Semi-Microkernels: Various works, particularly in the networking domain [41, 77], have proposed combining the micro- and monolithic- kernel approaches, termed semi-microkernel (SMK). Applications communicate with the SMK using interprocess-communication. In these works, the SMK process runs in userspace, providing critical OS services entirely in userspace. Filesystems as Processes [78] discusses the perfor-

mance and security requirements of developing filesystems while bypassing the kernel. uFS [39], a SMK filesystem dedicates CPU resources to handle filesystem threads to optimize latency on NVMeS. To the best of our knowledge, the only work which applies this approach to I/O is uFS, which is a specific filesystem implementation, not a platform for developing and managing I/O systems. It does not discuss development of other layers of the I/O stack, developing alternative representations to files, or the deployment and upgrading of I/O systems.

Library I/O Stacks: Development of I/O stacks as loadable libraries directly linked into applications has also been presented. Demikernel [79, 79] splits I/O operations into a control path and a data path. Control operations are directed into a protected OS kernel. The data path is implemented by LibOS, an abstraction over the interface of the kernel-bypass device, and all I/O and network operations are handled by it. The same design pattern is followed by Nova [30] and SplitFS [35]. Another work, Simurgh [80], proposes a PMEM filesystem which implements all I/O functionality within a single library while protecting against buggy, but not necessarily malicious, code. Similarly to the SMK, these works primarily discuss the implementations of independent I/O systems. They are developed in isolation and can conflict when deployed simultaneously, cannot be upgraded while applications are running, are only used for storage where kernel-bypass accelerators exist, and require users to manage multiplexing concurrently-deployed library I/O systems.

2.2.3 Motivation

In summary, the following observations motivate the design and implementation of this work. First, there is a clear trade-off between I/O performance and storage features. Systems that execute I/O operations in the kernel enjoy the wide support for hardware and the provided secure enclave at the cost of performance and limited configurability. On the other hand, systems that execute I/O in userspace recover some performance and enhance customizability at the cost of limited hardware support, manual implementations of security semantics, and often troublesome deployments. Second, there has been a clear trend directed at the programmability of I/O systems. With works noting the difficulty of upgrading and debugging kernel code, leading to potential reduction of innovation and a clear increase in security issues and bugs from unknown and unpatched vulnerabilities. Finally, over the last couple decades, there is a growth in I/O interfaces deviating from the typical UNIX file abstraction (e.g., POSIX files) including key-value stores, NOSQL databases, log stores, etc. However, currently all these interfaces remain subservient to the file abstraction, requiring a translation to a file, and enforcing these systems to follow the underlying assumptions of the file interface (i.e., POSIX compliance). The translations and semantic compliance can lead to significant performance degradation.

To address these issues, LabStor presents a platform for developing I/O stacks in userspace, which provides high performance, configurability, and development velocity, while maintaining the hardware diversity, manageability and security that comes from kernel-level I/O stacks. At the same time, LabStor leverages its configurability to provide users with the ability to untether themselves from the file interfaces by implementing their own memory-to-disk data mappings. Figure 2.1 showcases a comparison of related work and their design goals.

2.3 The LabStor Platform

LabStor is an I/O platform designed to provide developers with the ability to implement highly performant, custom, and modular I/O stacks entirely in userspace. With LabStor, users can create and customize all I/O policies (e.g., I/O scheduling), utilize a diverse set of storage devices (including modern hardware such as PMEM [56] or NVMe [57, 58]), push updates and new features in the I/O stack without significant service interruption, and maintain security semantics. Most importantly, all this functionality comes without the cumbersome, often difficult, kernel programming since LabStor brings the I/O stack entirely in userspace. In addition, LabStor liberates users from the restrictive POSIX file interface, allowing alternative representations to be mapped directly to the hardware by enabling direct access to device drivers from userspace. LabStor pushes towards full modularization of the stack by introducing the following: a) the LabStor Module (LabMod): an object that contains code implementing well-defined functionality (e.g., device drivers, block I/O layer, page caches, I/O schedulers, and filesystems). b) the LabStor Stack (LabStack): a set of rules governing a composition of multiple LabMods that implement an entire I/O stack. c) the LabStor Runtime: the management and communication infrastructure to execute and deploy LabStacks.

2.3.1 Towards Fully Modular I/O Stacks

Traditional I/O stacks used in HPC have been historically developed within the kernel, which provides a well-tested, well-documented, secure and uniform interface to a wide variety of storage hardware. However, this approach followed monolithic designs that led to fragmentation in the storage space in terms of innovation, features, performance, and semantic richness. Further, monolithic designs suffer from a lack of policy decisions (e.g., I/O schedulers), limited expressiveness, minimal code re-use, and cumbersome development pipelines. Such problems are exacerbated in distributed I/O stacks where functionality and/or semantics are often duplicated. For instance, a PFS may have implemented access control while the underlying storage stack of each participating node already incorporates such semantics [29]. To alleviate some of these limitations, recent efforts [15] have demonstrated the benefit of developing I/O systems outside the boundaries of the kernel. However, these approaches impose assumptions such as the UNIX file abstraction and the strict POSIX standard, limiting expressiveness, policy decisions, and configurability. More effort is required to make it easier for developers to deliver new features fast, reliably, and portably.

LabStor, as a development platform, promotes the Single Responsibility Principle where code reusability, scalability, and speedy development is achieved by loosely coupled components, organized in smaller code bases, allowing easier debugging, faster deployments, and interoperability between I/O subsystems. To boost innovation, LabStor introduces the concept of the *LabMod*: an independent, self-contained code object implementing a well-defined, distinct, single-purpose functionality. LabMods can be independently developed, maintained, and released as plug-ins. LabMods can be interchanged, stacked, and incrementally-upgraded.

Fundamentally, LabMods are comprised of four elements: type, operation, state, and connector. The type is the set of APIs the LabMod implements. The

operation takes a well-defined input, processes the input, and produces a well-defined output. State is internal data stored within the LabMod required for the operation's success. Lastly, the connector exposes the LabMod operation to applications. Connectors and operators can live in two separate address spaces, in the same address space, or even in kernel space. For example, a client may call a POSIX LabMod and use its connector to pass a POSIX I/O request as input to a filesystem operation. The operation then converts the I/O request to one or more block I/O requests by allocating disk blocks using the allocator stored in the filesystem's state. The filesystem then outputs the block I/O requests to the next LabMod in a stack (e.g., I/O scheduler, page cache, compression).

LabMods provide developers full creative freedom over their functionality and interface. For example, LabMods can be built for filesystems, (No)SQL databases, I/O schedulers, compression, drivers, etc. However, to enable the LabMod properties of upgradeability, stackability, and interchangeability, developers must implement specific APIs. For instance, to enable live upgrades, LabMods must implement the *StateUpdate* API, which copies state from the old LabMod into the new. As another example, to capture performance counters of LabMods, developers must implement the *EstProcessingTime* and *EstTotalTime* APIs. While high development speed is important, providing developers with the ability to test, probe and debug their code in an easy manner is just as important. LabStor provides a debugging mode that allows LabMods to be run in isolation and supports existing tools such as GDB or Valgrind to fully test their individual LabMods before deploying them in production. Lastly, LabStor comes equipped with various LabMods for interacting with storage and managing the complexities of multiplexing I/O systems.

Driver LabMods: To minimize kernel programming while maintaining compatibility with a wide variety of storage, LabStor provides an enhanced, extensible alterna-

tive to Linux’s VFS and block layer for interacting with storage devices entirely in userspace. Unlike the Linux block layer, which is limited to the block device abstraction [44], LabStor provides users and developers a comprehensive menu of storage hardware APIs through the use of Driver LabMods. This includes LabMods which expose the Linux kernel’s multi-queue device driver hardware queues directly to developers, bypassing several layers of the kernel stack, in addition to various LabMods supporting userspace I/O mechanisms (e.g., SPDK, DAX), which may provide APIs other than block (e.g., zoned namespace and queues). From these fundamental LabMods, all other layers of an I/O stack (e.g., scheduling, caching, etc.,) now become programmable in userspace. Additionally, as LabStor runs primarily in userspace, LabMods can utilize well-established libraries, including data structure libraries (e.g., Boost) and machine learning libraries (e.g. mlpack [81]). This allows developers to make use of an extensive pre-existing code base that is already well tested and documented. For example, time series analysis can be used to predict characteristics of future I/O requests to reduce seek penalties on HDDs or decide which pages to evict from the page cache.

Management LabMods: One design goal of LabStor is to provide an expressive and manageable I/O platform. Users should be able to express their I/O requirements through the API that best fits their design (i.e., file, key-value, SQL). This is achieved through the concept of a Generic LabMod which exposes an I/O interface (e.g., Generic Filesystem receives POSIX calls). Generic LabMods are in charge of creating I/O requests and forwarding them to the appropriate I/O system that implements these calls. Generic LabMods are loaded into clients using *LD_PRELOAD*, enabling seamless support for legacy applications and the development of new I/O interfaces. To multiplex I/O systems without user intervention or code changes, Generic LabMods manage state that is common among I/O systems of a particular type, similar to the VFS. LabStor currently provides two Generic LabMods: a) *GenericFS*,

which manages the allocation of file descriptors and the routing of I/O requests to the proper filesystem (FS) implementation, and b) *GenericKVS*, which only does the latter for key-value stores (KVS).

2.3.2 Towards Composable I/O Services

Modern scientific computing consists of a wide variety of workloads such as simulation, data analytics, and machine learning applications, with many distinct and often conflicting I/O requirements [82]. It is unrealistic to expect a single static I/O stack to efficiently satisfy this variety of I/O requirements, including semantics and targeted performance characteristics. A software defined storage approach brings hope to a new era of performant I/O systems tailored to match the I/O demands of large-scale scientific applications. To this end, LabStor introduces *LabStacks*: a user-defined combination of compatible LabMods into a single I/O system. LabStor provides common capabilities such as device drivers, I/O schedulers, caching, and filesystems as building blocks in the form of LabMods, while introducing new and exotic ideas, such as configurable consistency or ML-driven cache eviction algorithms. Using LabStacks, users can construct customized, workload-specific, and hardware-optimized I/O services. For example, an application which generates large amounts of compressible data (e.g., VPIC [83]) to an NVMe could benefit from a LabStack which contains a compression LabMod and SPDK Driver LabMod.

LabStacks are defined in a specification file which includes the following attributes: a) a mount point, which is a human-readable path (e.g., /home/user); b) a set of governing rules, such as priority hints and execution method; and c) a DAG of LabMods, where each vertex contains the LabMod name, LabMod UUID, attributes for initialization, and a set of other outputs (e.g. other LabMods), represented as a list of LabMod UUIDs and LabStack mount points. The LabMod UUID is a human-readable name which represents a unique instance of a LabMod. The execution

method determines whether the Labstack DAG is executed synchronously (directly in the client application) or asynchronously (divided among threads, allowing stages to be pipelined).

LabStacks can be initialized on a mount point like any other I/O stack. To do so, users register their LabStacks within a LabStack Namespace by simply passing the specification file to the (overloaded) *mount* command. First, *mount* will ensure that all LabMods in the DAG are instantiated by populating a Module Registry, a key-value store where keys are LabMod UUIDs and values are the LabMod instances. A LabMod is only instantiated if its UUID did not exist in the registry. Next, the set of rules are parsed to properly configure the runtime of the LabStor platform. Lastly, a validation step is performed to verify that the LabMods are compatible and then the LabStack is inducted into the LabStack Namespace. LabStack DAGs can be modified dynamically after mounting via the *modify.stack* command, which enables the insertion/removal of vertices. LabMod instances in the Module Registry can also be hot swapped live via *modify.mods*. For example, swapping one LabMod I/O scheduler for another.

LabStacks bring many benefits into the complex space of data management, as shown in the below examples. Any I/O stack is a path (or a projection) to the contents kept in storage. As a result, one can generate multiple views of the same data by deploying several LabStacks on top of the same device. This leads to a plethora of useful features.

Tunable Access Control: I/O stacks are responsible for providing data access control mechanisms. Traditionally, this has been achieved with user groups and globally-applied permission checks. Modern research has shown the benefits of tunable access control where data is exposed based on varying degrees of sensitivity. However, this is difficult to implement in a monolithic design and is typically offered as a

non-negotiable feature. With LabStor’s modular approach, one can deploy multiple LabStacks over the same content, each with varying degrees of access. Permission LabMods inside the stack can implement islands of data that are viewable by different actors. Not only is this easily implemented, but it can also dynamically change if the operator of the I/O service chooses so.

Interface Convergence: A growing theme in recent years is the hyper-convergence between different I/O interfaces [84, 85]. Modern workloads demand the support for multiple data access interfaces which puts pressure to system developers to widen the support for diverse APIs. This has led to two distinct approaches. Either the user must adapt their code to the provided interface or translation middleware layers are employed, elongating the data path and imposing additional software overheads. In LabStor, data can be accessed by seamlessly multiplexing LabStacks. For example one with POSIX interface and another with get/put/delete over the same content.

Dynamic Semantics Imposition: Application I/O patterns can be highly variable [3, 20, 86, 87], requiring live changes to the I/O stack to maximize performance. However, this requirement is limited by the rigidity of traditional I/O stacks. LabStacks enable the ability to dynamically impose new semantics into the I/O stack by allowing LabMods to be inserted/removed while applications are still running. For instance, a compression LabMod can be added to a LabStack for a period of time, or an I/O scheduler can be swapped for another depending on the current I/O traffic.

Active Storage: Offloading data-intensive tasks to storage has been shown to benefit the end user both in terms of performance and ease-of-use [20, 83]. For example, instead of applications manually employing compression before persisting data, a LabStack could contain a Compression LabMod which transparently compresses the data before writing to storage. Compression happens asynchronously and in batches which further boosts performance.

Decentralized I/O System Designs: Various works [30,35,59] have demonstrated the benefits of eliminating centralized authorities from the I/O path for latency-sensitive I/O requests. One approach is to decouple metadata and data operations, enabling security for metadata and increased performance for data operations. In LabStor this can be done by using two separate LabStacks: one for metadata that asynchronously executes in a separate runtime, and another for data that synchronously executes at the client using Driver LabMods such as SPDK. State required for the data operations (e.g., block allocations) can be stored in shared memory between the two LabStacks. Fully decentralized designs execute both metadata and data LabStacks client-side, improving latency (but at a cost to security).

2.3.3 A Powerful Alternative to the Linux Stack

The LabStor Runtime is the primary warehouse and execution engine of LabStacks. It runs in a separate address space from the clients in order to provide security and high-performance. The Runtime is also responsible for the loading, storage, execution and upgrading of LabStacks and LabMods. Figure 2.2 demonstrates the architecture of the Runtime which consists of the following components:

Kernel Operations (KO) Manager: to achieve extensive hardware support and manage all in-kernel operations, the KO Manager is implemented as a Linux kernel module which manages the deployment of Driver LabMods and the communication with them.

Inter-Process Communication (IPC) Manager: to communicate between the Client Library, Runtime and LabMods, the IPC Manager provides a highly-performant and secure infrastructure based on shared memory and a queuing system. The LabStor client initially connects to the LabStor Runtime through a UNIX domain socket, providing process credentials to the LabStor Runtime, which can be used for authen-

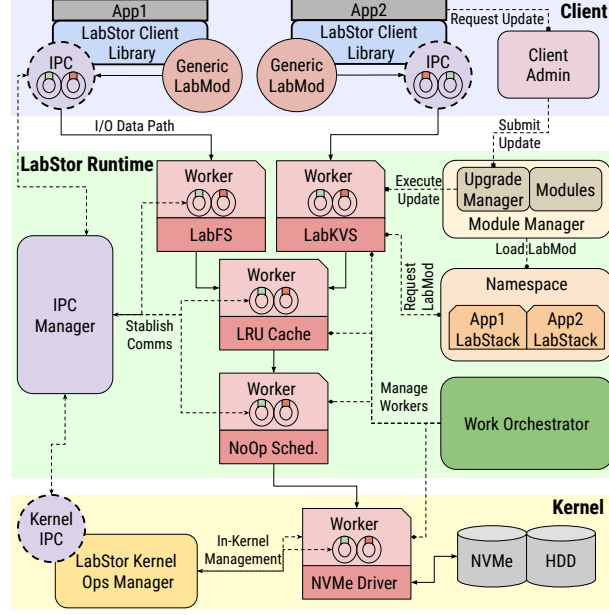


Figure 2.2. Component Diagram

tication. Similarly, the IPC Manager connects to the Kernel Operations Manager through a netlink socket.

Module Manager: to enable live upgrades, deployment, and querying of LabMods, the Module Manager maintains a Module Registry (a hashmap in shared memory) that holds all instantiated LabMods and their entrypoints. It also holds a queue, where requests for module upgrades are stored. The Module Manager implements several protocols including centralized and decentralized upgrade which are executed when invoked by the admin thread to process upgrades.

LabStack Namespace: to create, hold and multiplex LabStacks, the Namespace maintains a semantic shared-memory key-value store that holds LabStack as DAGs.

Workers: to provide security and reduce overhead caused by context switching, clients can send requests to workers which execute in a separate address space from the client. Workers receive requests by polling request queues and process the requests by querying the LabStack Namespace and Module Manager for the required

LabMods. Workers execute either in the Runtime (using pthreads), in kernel space (using kthreads), or directly in the client thread. Workers also periodically monitor LabMods to get performance metrics, useful to work orchestration policies.

Work Orchestrator: to ensure high performance and resource utilization, the Work Orchestrator manages the assignment of request queues to workers and the assignment of workers and application processes to CPU cores. In order to manage workers executing in-kernel, the orchestrator communicates with the LabStor Kernel Module to spawn, freeze, and terminate kernel threads.

2.3.3.1 Efficient and Secure Inter-LabMod Communication To enable zero-copy, high-performance communication between LabMods, LabStor relies on shared memory. However, one consequence of this design is that faulty and malicious processes have more potential to cause harm to other running processes due to the nature of shared memory. Care must be taken into providing privilege to only the processes that need visibility to the shared memory. In addition, memory needs to be shared between kernel and user space, which requires specialized memory sharing techniques that are not provided by higher-level shared-memory libraries, such as Boost.

To provide these shared memory capabilities, a novel shared memory LabMod (ShMemMod) is provided by LabStor, which uses *vmalloc* for allocating regions of shared memory and *remap_pfn_range* for mapping the allocated regions into a user's address space. The memory can only be mapped by specific processes which have been granted access to the memory by the LabStor Runtime, enabling both high-performance and security, even among processes launched by the same user.

Another communication primitive used by the IPC manager are Queue Pairs (QPs). They can be stored in shared-memory or private memory. QPs have a number of properties. *Primary queues* are the queues where clients initiate requests. *Interme-*

diate queues hold requests spawned as a result of another request. Typically, primary queues are stored in shared memory and intermediate queues are stored in private memory. In addition to this, queues can be marked as ordered or unordered. Ordered queues must be processed in sequence on a single worker. Unordered queues can be processed by multiple workers.

2.3.3.2 Live Upgradable and Hot-Pluggable LabMods To support the ability to upgrade and hot-plug LabMods without service interruption, the Module Manager implements two live upgrade protocols: centralized and decentralized. This is because LabMod operators can exist either in the Runtime’s address space or in the client’s address space. Due to space, we only detail the centralized case. The centralized protocol updates the Runtime, whereas the decentralized updates all running clients. In both cases, an upgrade request is submitted to the Module Manager using the *modify.mods*. The upgrade request contains the LabMod name to upgrade, a path to the updated code, and the upgrade type.

Centralized Upgrade: An upgrade request is initially placed in the upgrade queue in the Module Manager. The *Runtime Admin* will periodically (every t ms, configurable by the user) poll for updates. If there are any, the Module Manager will mark all of the primary queues as *UPDATE_PENDING*. Eventually, workers processing primary queues will acknowledge the *UPDATE_PENDING* state by setting the flag *UPDATE_ACKED*. Intermediate queues will continue processing until all events have been completed. When all primary queues have been paused and all intermediate requests have completed, the Module Manager will process all upgrades. For each upgrade, it will traverse every LabMod in the Module Registry and call the *StateUpdate* API for each LabMod of this type, which transfers the state from the old LabMod to the new. *StateUpdate* can be modified by developers to tailor the upgrade process to their needs. After all upgrades have been processed, the Module Manager will unmark

all the primary queues and requests will begin flowing through the system again.

2.3.3.3 Crash Recovery To provide security and resource management capabilities, LabStacks have the option to execute within the the LabStor Runtime. However, as LabStacks can contain untrusted and potentially buggy code, there is potential for the Runtime to crash. Without proper management, crashes such as this can cause severe setbacks, as potentially long-running applications would have to be restarted. To prevent this, the LabStor Runtime can be restarted and repaired while applications continue to run. To communicate with the Runtime, clients place requests in request queues and poll for completion using *Wait*. If the LabStor Runtime crashes, *Wait* will eventually detect that the Runtime is offline and wait for it to be restarted by the administrator (for a configurable period of time). If restarted, the LabStor client library in each process will iterate over the LabStack Namespace, invoke the *StateRepair* API in each LabMod, and then continue.

2.3.3.4 Load Balancing & Scheduling The Work Orchestrator (WO) is a userspace process and thread scheduling framework, similar to the concept of FlexSC [88]. FlexSC proposes a specific policy to dedicate CPU cores to handle latency-sensitive system call work to avoid context switching, whereas LabStor provides a general framework to make different policies. LabStor WOs typically should make two considerations. First, LabStacks can generate both latency-sensitive and compute-intensive requests, requiring intelligent division of labor. In addition the I/O traffic on systems can be dynamic (e.g., checkpoint-restart workloads), requiring intelligent scaling of CPU resources to provide efficient resource utilization. To manage both of these challenges, the WO defines a *rebalance* operation, which takes as input n queues and m workers. This operation is called in two cases: first, when a new client connects; and second, every t ms, configurable by the user. To decommission a worker, the WO reassigns all request queues from the worker. Workers can be oversubscribed to a

core by placing request queues on workers pinned to the same core. When a worker hasn't processed a request for a period of time (in μs , also configurable), the worker will yield to avoid busy waiting for an entire WO epoch. Load balancing queues has a long and deep history of research dating back more than 30 years, and many algorithms have been proposed (e.g., Markov Chains [89] and supermarket model [90]) for solving the *rebalance* problem. For this reason, the WO is also modular.

LabStor currently provides a *dynamic work orchestration policy*, which aims to minimize the number of workers and context switches, while keeping performance loss under a configurable threshold and optimizing for latency-sensitive requests. First, rebalance divides the queues into two groups based on the maximum expected processing time of the request (*EstProcessingTime* API in the LabMod) and the number of queued requests: latency-sensitive queues (LQs) and computational queues (CQs). The queues are then partitioned among workers, where LQs are placed on a subset of workers, and CQs on another subset. With this partitioning, a modified Knapsack problem is solved where multiple sacks (the workers) aim to fit all items (the queues) such that each sack has equal weight (total estimated processing time of the queue). The partitioning with the fewest number of workers within the threshold is selected. Workers processing latency-sensitive requests are dedicated to CPU cores in order to avoid interference caused by context switching.

2.3.4 Deployment Model

Installing LabStor: LabStor can be downloaded from github (<https://github.com/scs-lab/labstor>) and compiled/installed using CMake. After compilation, a kernel module (the LabStor Kernel Ops Manager), the LabStor Runtime, various utility commands (`mount.stack`, `modify.stack`, etc.), and a series of LabMods (libraries) are presented to the user. The kernel module is inserted into the kernel using *insmod*. After this, the Runtime needs to be started. Trusted users can modify the Runtime configuration YAML, which contains information such as LabMod locations and work orchestration policies.

Installing LabMods: Once developed, users can install LabMods into LabMod repos, which are directories searched by the Runtime to determine the set of available LabMods. Repos can be defined in the LabStor configuration file and adjusted during runtime using the *mount.repo* and *unmount.repo* commands. These commands are unprivileged and can be executed by any user. A configurable maximum number of repos per-user is defined in the LabStor configuration file. A LabMod repo which is owned by the same user as the LabStor Runtime is considered trustworthy by default. Untrusted LabMods (which could potentially contain buggy or malicious code) can still be employed and debugged; however, the execution of these LabMods must be in a separate address space from the Runtime in order to ensure security.

Mounting LabStacks: After installing the LabMod repos, users can define LabStacks. LabStacks are defined as YAML files, which contain a DAG of LabMods and a set of governing rules, including a set of users which have authority to modify the LabStack. To mount the LabStack, users can call *mount.stack*. While access control is provided primarily by the LabMod implementation, LabStor provides mechanisms to prevent untrusted users from unduly modifying or executing other LabStacks. The maximum length of a LabStack is configurable.

Modifying LabStacks and Upgrading LabMods: After mounting, a LabStack can be modified dynamically by uploading an updated YAML file of the LabStack, where the differences between the original and updated LabStacks will be applied. Code upgrades for LabMods can be specified by adding an upgrade request to the desired LabMod in the LabMod DAG. LabStack modifications and upgrades can be rejected for unprivileged users during mounting.

Application-Side: In order for applications to interact with LabStacks, LabStor provides a client library which is used to communicate with the Runtime to mount, modify, query, and execute LabStacks.

2.3.5 Use Cases

The current release of LabStor provides example LabMod implementations of both a filesystem and a key value store, respectively called LabFS and LabKVS. Developers can implement the same APIs to create their own.

An Example POSIX Filesystem: Many legacy applications depend on POSIX. For compatability, LabStor provides LabFS, a POSIX-compliant filesystem. LabFS is a log-structured, crash-consistent filesystem which provides specific optimizations for NVMe and PMEM and provenance tracking. It uses a scalable per-worker block allocator, which evenly divides device blocks among the pool of workers, initially defined in the Runtime configuration. Workers can steal from one another if more space is needed. If the number of workers decreases, free blocks of the decommissioned workers are assigned to running workers. If new workers are added, they will steal a (configurable) number of blocks from the other workers. LabFS uses a per-worker log for tracking metadata operations (file creations, etc.). As opposed to storing inodes and bitmaps on-disk as traditional FSES do, LabFS only stores the log and reconstructs inodes in-memory by traversing the log. All inodes are stored in a hashmap

to maximize scalability. In addition, LabFS can be configured to bypass the Runtime for metadata and data operations to further reduce latency.

An Example Key-Value Store: LabKVS is similarly designed to LabFS; however, LabKVS implements a put/get/remove API, which creates keys and stores data using a single syscall, as opposed to the three (open-modify-close) required by POSIX.

A real deployment use case: a custom file system stack: Figure 2.3 showcases a real use case of LabStor. In this example, a series of asynchronous LabStacks implementing full file system I/O stack are deployed on a machine. The example makes use of the GenericFS and the LabFS labMods, but the same logic could apply for other file system implementations.

Initially, various LabStacks, including “fs::/b”, have been mounted in the LabStack Namespace (shown in Figure 2.3). A client app is then launched with the GenericFS LabMod to intercept POSIX calls. The GenericFS LabMod can be LD_PRELOADED. The client first connects to the Runtime using the LabStor Client Library and IPC Manager. Eventually, the app calls *open()* to create the file “fs::/b/hi.txt”, which is intercepted by GenericFS. GenericFS determines the LabStack corresponding to this path by querying the LabStack Namespace. First, it checks if “fs::/b/hi.txt” is in the Namespace. Since it is not, it checks if the parent directory (“fs::/b”) is. Since it is (s_id: 2), GenericFS loads the LabStack DAG and allocates a file descriptor (fd). GenericFS then queries the Module Registry for the first LabMod in the DAG (LabFS) and calls its connector. The connector constructs a request containing the fd, path remainder (“/hi.txt”), LabMod ID (1), and LabStack ID (2); queries the IPC Manager for a queue pair (QP, qid: 1); places the request in the submission queue (SQ); and then synchronously waits for the request to complete by polling the completion queue (CQ). Eventually, Worker 1, which had been assigned this queue by the Work Orchestrator, polls the I/O request from the SQ.

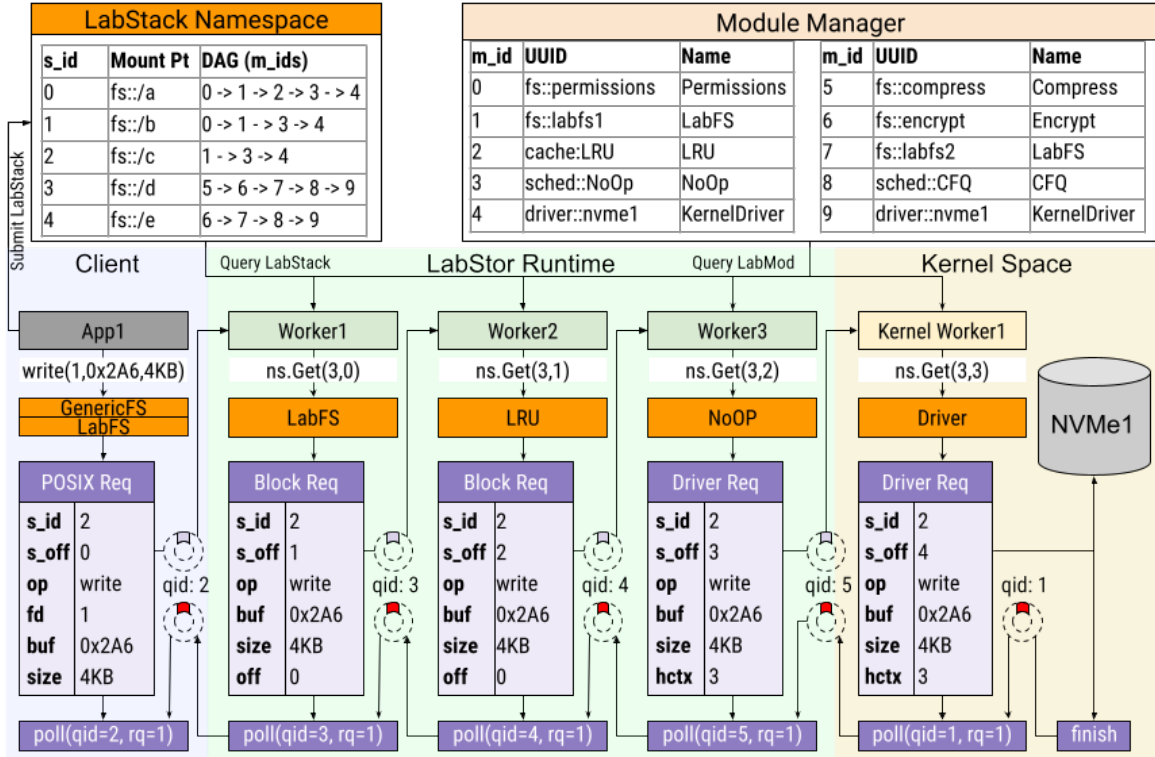


Figure 2.3. LabStack Example

The worker loads LabFS from the Module Registry and passes the I/O request and QP to its operator. The LabFS operator processes the *open*, which creates an inode and then places a completion request in the CQ. The client receives this completion request, internally stores the mapping between fd 1 and LabStack 2 in a table, and then returns the fd back to the app.

After the open, the client writes 4KB of data to fd 1. GenericFS intercepts the write, checks its internal fd table, and determines that LabStack 2 should execute the command. The LabStack is loaded from the Namespace and the first LabMod in the DAG is loaded from the Module Registry. The LabMod's connector is called, which constructs a POSIX I/O request and places it in a queue pair (QP). Within the Runtime, Worker 1 polls this request, loads LabFS from the Module Registry, and passes the I/O request to its operator. The operator then performs block allocation and forwards a block I/O request to the next stage in the DAG. This is done by querying the

LabStack Namespace for the LabStack DAG; determining the next set of LabMods to forward the request to (only the LRU LabMod in this case); constructing Block I/O requests destined for the next LabMod; loading a QP from the IPC Manager; and then placing the requests in the QP. The LabFS operator will asynchronously poll for the completion of the LRU request, allowing Worker 1 to continue processing other requests, without stalling for the LRU event to complete. This pattern of asynchronous message passing and polling continues until all events have completed.

2.3.6 Implementation Details

LabStor is implemented in 18K lines of C/C++ code and has been tested on kernel 5.4. LabStor uses pthreads to create and pin threads. LabStor comes equipped with a full I/O stack, including LabMods for page caching (LRU), tunable consistency guarantees, permissions checking, compression, I/O scheduling policies, and I/O systems (LabFS & LabKVS).

Security: Currently, our prototype enforces security by allowing only authorized root users to modify LabMods and LabStacks, similar to how mount commands are managed in Linux.

Fork/Clone/Execve: To maintain compliance with POSIX, GenericFS intercepts the `clone` and `execve` system calls. Both initialize new address spaces and require open file descriptors (`fd`) to be shared after the creation of a new address space. When called, the IPC Manager will disconnect from the LabStor Runtime, re-connect to establish new shared-memory queue pairs, and then send a message to the LabStor Runtime requesting that all file descriptors from the parent process be copied to the newly created process. `fork` is implemented on top of `clone`. For `execve`, open `fd` state is copied to the LabStor Runtime and is reloaded upon completion.

Kernel Driver LabMod: Most of Linux’s storage drivers (e.g., HDD, SATA SSD,

NVMe) follow the Multi-Queue (MQ) I/O path. LabStor exposes these drivers through three APIs: `submit_io_to_hctx`, which submits an I/O request directly to a hardware dispatch queue, `submit_io_blk`, which submits an I/O request using the standard Linux block layer, and `poll_completions`, which polls for I/O completion events on poll-based storage devices (e.g., NVMe). For MQ device drivers, `blk_mq_alloc_request_hctx` and `blk_mq_try_issue_directly` are used to place I/O requests directly in the hardware dispatch queues (hctx) of the driver. These functions are maintained by the Linux community, and have been present since kernel 4.8 in 2016 [91]. For non-MQ device drivers (e.g., PMEM drivers), we use the `submit_bio` function.

Kernel Modifications: LabStor currently re-implements the function `blk_mq_try_issue_directly` (amounting to 215 LOC) to avoid kernel recompilation, as this function is not public. This could be fixed by either making this function public or extending the `bio` structure to designate the hctx where a request should be placed. Either method would require minimal code change (roughly 5 LOC).

2.3.7 Discussion and Considerations

LabStor limitations: LabStor filesystems cannot be used for booting the main OS; only kernel-level filesystems can be used for this. The current prototype of LabStor does not provide support for networking, although it can be extended to do so. LabStor is intended to avoid compiling a custom kernel, although our code has only been tested on kernel 5.4, and may result in compilation issues on other kernels. LabStor is also designed with the assumption that the machine is many-core (> 2 cores) and has a 64-bit address space.

Re-implementation Overhead: Within LabStor, developers can choose to re-architect the entire I/O stack to achieve much higher performance than the tradi-

tional kernel stack. However, for situations where it is more desirable to rely on the already-tested policies provided by the kernel, LabMods built on top of kernel APIs such as I/O uring can be used to inherit some of the kernel’s functionality.

Applicability outside HPC: In this work, we explore the benefits of LabStor in the context HPC. However, our approach also has impacts to multi-tenant clouds and virtual machines. One future work could be to apply LabStor as an I/O hypervisor to avoid repetition of the I/O path in a VM while also maintaining isolation guarantees. Additionally, within LabStor, new I/O scheduling and work orchestration policies can be implemented, which factor the I/O behavior of running applications, service-level objectives, and core oversubscription in their scheduling decisions.

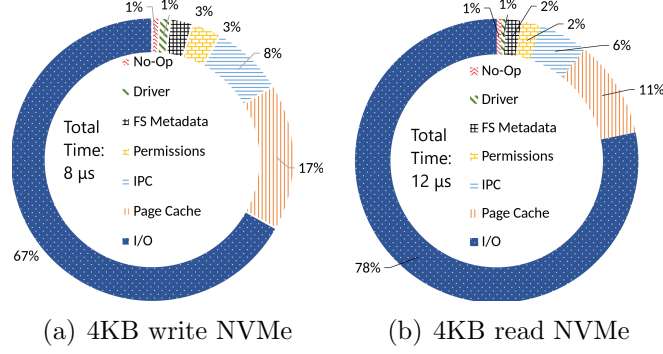


Figure 2.4. I/O stack anatomy

2.4 Evaluations

Testbed: We ran all of our experiments in Chameleon [92] using the storage hierarchy appliance, equipped with NVMe (Intel P3700; 2TB), SSD (Intel SSDSC2BX01; 1.6TB) HDD (Seagate ST600MP0005; 600GB), and 512 GiB of RAM. It contains 2xIntel(R) Xeon(R) CPU E5-2670 v3 @ 2.30GHz for a total of 24 cores and 48 threads.

Software: We use Ubuntu 20.04 with kernel 5.4. To emulate PMEM, we modify the OS bootloader [67]. For synthetic benchmarks we use FIO 3.28 and FxMark [93]. For non-synthetic evaluations, we use LABIOS [84] as an object store and FileBench-1.4.9.1 [94] for generating I/O traffic. We present the average of 5 runs. For the following evaluations, we define the following LabStacks: a) *Lab-All*: permissions checks, LRU cache, NoOp sched, Kernel.Driver, `async_exec_mode`. b) *Lab-Min*: removes permissions. c) *Lab-D*: remove permissions, `sync_exec_mode`.

Lines of Code Study: We compared the implementation of compression in LabStor against the virtual filesystem layer of the OS kernel. We found that across BTRFS, F2FS, and JFFS2, over 3K lines of code were used to develop the transparent compression in total. In LabStor, our reusable compression module for filesystems was a total of 327 LOC, a 10x reduction.

2.4.1 Internal Evaluations

I/O Stack Anatomy: To understand the impact software has on I/O operations and quantify the overheads imposed by the filesystem implementation, we run a test where we read/write 4KB of data from/to an NVMe drive using LabFS. We capture the amount of time spent in different LabMods on the data path. A LabStack, resembling that of a traditional I/O stack, is configured to use LabFS, permissions checking, No-Op I/O scheduling, LRU Page Cache, and the Kernel Driver LabMod. The LabStor Runtime uses a single worker to process the request. Overall, it can be seen from Figure 2.4(a) I/O takes the most time as expected. Software amounts to 34% of the overall time. The page cache takes 17% of time due to data copying. Next, inter-process communication using shared memory takes 8.4% of time. Since the Runtime is on a separate core, the request needs to be fetched from another core’s cache or directly from DRAM, both of which are expensive. Filesystem metadata management and permissions checking each take 3% of time due to memory allocations and data structure manipulations (block allocation, logging block in inodes, querying the inode hashmap). The No-Op I/O scheduler only amounts to about .5% of I/O time, as it only keys a request to a hardware queue. Lastly, the driver amounts to approximately 1% of I/O time. The results are similar for reads. Overall, it is apparent that for each feature added to the I/O path, a performance penalty is paid. While certain penalties (e.g., caching) can be bypassed in a typical I/O stack, certain costs are applied uniformly to all I/O operations and are non-negotiable. Maintaining security semantics, for example, requires overheads of 3% for permissions checking and 8.4% for ensuring applications cannot directly modify metadata, protecting it behind costly IPCs. Providing more flexibility on the I/O path can result in significant performance gains.

Live Upgrade: In this evaluation, we measure the amount of service interruption caused by updating a component in the LabStack during runtime. We run an applica-

Table 2.1. LabMod live upgrade performance in seconds

#Upgrades	0	256	512	1024
Centralized	29.08	30.21	32.536	34.338
Decentralized	29.08	30.51	33.56	35.81

tion which messages a dummy module 100,000 times using a single thread. Roughly 20 seconds after the app is launched, the dummy module is upgraded. We measure the total running time of the app with a centralized upgrade and with a decentralized upgrade 2.3.3.2. We vary the number of upgrade requests and report the application’s running time in seconds. From Table 2.1, it can be observed that the cost of a live upgrade was roughly 5ms which does not noticeably impact the application’s running time until thousands of upgrade requests are queued (an addition of 5 seconds after 1024 upgrades). There are three main factors which impact the performance of an upgrade: First, the storage medium the upgrade is located on; Second, the cost of loading the update into memory; Third, the cost of transferring state. In this case, the dummy module is 1MB and located on an NVMe. The I/O cost accounted for the majority of time spent in the upgrade process. The state needed to be transferred was simply a few bytes of pointers. Overall, this is a substantial improvement over a reboot per update, which in Chameleon Cloud, takes roughly 300 seconds – five orders of magnitude slower.

Work Orchestration: Dynamic CPU Allocation: In this evaluation, we measure the ability of the Work Orchestrator to dynamically adjust to changes in load. We run a workload where each client thread randomly writes 1GB of data with 4KB request sizes and vary the number of clients (between 1 and 16). The LabStack tested uses no-op scheduling with Kernel Driver LabMod over NVMe. We compare three



Figure 2.5. Dynamic Work Orchestration

worker configurations: 1 worker, 8 workers, and a dynamic number of workers. We measure IOPS and average CPU utilization. From Figure 2.5(a), it can be observed that when there are only 2 clients, a single worker is able to saturate the load without performance loss. However, after 4 clients, the single worker is overloaded and IOPS decrease by 50%. While 8 workers achieve maximum performance, it has 25% higher CPU utilization than the dynamic policy, which only requires 4 cores. At 16 clients, both the dynamic and 8-worker configurations achieve similar performance and CPU utilization. Overall, it is observed that dynamic work orchestration can balance resource utilization without sacrificing performance.

Work Orchestration: Request Partitioning: In this evaluation, we measure how queue scheduling policies impact performance. To do this, we deploy two LabStacks: latency-sensitive (L) and compressor (C). The L-LabStack contains an LRU cache, No-Op I/O scheduler, and uses the Kernel Driver LabMod as a backend. The C-LabStack adds compression. We run a metadata-intensive workload (L-App) which

creates 5,000 files per-thread over the L-LabStack, and a large I/O workload (C-App) which writes 128GB of data per-thread with request sizes of 32MB through the C-LabStack. Both the number of L-App and C-App threads are fixed at 8. We vary the number of Runtime workers to be between 1 and 8. We compare two work orchestration policies: round-robin (RR) and dynamic and measure the average latency of the L-App and bandwidth of the C-App. From Figure 2.5(b), it can be observed that the RR policy achieves the highest bandwidth in each test. For this policy, all workers are processing the C-App, since queues are evenly divided among workers. Since the processing time of L-Apps is about $3\mu s$, whereas the compression takes roughly $20ms$, bandwidth is impacted negligibly. RR also scales linearly with the number of clients, since all workers process the C-App. However, in general, the RR policy causes severe latency penalty, as the L-App must wait for multiple $20ms$ compressions before being processed. LabStor’s dynamic scheduling, however, sends the L-App queues to separate workers from the C-App queues, significantly improving latency, but at a cost to bandwidth. As the number of workers increases, the bandwidth cost drops from 30% down to 6%, since the L-Apps are all placed on the same worker. Overall, it can be concluded that the Orchestrator can balance requests with varying computational complexities fairly among a given set of resources.

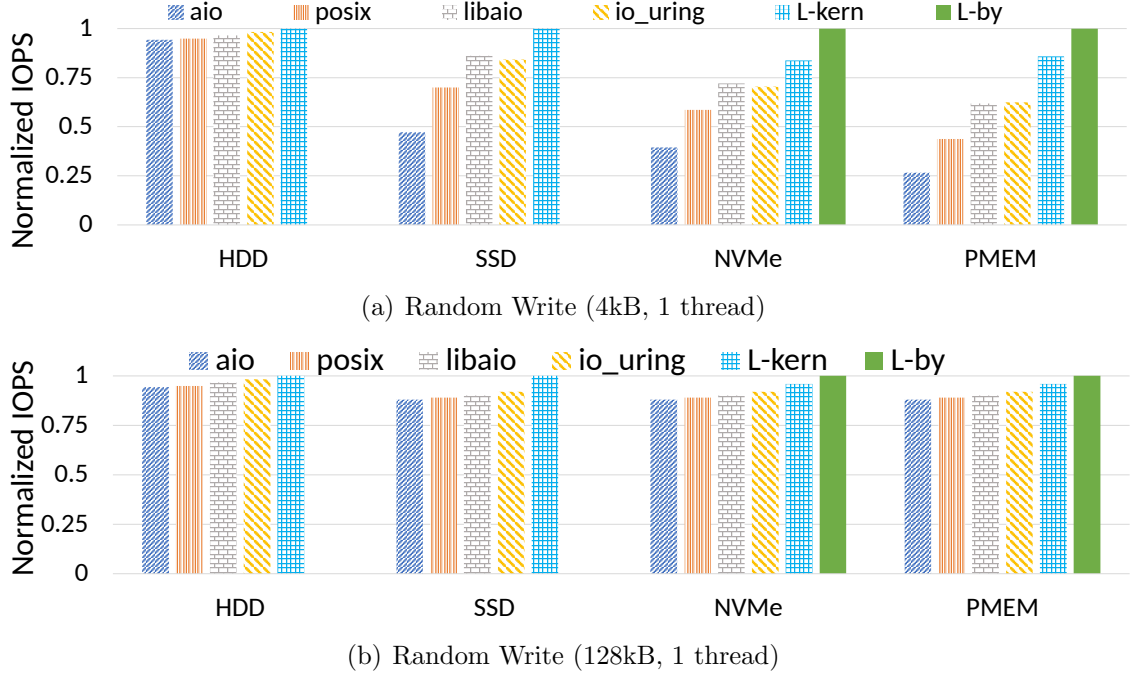


Figure 2.6. Storage API performance. IOPS are normalized.

2.4.2 Stress Tests

Storage Interface Performance: In this evaluation, we measure the performance of LabStor’s storage LabMods compared to traditional kernel-based approaches for interacting with storage in userspace. For these tests, LabStacks consisting only of DAX, SPDK or Kernel Driver LabMods are compared to using POSIX I/O, POSIX AIO, libaio, and I/O Uring to write directly to device files (e.g., `/dev/nvme0n1`). We repeat all tests for various storage hardware, including PMEM, NVMe SSD, SATA SSD and SATA HDD. We measure end-to-end application performance. We present the results on Figure 2.6. It can be observed that the I/O paths provided by Labstor outperform existing solutions significantly on low-latency device types (SSD, NVMe, PMEM). Unlike POSIX syscalls, LabStor uses shared-memory queues as opposed to context switches, which avoids software overhead and is friendlier to CPU caches. POSIX AIO suffers additional overhead due to the cost of context switching to the AIO thread, amounting up to 60-70% overhead on NVMe and PMEM. libaio and

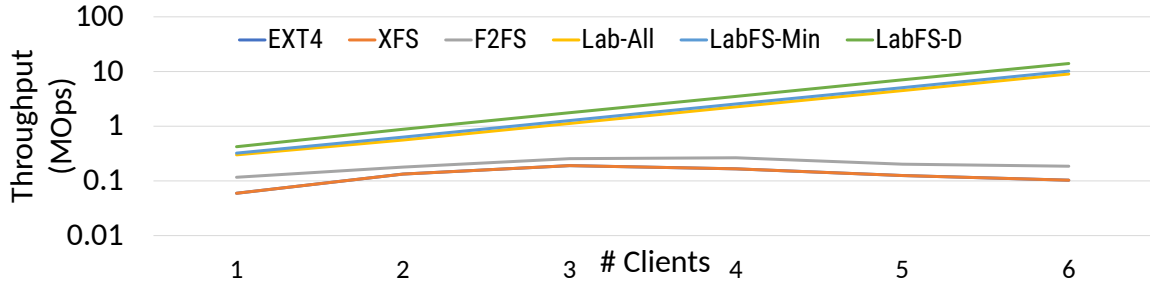


Figure 2.7. Metadata throughput

I/O Uring obtain significantly improved performance over typical POSIX syscalls, as they avoid context switching. However, for 4KB I/Os, the Kernel Driver LabMod outperforms all other solutions by at least 15% by avoiding context switches and data copies on NVMe. The SPDK LabMod outperforms the Kernel Driver LabMod by an additional 12% on NVMe by avoiding the complex allocation of structures required by the Kernel Driver. As the request size increases, the benefits of the reduced software overhead decrease. By the 128KB size, performance difference between SPDK and POSIX is roughly 6%. Overall, it is apparent that different interfaces to storage can have significant impacts on performance depending on the workload and device type. Simply relying on POSIX to build storage systems and incurring the costs of the kernel’s stack can yield dramatic latency costs of up to 60%.

Metadata Throughput: In this evaluation, we measure the overhead of various LabStor configurations on metadata performance: a common workload in various HPC applications [95–97]. We compare three configurations of LabFS to various I/O systems (EXT4, XFS, F2FS) on workloads which stress file creation using Fx-Mark [93]. We vary the number of client threads to be between 1 and 24. The LabStor Runtime is configured with 16 workers. From Figure 2.7, it can be seen that all LabFS configs outperform the alternatives by up to 3x in the single-threaded case while also maintaining scalability in each of the scenarios. For the single-thread case, LabFS-All primarily outperforms the alternatives due to a reduction in context

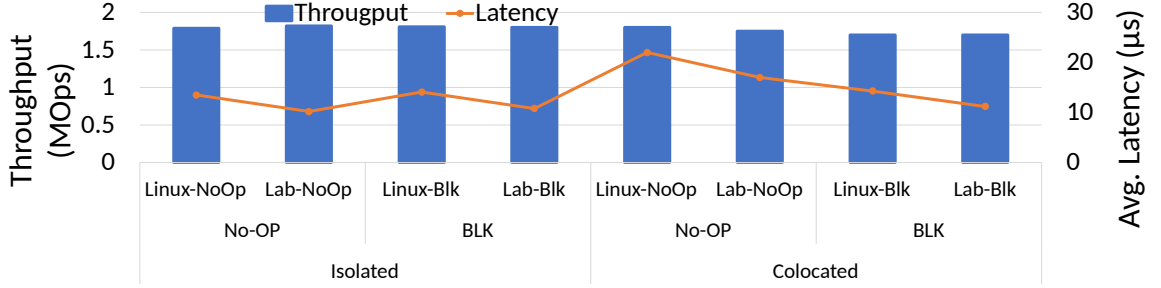


Figure 2.8. Performance of different I/O Schedulers

switches caused by syscalls. When the permissions LabMod is removed from the LabStack, performance improves by an additional 7%. Finally, removing the centralized authority entirely improves performance by an additional 20% in the single-threaded case by removing IPCs. As the number of clients increases, LabFS scales well since LabFS stores all files in a single hashmap, which supports insert, rename, and delete operations with minimal contention and overhead. LabFS’s block allocator divides blocks among workers, also minimizing contention. However, the kernel filesystems scale very poorly, as they use locking in order to ensure the correctness of their data structures. Overall, simply relying on kernel filesystems and the VFS as backends for metadata-sensitive workloads can result in severe performance penalties due to software overheads and locking. New high-performance I/O systems can be built within LabStor using its lightweight IPC mechanisms and scalably managed by its Runtime. In addition, by giving end-users more choice of what functionality (e.g., permissions) is truly required by their I/O systems, software overhead can be further reduced.

Developing & Customizing I/O Policies: In this evaluation, we demonstrate the power of the LabStor platform to develop low-level I/O policy decisions while maintaining high performance. To do this, we integrate the No-Op and blk-switch [44] I/O schedulers into LabStor and compare against their in-kernel counterparts. No-op maps I/O requests to device queues based on the CPU core the request originated. Blk-switch takes into consideration the load emplaced on a queue. We deploy two

applications: throughput-bound (T-App) and latency-bound (L-App). The T-App produces 64KB random writes per-thread with an I/O depth of 32. The L-App produces 4KB random writes per-thread (I/O depth 1). Both apps run for a period of 1 minute. Both the T-App and L-App have 8 threads, and the LabStor Runtime is configured to have 8 workers. We measure average and P99 latency when the L-Apps and T-Apps are isolated and colocated.

Table 2.2. P99 Latency of Linux and LabStor schedulers

		Isolated	Colocated
NoOp	Linux-NoOp	110 μs	945 μs
	Lab-NoOp	89 μs	889 μs
Blk	Linux-Blk	120 μs	122 μs
	Lab-Blk	95 μs	96 μs

Figure 2.8 shows that when the processes are isolated, No-Op performs at least as well as blk-switch since T-Apps and L-Apps are mapped to separate queues. This style of workload is similar to HPC storage nodes, where the node is dedicated to processing I/O requests from a single application (the parallel filesystem). However, after colocating, the performance and QoS of the L-Apps degrade due to head-of-line blocking. This style of workload is more similar to that of a multi-tenant environment (e.g., Cloud or HPC compute nodes), where many conflicting applications may be running. However, by avoiding context switching and upper layers of the I/O stack, LabStor decreases latency by 20% over the kernel version of blk-switch. In addition, LabStor’s No-Op reduces latency by an additional 5% when the apps are isolated. Overall, low-level policy decisions can be developed and customized in LabStor with minimal overhead, and the choice of policy can have noticeable performance impacts depending on the workload.

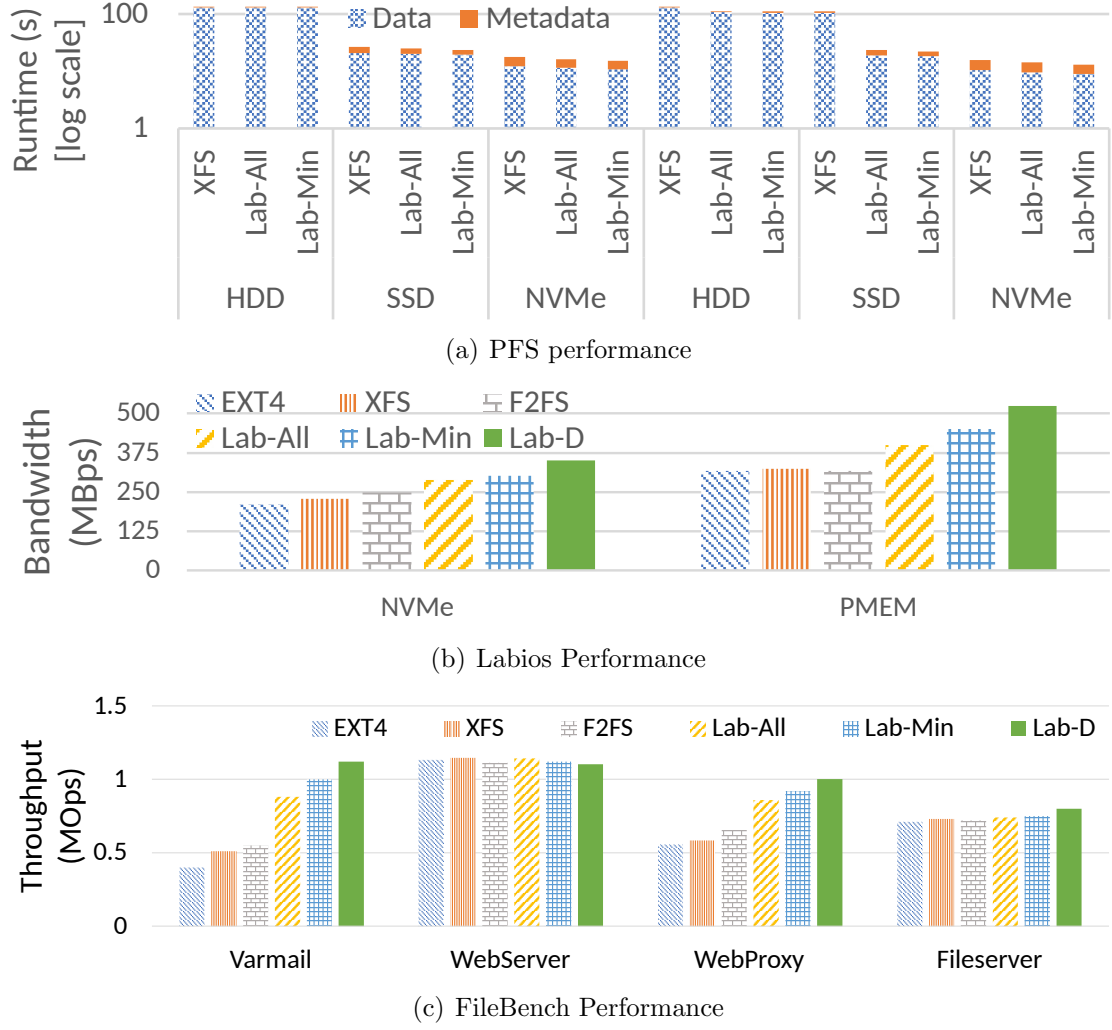


Figure 2.9. LabStor under different storage systems

2.4.3 End-to-end Performance with Real Workloads

In these evaluations, we demonstrate the end-to-end performance of different LabStor configurations for various real workloads.

PFS: In this evaluation, we demonstrate the benefit of deploying a PFS over customized LabStacks. We use OrangeFS with the metadata server deployed separately from the data servers and with a stripe size of 64KB. The metadata server runs on a node with NVMe SSDs. We run two workloads: VPIC [98] and BD-CATS [99]. VPIC is a particle simulation code where each process produces particle data and

writes them at each time step. VPIC writes 8 million particles where each particle is a vector of 8 floating point values. We run VPIC with 640 processes in 16 time steps, totaling 165GB of data. BD-CATS reads the data generated by VPIC to perform a parallel clustering algorithm. BD-CATS also uses 640 processes.

From Figure 2.9(a), it is shown that a PFS gains performance improvements of 6-12% when deployed over a customized I/O stack. These benefits primarily come from the increased throughput of the metadata server, which manages the location of stripes. Overall, roughly 100 million metadata operations are made to the metadata server in both workloads, amounting to roughly 4-6 seconds of execution time. The remaining time is spent in I/O and network. The improved metadata performance stems from using kernel-bypass I/O (LabFS-All) and by minimizing permissions checking overheads (LabFS-Min). When the data storage servers are backed by HDDs, the benefit of improved metadata performance is overshadowed by the cost of I/O. However, as the overhead of I/O decreases by switching to SSDs and NVMeS, performance benefits are more noticeable.

Distributed Object Store: In this evaluation, we demonstrate the benefit of customizing I/O interfaces on a state-of-the-art distributed I/O system, LABIOS [84], which bridges the gap between Cloud and HPC workloads through the use of a new data representation – the label. In this test, we measured the I/O bandwidth and throughput of LABIOS Workers, which are responsible for storing and retrieving data. We perform a workload which triggers LABIOS to generate 8KB I/Os. Typically, LABIOS stores labels by translating them to a UNIX file which is written on the disk by POSIX I/O. Each label write triggers a sequence of POSIX calls (`fopen()`-`fseek()`-`fwrite()`-`fclose()`). This behavior is common among distributed NoSQL DBs, Document Stores, and Key-Value Stores. We compare the performance of using various backends including LabKVS for the workload. We ran this experiment over

several types of drives but we present only the NVMe results since it eliminates the differences of the medium and highlights the impact of the stacks used. We avoided presenting the results HDDs since no performance was gained due to seek penalties.

From Figure 2.9(b), it can be seen that filesystem performance degrades by at least 12% when compared to LabKVS on NVMe and PMEM. This is because filesystems impose the POSIX abstraction over storage, which does not translate well to a key-value store workload, requiring many syscalls in order to place data. As opposed to the typical open-modify-close procedure required by filesystems, LabKVS simply performs put/get, which reduces the number of syscalls from 4 down to 1. In addition, by relaxing the access control guarantees of LabKVS, up to an additional 16% performance is gained. Overall, simply relying on the age-old POSIX API limits the expressiveness of I/O systems, requiring them to comply with awkward file translations and increased software overhead.

Cloud Workloads: Filebench contains four workloads: varmail, webserver, webproxy, and fileserver. We use the default [94] configurations of filebench for the workloads and run over NVMe and PMEM. The results of the NVMe evaluation can be seen on Figure 2.9. The PMEM experiments return identical trends as those executed over NVMe. In most cases, LabStacks containing LabFS perform markedly better than the alternatives (up to 2.5x throughput) by reducing context switching and the I/O path length. The main exception is fileserver, which performs many large I/Os and is thus dominated by I/O time.

2.5 Conclusion

In this work, we address the limitations of developing I/O stacks within the confines of monolithic kernels, such as Linux. We present LabStor: a modular, extensible, and high-performance I/O platform. We showcase how modularity can be leveraged to provide expressive, customizable and high-velocity I/O stacks. We leverage this modularity to build composable I/O stacks that provide users with the ability to upgrade and manage their I/O stacks in an easy and efficient manner. Finally, we showcase how utilizing both in-kernel and userspace drivers can help tie these ideas together into a functional runtime environment. Experimental results demonstrate that customized LabStor I/O stacks can yield performance gains of up to 60% over alternatives under various workloads and storage devices.

CHAPTER 3

CHIMAERA: A DISTRIBUTED SEMI-MICROKERNEL FOR COMPOSING
HETEROGENEOUS DATA SERVICES

3.1 Introduction

Large-scale, data-intensive workloads in HPC and Cloud, such as scientific simulation [100], data analytics [99], deep learning [86], and web servers [94], have unique I/O requirements that no single I/O stack and hardware configuration can comprehensively address. This has led to the development of complex machine architectures with highly heterogeneous, disaggregated storage and processing hardware. Modern storage accelerators (e.g., NVMe, Phase Change Memory, and Compute Express Link) have been adopted in several HPC and Cloud sites due to their advances in terms of capacity, bandwidth and latency, offering performance within an order of magnitude of DRAM while providing high density [101]. ORNL’s Summit [5], Lawrence Livermore’s El Capitan [6], and Google Cloud [8], for example, offer machines with high-bandwidth, low-latency, node-local NVMe, whereas Argonne’s Aurora [7] machine contains both NVMe and 3D XPoint modules. Meanwhile, SmartNICs [9–11] and computational storage devices [12, 13] are being investigated to offload OS functionality from server CPUs to reduce contention with running applications and isolate

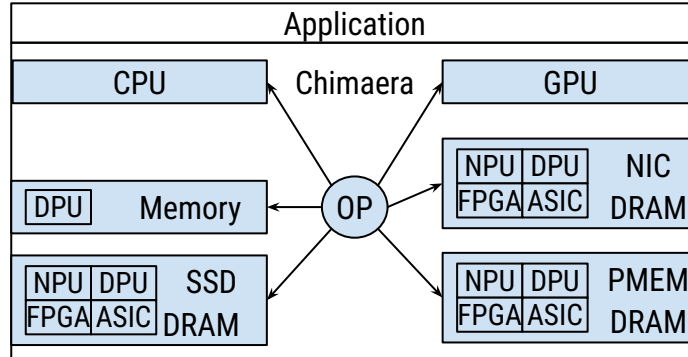


Figure 3.1. Depiction of Chimaera for scheduling an application operation (OP) on heterogeneous hardware

hardware failures to avoid machine downtime. With the emerging Compute Express Link (CXL) protocol, the memory and storage embedded within each of these processors will be coherently and efficiently exposed to one another through a non-uniform memory access API [102]. While these hardware innovations offer great potential to improve performance and reliability, leveraging them to meet diverse, oftentimes conflicting application I/O requirements remains an ongoing challenge.

To manage this complex, heterogeneous landscape, specialized I/O stacks are being developed to handle particular workloads. For example, scientific simulation codes are oftentimes divided into synchronous, alternating phases, where large-scale computations are performed and then persisted in intense, coordinated I/O stages [103], leading to the development of parallel filesystems [16, 104] and multi-tiered I/O buffering systems [3, 105, 106]. Alternatively, web servers oftentimes perform small, synchronous I/O operations and require extremely low latency [107], leading to various hardware-accelerated solutions, such as distributed PMEM databases that execute queries transactions on SmartNICs [108, 109]. There has also been a growing interest in near-data computation and in-transit analytics, where statistics and transformations are applied transparently as a functionality of the I/O stack [19, 83, 84], leading to solutions such as SmartNIC-accelerated streaming systems [11, 110]. While specialized I/O stacks offer great performance and resource utilization benefits in the cases they were designed for, this individualistic design philosophy leads to several concerns.

First, I/O stacks are developed with a rigid set of tightly coupled features, leading to costly redevelopment efforts and lack of extensibility [1, 14]. For example, several kernel-level filesystems such as BTRFS and F2FS offer transparent encryption and compression capabilities, but repeat nearly the same code logic across systems, resulting in thousands of lines of repeated code [1]. In addi-

tion, many I/O stack solutions do not provide the ability to upgrade code and release features dynamically [16–18], requiring significant service interruption to evolve [15]. Meanwhile, there is growing interest in the scientific domain for near-data processing [19, 20], where complex, multi-variate, workload-specific operators such as partial differential equations are derived transparently in the I/O stack to reduce data movements, requiring the ability to customly develop and deploy in-transit data operators. I/O stacks should follow an evolutionary design, where code can be easily reused, customly developed, and carefully selected.

Second, I/O stacks are developed to offload manually-selected operations to a particular processor (e.g. CPU, SmartNIC), but not across processors. Oftentimes, this offloading is non-negotiable [10, 12, 110, 111]. However, this approach can lead to unportable/hardware-dependent designs, suboptimal resource utilization, contention, and time-consuming data movements caused by NUMA. For example, compression is an expensive operation. In a highly parallel, coordinated scientific workflow where I/O sizes are large, always offloading in-transit compression operations can result in overutilization when the accelerator has lower core frequencies, core counts, and memory than the CPU [10]. Alternatively, never offloading compression can result in contention with application threads, also resulting in performance degradation due to phenomena such as the straggler effect [22]. I/O stack operations should be intelligently orchestrated and migrated across heterogeneous processors with consideration to portability, computational complexity, scalability, NUMA, and current utilization.

Lastly, in multi-tenant situations such as hyperconverged HPC and Cloud platforms, I/O stacks designed for specific workloads result in suboptimal resource utilization and contention when executing simultaneously [23, 24]. For example, a latency-sensitive application (e.g., web server) ex-

executing on the same machine as a bandwidth-intense application (e.g., scientific simulation) can result in severe I/O and network contention [10, 25], causing high latencies, poor quality-of-service, and bandwidth starvation without careful mediation. Alternatively, two I/O stacks that both perform in-transit computations can result in contention and poor resource utilization when scheduled on the same processor [26]. I/O stacks must be intelligently scheduled and monitored to minimize I/O, network, and processor contention across applications with varying computational and I/O patterns.

In this work, we propose the design and implementation of Chimeara: a distributed semi-microkernel for composing specialized, multi-tenant data services that can be automatically scheduled and migrated across heterogeneous processing resources. Chimeara provides the foundations necessary to customly develop important storage features (e.g., reliability and compression) to address a variety of workloads and accelerators while automatically deciding and adjusting where to execute them based on availability and resource utilization. The contributions of this work are as follows:

1. A heterogeneous, flexible, and upgradeable **ChiMod**, which improves development velocity by allowing single-purpose, data-centric, custom code objects (e.g., data transformations, device drivers) to be developed and dynamically deployed across processors.
2. A malleable, migrateable, and recoverable **ChiContainer**, which improves resource utilization and reliability by allowing state and computations to be shifted across processors and machines
3. A heterogeneous, distributed, and multi-tenant **ChiOrchestrator**, which improves resource utilization and reduces contention by automatically scheduling

Title	Accelerator	Processor	I/O Interface	Offload Calculations	Distributed	Vertical Scheduling	Horizontal Scheduling	Multi Tenant
PolarDB	CSD	FPGA	SQL	SQL Queries, Compression	Yes	No	No	No
DevFS	CSD	DPU	Filesystem	Filesystem	No	No	No	No
DockerSSD	CSD	FPGA	Mem-only	Network Func, Filesystem	Yes	No	No	No
SmartSSD	CSD	FPGA	SQL	Spark SQL	Yes	No	No	No
E3	SmartNIC	DPU	E3 collections	Microservices	Yes	No	Yes	Yes
LineFS	SmartNIC	DPU	Filesystem	Filesystem, Compression	Yes	No	No	No
MinOS	SmartNIC	DPU	Filesystem	Filesystem	Yes	No	No	No
SmartShuffle	SmartNIC	DPU	Streaming	MapReduce Shuffle	Yes	Yes	No	No
WindFlow	SmartNIC	DPU	Streaming	Network Traffic Analysis	Yes	No	No	No
Sirius	SmartNIC	DPU	Mem-only	Network Func (e.g., firewall)	Yes	No	Yes	No
SuperNIC	SmartNIC	FPGA	Mem-only	Network Func (e.g., firewall)	Yes	Yes	No	Yes
UpPipe	PIM	DPU	Mem-only	RNA sequences	No	No	No	No
Chimaera	Any	Any	Extensible	Any task supporting hardware	Yes	Yes	Yes	Yes

Figure 3.2. Comparison of heterogeneous I/O stacks. Horizontal means moving state + computations across processors or nodes. Vertical means moving computations across processors within a single machine depending on current load.

compatible ChiMods to processors based on QoS objectives and active monitoring.

4. An example high-performance I/O stack **ChiFS** with support for active, cross-accelerator data analytics, derivation, and transformations, demonstrating the benefits and limitations of accelerator offloading for I/O stack and application operations

3.2 Background and Related Work

The architecture of modern HPC and Cloud machines is becoming increasingly complex with the introduction of heterogeneous storage and processing resources. Developing I/O stacks that leverage these highly diverse technologies to accelerate applications is an ongoing challenge, with many works proposing specific techniques for each one.

3.2.1 Heterogeneous, Disaggregated Machine Architectures

3.2.1.1 General-Purpose Processors Processors with general instruction sets have emerged to accelerate data operations and offload work from expensive server CPUs, referred to as Data Processing Units (DPUs). These accelerators are being attached directly on hardware devices to promote near-data processing and improve reliability by isolating faults. Applications running on a DPU can continue to run even in the event of OS failures on the host CPU. Two examples include **computational storage devices (CSDs)** and **SmartNICs**. For example, a Mellanox BlueField P-Series SmartNIC provides a DPU [112] with 16 Cores ARM A72 cores (800MHz), Dual 100GbE/EDR VPI Ports, PCIe Gen4.0 x32, and an accelerator for encryption operations. These SmartNICs are fully programmable and host a Linux-based OS, enabling complex programs such as filesystems to be executed partially or entirely outside of the CPU. Typically, these DPUs have lower core counts and frequencies to reduce energy consumption and cost [113]. For example, Frontier, the currently fastest supercomputer according to Top500 [114], has 4x the CPU core count (64) and 3x core frequency (2GHz) compared to the BlueField. Careful consideration must be made to the computational requirements of programs running on the DPUs to ensure performance.

3.2.1.2 Special-Purpose Processors Several processors have also been developed that are tailored to execute particular types of programs efficiently, including Graphics Processing Units (GPUs), Processing-in-Memory (PIM) chips, Field Programmable Gate Arrays (FPGAs), Application-Specific Integrated Circuits (ASICs), and Neural Processing Units (NPUs). These processors follow a push-based model, where applications running on a general-purpose processor send tasks to them as part of a broader program.

3.2.1.3 Resource Disaggregation HPC and Cloud systems are moving towards disaggregated designs. Remote Direct Memory Access (RDMA) allows for an application to access memory of a remote machine without CPU involvement using processors on a NIC. Compute Express Link (CXL) [115] is emerging that will allow the memory and storage of all hardware devices, including accelerators (e.g., GPUs), to be accessed directly between each other using Nonuniform Memory Access (NUMA) APIs such as POSIX mmap and RDMA [116], effectively turning modern machines into nonuniform single-memory systems with nonuniform processing cores.

3.2.2 Specialized I/O Stacks

3.2.2.1 Hardware-Specific I/O Stacks HPC and Cloud sites are moving towards accelerators to offload I/O-related tasks from the CPU to improve energy efficiency, cost, and overall application performance [113]. To accomplish this, I/O stacks are being developed for particular accelerator technologies [9–12, 108, 110, 111, 113, 117–120]. DevFS [12] describes a single-node PMEM filesystem that executes on processors coupled with the PMEM device itself, offloading metadata management, crash consistency, and security to the storage device. LineFS [10] describes a distributed SmartNIC PMEM filesystem which additionally offloads networking operations, replication, and compression. Alternatively, works such as SmartSSD [110] and SmartShuffle [11] move data streaming pipelines to SmartNICs and CSDs, allowing stream-specific computations to occur as background tasks instead directly on the host CPU. However, these works are specialized solutions designed to solve particular problem types over particular hardware types and come with a rigid set of tightly coupled features. They do not explore extensible and evolving I/O stack designs. In addition, they focus on offloading tasks to a particular accelerator type, such as SmartNICs, and do not currently consider heterogeneous machines containing various processing and storage hardware. Lastly, these works have limited consideration to cross-application

I/O- and compute- interference in multi-tenant deployments. They investigate only relieving contention from the CPU, but not on the accelerators themselves.

3.2.2.2 Flexible I/O Stacks Work has been done to explore I/O stacks that can be adjusted to particular workload and hardware types. FUSE [121] moves filesystem development from the kernel to userspace, improving development pace – though at the expense of high latency due to context switching [38]. Alternatively, LabStor [1] proposes a composable, extensible, and upgradeable single-node I/O stack development platform for I/O systems in general (i.e., not just filesystems), enabling workload-specific I/O paths and I/O modules (e.g., compression) to be built and reducing overhead through kernel-bypass and queuing. Labios [84] is a distributed label-based I/O system that proposes a flexible storage stack design that maps conflicting data representations into a unified label representation that encompasses the data and unary operations (e.g., compression) to perform on that data. While these works follow a semi-microkernel design, have some ability to customize I/O paths, and offer portability across storage hardware, they do not explore multi-tenancy, processor heterogeneity, and dynamic load balancing.

3.2.2.3 Elastic I/O Stacks Load balancing algorithms are applied in many data-centric applications to improve resource utilization and reliability [122–125]. Container orchestration frameworks, such as Kubernetes [126], are often used to make reliable and elastic web services [127, 128]. Kubernetes provides the concept of pods (e.g., website) representing a related group of containers (e.g., a database query engine). The number of pods can be scaled depending on factors such as CPU utilization and dynamic node additions to the cluster. Fundamentally, pods rely on an underlying shared storage system (e.g., distributed database) to persist data from the containers, which simplifies algorithms for crash recovery, scaling, and upgrading. While some work has been done to use these frameworks to load balance I/O systems [129] such as CephFS [130], the I/O system itself is developed outside of the framework. In addition, the scaling algorithms are agnostic to heterogeneous accelerators and mostly rely on OS process schedulers for mediating multi-tenant contention.

3.2.2.4 Background Summary Machine architectures are becoming increasingly complex and diverse. However, I/O stacks are designed in isolation to solve particular types of problems and confined to particular machine designs. This has led to significant redevelopment efforts, fragmentation of storage features, lack of hardware support, and suboptimal multi-tenant coordination. To our knowledge, this is the first work to propose an I/O stack development platform that intelligently schedules arbitrary I/O stack operations across non-uniform, disaggregated accelerators in a distributed environment for a variety of workloads.

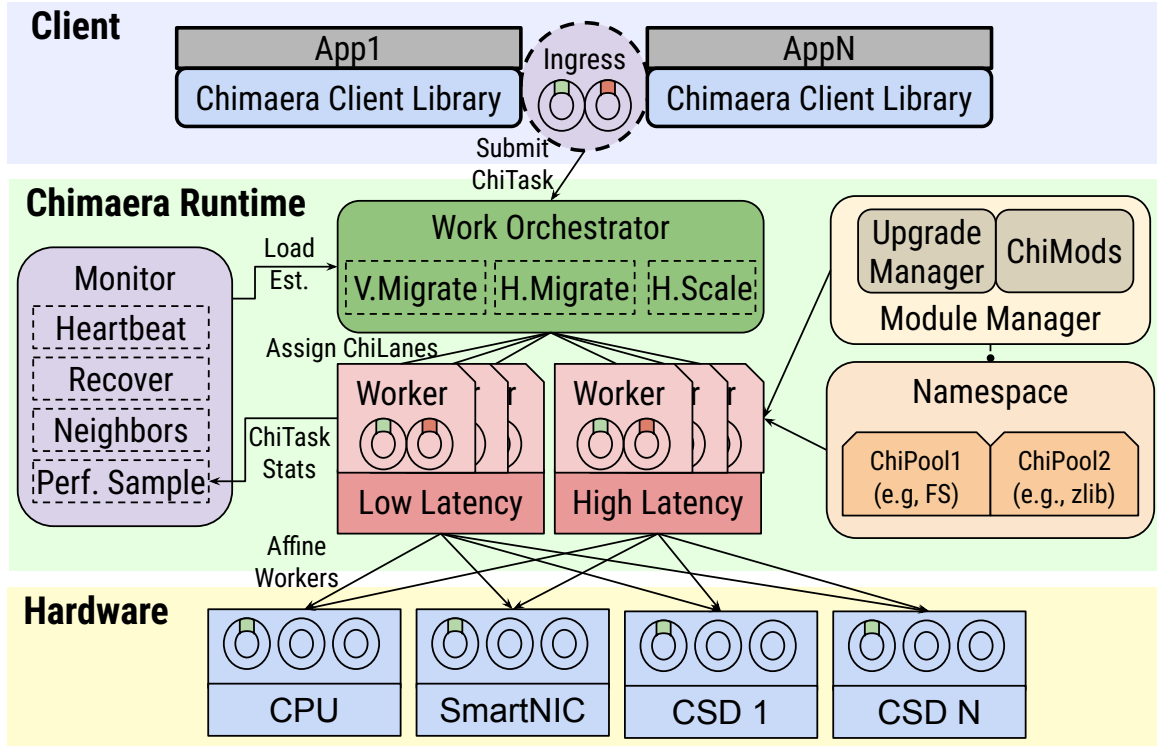


Figure 3.3. High-level architecture of Chimaera.

3.3 Design

Chimaera is a distributed semi-microkernel designed to support a wide variety of conflicting I/O workloads simultaneously while automatically leveraging heterogeneous processing resources to maintain high resource utilization and performance. With Chimaera, I/O features (e.g., data transformations, I/O schedulers) can be customly developed, dynamically uploaded, and incrementally upgraded without significant service interruption through the concept of **ChiMods**. ChiMods natively provide callbacks required to implement reliable, high-performance distributed systems and can be developed for a variety of processor and storage types, encompass a broad range of functionality, and composed to form specialized I/O paths to satisfy particular application I/O requirements. Several I/O paths can be defined simultaneously and carefully orchestrated by the distributed **ChiOrchestrator** to avoid network, processor, and I/O interference while satisfying workload-specific I/O requirements.

To ensure high resource utilization, I/O stack operations are intelligently scheduled and migrated across heterogeneous processing resources by dynamically learning the I/O and computational complexity of ChiMods while considering current resource utilization. The high-level architecture of Chimaera is depicted in Figure 3.3. Chimaera is designed with the following objectives in mind:

1. **Flexible:** Chimaera should enable customization of the I/O path to satisfy diverse workload types.
2. **Programmable:** Chimaera should support the ability to develop and install custom feature additions.
3. **Evolutionary:** Chimaera should support the ability to upgrade code without significant service interruption.
4. **Heterogeneous:** Chimaera should support heterogeneous processing and storage resources.
5. **Reliable:** Chimaera should support the ability to respond to hardware and node failures.
6. **Orchestrated:** Chimaera should coordinate the scheduling of I/O stacks to processors to minimize contention.

3.3.1 Flexible, Programmable, and Evolutionary I/O Stacks

I/O stacks are developed in isolation to attain high performance and resource utilization for particular problem types and hardware configurations. However, this individualistic approach has led to a fragmentation in the storage space in terms of innovation, features, hardware compatability, and semantic richness. These I/O stacks impose a tightly coupled, rigid set of features on every I/O operation and

Method	Input	Output	Description
Run	u32 method ChiTask *task	-	Run a method (e.g., Create) by ID
Create	CreateTask *task	ChiContainer *new	Constructs + initializes a state
Destroy	DestroyTask *task	-	Destroys state
Upgrade	UpgradeTask *task ChiContainer *old	ChiContainer *new	Upgrade ChiMod to new version
Rescale	RescaleTask *task vec<DomainId> nodes bool expand	ChiContainer **new ChiContainer **migrate	Load balancing when nodes are added or removed
Recover	RecoverTask *task, DomainId dead bool deep	ChiContainer *new	Resolve inconsistencies and deeply recover dead state from crashes or hardware failure
(Horizontal) Migrate	MigrateTask *task DomainId to	ChiTask **subtasks	Move state to some other processor
ReinforceStats	ChiTask *task, double cpu_time, double runtime	-	Reinforce model predicting task time
PredictStats	ChiTask *task DomainId on	double cpu_time size_t io_size size_t memory	Predict task cpu time, io size, and memory requirement given processor (on)

Figure 3.4. Basic ChiMod API

are oftentimes confined to particular machine architectures, which limits code reuse, extensibility, portability, and performance. For example, HPC applications require a POSIX file interface, but do not always require strong consistency [131, 132] – a feature that limits the scalability of these applications [84] and is forced by parallel filesystems [16, 104]. Alternatively, scientific applications moving towards near-data processing designs require the ability to execute custom mathematical operators transparently to minimize data movements [19]. However, without considering heterogeneous processing resources such as GPUs and SmartNICs, these operators can cause significant contention with application threads. Chimaera proposes a paradigm shift towards a composable, evolutionary design philosophy that promotes single responsibility, heterogeneity, and loose coupling, which enables code reuse, dynamic feature imposition, live upgrades, cross-processor scheduling, and customization of entire distributed I/O stacks.

3.3.1.1 Programming Diverse, Heterogeneous Features through ChiMods

I/O stacks have traditionally been developed in isolation for particular problem types and architectures, resulting in rigid, tightly-coupled, hardware-dependent designs. To foster code reuse, promote active storage, and support portability across machine designs, Chimaera introduces the concept of **ChiMods**, which are independent, single-purpose code objects that represent a single state and a set of related operations to perform in the context of that state. For example, a ChiMod for an in-memory database (the state) may define and implement *DBcreate*, *DBread*, *DBupdate*, and *DBdelete* operations. Developers have full creative freedom over ChiMods and can represent a broad range of functionality, including filesystem metadata management, replication, compression, and transparent data analytics. ChiMods can contain multiple implementations and be compiled for multiple processor types, such as SmartNICs, PIMs, GPUs, FPGAs, and general-purpose CPUs. For example, a ChiMod built for KMeans clustering could have implementations in both C++ and Cuda to enable

processing on CPUs, DPUs, and GPUs. In this way, a scheduling entity can intelligently select an implementation through heterogeneous load balancing techniques to improve resource utilization and performance. Reliability is also improved as there are backup options in the case where an accelerator fails. To build a ChiMod, the API shown in Figure 3.4 must be implemented. This API incorporates the necessary callbacks required to build reliable, high-performance data services. This includes APIs for migrating state to new nodes, recovering state from hardware failures, and incrementally upgrading code. Developers can inherit this baseline API and extend it to support additional operations (e.g., database table create operations). ChiMods are stored in the distributed **Module Manager**, where they can be queried by a unique, semantic module name defined by the developer of the ChiMod.

3.3.1.2 Customly Configuring ChiMods into ChiPools of ChiContainers To form heterogeneous, application-specific distributed systems (e.g., a parallel filesystem), ChiMods must be allocated, parameterized, and instantiated on all processors expecting to execute their operations. To do this, Chimaera introduces the concept of a **ChiPool**, which is comprised of a semantic, globally-unique name defined by the user, a globally-unique integer identifier (**ChiPoolId**) allocated internally by Chimaera, a set of globally-addressable **ChiContainers**, and a set of **ChiDomains** representing ChiContainer addresses resilient to dynamic behavior, such as migration. Each ChiContainer is instantiated from a ChiMod using the *Create* API and represents a (single-node) state and a set of concurrent **ChiLanes** for holding operations to perform in the context of that state. The ChiContainer state is shared across all processors on the node through technologies such as CXL while ChiLanes can be affined to a core on any *general-purpose* processor on the node. ChiLanes may be partitioned to hold different types of operations. For example, a weakly consistent filesystem may have a reader lane and writer lane instead of a single lane for both. The number of ChiContainers can be expanded or contracted dynamically at runtime

to improve resource utilization, concurrency, and performance. For example, on a machine with a total of 100 nodes with an average of 16 cores per node, a ChiPool representing a key-value store may spawn 1600 ChiContainers that each represent a bucket. If 100 new nodes were added to Chimaera, a rebalancing operation could be performed where the number of ChiContainers is increased to 3200 and keys are re-hashed among buckets. ChiContainers can also be recovered if a ChiPool is notified a set of containers are no longer accessible. Multiple ChiPools can be created for the same ChiMod to support alternative I/O paths. For example, let's say we have a filesystem named ChiFS that supports tunable consistency. Two ChiFS instances can be created, one with the pool name "fs::posix" and the other "fs::weak_posix", representing I/O paths with and without strong consistency. ChiPools are stored in the distributed **Namespace** and can be queried by their user-defined name or ChiPoolId.

3.3.1.3 Building Loosely Coupled I/O Services through ChiTasks Loosely coupling components allows features to be independently developed and upgraded, which leads to improved code reuse and more frequent code releases. In addition, communications may occur across heterogeneous processors within a node or across nodes. To support these requirements, Chimaera introduces **ChiTasks**, which are lightweight, serializable messages that can be sent to ChiContainers. They store the ID of a ChiContainer operation to execute, its parameters, and various scheduling flags pertaining to safety and concurrency control. For example, a ChiTask representing a key-value store *put()* operation would contain the ID of the table being updated and a key-value pair. Many flags exist to describe how the ChiTask should be scheduled. For example, to improve concurrency, the flags `TASK_READ` and `TASK_WRITE` indicate whether the ChiTask manipulates data. Sequences of ChiTasks marked `TASK_READ` can be processed in parallel or out-of-order to improve performance. Alternatively, `TASK_PERIODIC` indicates that a task should be rescheduled

occasionally on a user-defined interval, which is useful for implementing services such as performance monitoring, data flushing, and heartbeats. Lastly, ChiTasks contain a **migration factor (MF)** that trades performance for energy, where $MF \in [1, \infty)$. For example, 1.2 indicates that 1.2x slowdown is an acceptable performance loss to migrate to a low-frequency core. For latency-sensitive tasks on the critical path, 1 is appropriate. These ChiTasks can be sent to Chimaera for execution using a queuing API that takes as input the ChiTask to schedule and a **ChiDomain Query** that represents a subset of ChiContainers and a communication pattern to apply (e.g., repeat until success, broadcast). This API can be called from within applications or within ChiContainers. Internally, this queuing leverages low-latency protocols such as shared memory, RDMA, and CXL to maintain high performance.

3.3.1.4 Enabling Code Evolution Through State Upgrades The ability to upgrade I/O stacks without requiring major service interruption is important for development speed. Upgrades can include various essential situations, such as bug fixes, performance enhancements, data structure changes, security patches, and new feature additions. To allow ChiMods to be incrementally upgraded and dynamically integrated into Chimaera, ChiMods provide developers with the ability to implement an *Upgrade* API, which takes as input a pointer to a pre-upgrade ChiContainer and an *UpgradeTask* containing paths to the new code. The upgrade does not necessarily affect all implementations of the ChiMod. For example, a ChiMod representing transparent anomaly detection for credit fraud may have CPU-based and GPU-based implementations. The CPU-based version may have a bug the GPU-based version does not. Upgrades are required to keep ChiTask structures backwards-compatible to allow tasks sent before the upgrade to execute correctly. Since upgrades may affect state and semantics, unstarted tasks are blocked and any in-progress tasks related to the ChiMod are executed to completion before the update proceeds. Since the upgrades are backwards-compatible, the update is applied per-processor in a decen-

tralized fashion to maximize scalability.

3.3.2 Cross-Processor Migration, Recovery, and Orchestration

Distributed systems must react to many events to maintain reliability and high resource utilization. Nodes may be dynamically (un)installed to increase scale, processes may crash abruptly due to power outages or software bugs, and hardware may deteriorate and become dysfunctional. However, load balancing, crash consistency, and reliability algorithms have traditionally been developed for machine architectures comprised of only a single OS running over a single processor. Various techniques have been explored such as preemptive migration [133], replication [134], and transactional metadata logging [135]. However, modern machines are integrating multiple accelerators per node, many of which run their own independent OS. These processors can be used to improve system reliability, scalability, and predictability. While some work has been done to leverage offloading I/O stack functionality, these works have focused on the assumption of just a single additional accelerator. Chimaera aims to provide the abstraction necessary for developers to build elastic, reliable systems in machines with many different processors.

3.3.2.1 Supporting Horizontal Migrations through ChiContainers There are several common situations where there is a need to migrate both computations and state, which we refer to as horizontal migration. Preemptive migration algorithms, for example, can be used to move data and its operations before a storage hardware fails. Alternatively, load balancing algorithms may migrate a state to reduce NUMA effects of computations executed on a different accelerator within a node. To accomplish this, ChiMods provide the *Migrate* API. Developers can call their internal APIs to send data to this new instance. For example, a ChiContainer representing a bucket in a key-value store could be migrated by repeatedly calling a custom *KVPut* API that re-puts every key from the old bucket to the new. After the entire state has been copied to the new instance, *Migrate* returns and the old ChiContainer is destroyed. *Migrate* is invoked automatically by the ChiOrchestrator or manually.

3.3.2.2 Supporting Vertical Migrations through ChiLanes With the rise of technologies such as CXL, accelerators can share memory regions coherently, avoiding the need for state transfers required by horizontal migrations in certain cases. In Chimaera, a vertical migration moves only computations that occur over a shared state between processors running on the same node. These migrations can be used to reduce contention on CPU resources by offloading tasks to other processors. In addition, vertical migration can be used to improve throughput and recovery in the event of an OS crash or hardware death by allowing tasks to be executed on a separate processor, avoiding the need to wait for a full restart or hardware replacement. To accomplish this, ChiContainers store tasks in ChiLanes, which are shared-memory, multiple-producer-single-consumer queues that can be affined to general-purpose processors (e.g., SmartNIC, CSDs, CPU, etc.) on the same node. Since ChiContainers are instantiated on every general-purpose processor on the node, this migration is accomplished simply by descheduling the ChiLane from its current worker and then scheduling on another processor’s worker set. Vertical migration is invoked automat-

ically in the ChiOrchestrator or manually by setting the ChiLane’s affinity.

3.3.2.3 Supporting Fault Tolerance Through Restart and State Recovery

Not all systems are reliable and require consideration to power outages, hardware deterioration, and fatal software bugs. There are three cases of failure that Chimaera addresses: 1) a subset of processors shut down, but other processors on the node can still access the data; 2) an entire node shuts down, but the data will still be there after rebooting; 3) a storage medium is lost and data must be recovered. To handle these situations, ChiMods provide the *Recover* API. This API is invoked from the processor where the ChiContainer is expected to be reconstructed and takes as input the ChiDomainId of container that died. In the first case, the ChiMonitor eventually detects the processor deaths, pauses them, and invokes vertical migration to move the stalled ChiLanes to available processors on the same node. Before allowing the ChiLanes to be processed, a non-deep *Recover* is invoked to correct any metadata inconsistencies the crash may have caused. For example, a filesystem ChiMod may iterate over a transaction log to ensure that the inode table is correct. In the second case, the ChiMonitor waits for the node to be rebooted and sends a recovery task there. In the last case where data itself is lost, a deep *Recover* is invoked to rebuild the data. For this, techniques such as replication and erasure coding can be implemented.

3.3.2.4 Remaining Addressable During Migration and Recovery through

ChiDomains Chimaera is a highly dynamic system, where ChiContainers can be dynamically migrated and recovered due to factors such as load balancing and reliability. This dynamic behavior makes it difficult to send ChiTasks to ChiContainers based solely on existing address infrastructures, such as IPv6. To maintain addressability when the physical location of a ChiContainer is changing, Chimaera introduces ChiDomains. A **ChiDomain** is a mapping between a **ChiDomainId** (a logical address) to a set of either other ChiDomainIds or physical node addresses (e.g., IPv6). A ChiDomainId includes the ChiPoolId, a major number, and a minor number. For example, Chimaera automatically creates a domain for each ChiPool to represent the mapping between ChiContainers and nodes, which is referred to as the **Global Domain**. The major is 1 and the minor is the ChiContainer ID (1 to #ChiContainers). E.g., 8.1.27 represents pool 8, container 27. During migrations, the Global Domain is updated to reflect the new mapping. Additionally, the domain representing the set of ChiContainers local to a processor (relevant to caching algorithms) is created and maintained automatically. Lastly, custom domains can be created by ChiContainers or manually. For example, a domain can represent replication, where the ChiDomainId maps to multiple ChiContainer addresses each representing a mirror. Domains are stored in the **Domain Table**, where they can be queried by a ChiDomainId.

3.3.3 Mediating Contention on Heterogeneous Hardware

Applications spanning HPC and Cloud have diverse I/O requirements that no single system fully addresses. This has led to complex machine architectures and I/O stacks to balance subsets of these requirements. However, in hyperconverged platforms where several conflicting applications run simultaneously over a complex hardware set, maintaining high resource utilization while balancing these diverse I/O requirements becomes an even more arduous challenge. As opposed to requiring application and I/O stack developers to carefully develop their code to consider when and where to place tasks among heterogeneous resources, Chimaera proposes a scheduling layer to decide this intelligently and automatically based on current resource utilization, energy efficiency, and expected runtime. This scheduling allows ChiLanes and ChiContainers to be dynamically migrated and ChiMod implementations to be dynamically selected.

3.3.3.1 Coordinating I/O Stack Scheduling with the Chimaera Runtime

The Chimaera Runtime is responsible for routing ChiTasks to ChiContainers, orchestrating ChiContainers, and executing ChiTasks in ChiLanes. To send tasks to Chimaera, a shared-memory ingress queue is created per-user. Chimaera routes ingress tasks to ChiLanes based on their ChiDomain Query. To process ChiLanes, the runtime spawns a set of worker threads and allocates a subset of cores to run them. The maximum number of dedicated cores per NUMA node is user-defined and configurable per-processor. To improve performance, workers are divided into two categories: low-latency and high-latency. Latency-sensitive workers are dedicated to CPU cores, while high-latency workers can be overlapped with one another. The ratio of latency-sensitive to expensive is user-configurable. Workers can be dynamically merged to reduce CPU utilization if their load is beneath a user-configurable threshold (in milliseconds).

Algorithm 1 Migration Algorithms

```

1: function VMIGRATE(node)
2:   for lane in WorkOrch.lanes do
3:      $p = \min_{p \in \text{node.cpus}} (p.\text{load} + V\text{Load}(p, \text{lane})) / p.\text{cores}$ 
4:      $w = \min_{w \in p.\text{workers}[\text{IsLowLat}(\text{lane})]} w.\text{load}$ 
5:     lane.Affine(p, w)
6:   end for
7: end function
8: function HMIGRATE(nbors = Monitor.neighbors)
9:   for cont in WorkOrch.containers do
10:     $p = \min_{p \in \text{nbors}} (p.\text{load} + H\text{Load}(p, \text{cont})) / p.\text{cores}$ 
11:    mtask = MigrateTask()
12:    If  $p \neq \text{cont}.p$ , do cont.Migrate(mtask, p)
13:  end for
14: end function
15: function HLOAD(p, cont)
16:    $\text{laneLoads} = \sum_{\text{lane} \in \text{cont}} V\text{Load}(p, \text{lane})$ 
17:    $\text{migrateLoad} = \text{TaskLoad}(p, \text{MigrateTask}())$ 
18:   return  $(p \neq \text{cont}.p)(\text{laneLoads} + \text{migrateLoad})$ 
19: end function
20: function VLOAD(p, lane)
21:   return  $(p \neq \text{lane}.p) \sum_{\text{task} \in \text{lane}} \text{TaskLoad}(p, \text{task})$ 
22: end function
23: function TASKLOAD(p, task)
24:    $\text{load} = \text{task.PredictStats}(p).\text{procTime}$ 
25:    $f = \text{Monitor.coreFreqRange}$ 
26:    $\text{normpf} = (p.\text{freq} - f.\text{min}) / (f.\text{max} - f.\text{min})$ 
27:    $MF = (1 - \text{task}.MF)\text{normpf} + \text{task}.MF$ 
28:   return  $\text{load} * MF$ 
29: end function
30: function ISLOWLAT(lane)
31:    $\hat{\text{load}} = V\text{Load}(\text{lane}.p, \text{lane}) / \text{len}(\text{lane})$ 
32:    $\hat{IO} = \sum_{\text{task} \in \text{lane}} \text{PredictStats}(p).\text{ioSize} / \text{len}(\text{lane})$ 
33:    $CC = \text{ChimaeraConf}$ 
34:   return  $\hat{\text{load}} \leq CC.\text{lowLoad}$  and  $\hat{IO} \leq CC.\text{lowIO}$ 
35: end function

```

3.3.3.2 Actively Learning I/O Stack Behavior ChiMods can represent a broad range of behavior, from computationally-complex data operators to latency-sensitive metadata queries. Understanding these characteristics is critical for improving scheduling decisions. To do this, Chimaera allows ChiMod developers to

define *PredictStats* and *ReinforceStats* APIs. *PredictStats* returns a prediction of I/O size, memory use, and processor time (i.e., **load**) – the amount of time a ChiTask is scheduled and running (i.e., not blocked). This prediction should capture NUMA effects, core frequency, and CPU utilization, which the ChiMonitor tracks. Simple techniques such as binning and long-running average can be applied. Alternatively, machine learning libraries such as Mllib can be used to implement reinforcement learning techniques, such as piecewise linear regression, random forests, and short-term memory models to understand behavior. This flexibility allows developers to choose models that suit their needs rather than forcing one-size-fits-all approaches. To train these models, Chimaera monitors the processor time and runtime of ChiTasks and stores them in an in-memory log per-ChiContainer, with a configurable minimum and maximum number of samples. This sampling occurs either when the model has not yet been trained or periodically on an interval. The interval should be chosen such that performance is not significantly affected by time measurements and reinforcement. After the sampling period is over or when a certain number of samples are reached, all collected measurements are passed to the ChiContainer’s custom *ReinforceStats* method. ChiMod developers can internally persist these models and implement transfer learning techniques to share them across ChiContainers to improve model quality.

3.3.3.3 Determine the Best ChiLanes to Migrate Vertically Vertical migrations can offload burden from processors and reduce contention within a node. This migration is triggered periodically (user-configurable) per-processor to ensure adaptiveness to changing circumstance. Since these migrations do not perform data copies, the vertical migration period is in the order of milliseconds – similar to POSIX threads. The main objective of this algorithm is to offload ChiLanes to more energy-efficient (i.e., lower core frequency) processors under the conditions that ChiTasks slow down within a threshold (the migration factor) and that the processor does not become overloaded. To do this, the ChiLane is first sent to the processor where its predicted average load per Chimaera core post migration is minimized (line 3). ChiLane load (VLoad, line 21) is calculated by summing all individual task loads, but returns 0 if the processor p is where the ChiLane already is. TaskLoad uses *PredictStats* to estimate processor time $task.procTime$ and divides by an adjusted migration factor MF (lines 25-27), where the fastest processor has $MF = 1$ and the slowest has $MF = task.MF \geq 1$. This makes the load calculation smaller on slower processors, improving its chance of selection. After the processor is chosen, the ChiLane is scheduled to a worker based on the latency requirements of the ChiLane and the current load of the workers. A ChiLane is considered low latency (lines 30-35) if the average processing time and I/O size are small. The definitions of small are user-configurable (by default, $CC.lowLoad = 10\mu s$ and $CC.lowIo = 16KB$. $CC.lowLoad$ was chosen through a benchmark measuring average metadata performance in ChiFS, while $CC.lowIo$ was chosen based on previously published work that demonstrated I/O partitioning can significantly reduce head-of-line blocking [1, 25].

3.3.3.4 Determine the Best ChiContainers to Migrate Horizontally Horizontal migrations can be used to balance load across nodes for a situation such as dynamic node addition or to address NUMA effects in vertical migration. This form of migration is typically more expensive as it requires some state to be copied in order for computations to be correct. For this reason, the horizontal migration period is in the order of seconds. Horizontal migrations move all ChiLanes and state to either a different node or a different processor on the same node. The overall migration algorithm (HMMigrate, line 8) is similar to vertical migration – the main difference is the load calculation spans all ChiLanes and includes the total estimated processing time of the MigrateTask. This ensures the migration only occurs if the load on a different processor is so much lower that the migration cost is worthwhile.

3.3.3.5 Determine the Best ChiMod Implementation to Use It is possible that only certain tasks on a ChiLane should be migrated to a wimpy processor or to some other (potentially powerful) accelerator (e.g., GPU, FPGA) if compatible. A filesystem, for example, may schedule read and writes to the same queue to maintain strong consistency. Synchronous reads should not be offloaded to avoid high PCIe latencies, while asynchronous writes in the same queue may be worthwhile. To accomplish this, the ChiMod implementation is chosen dynamically when it is scheduled. The PredictStats function is used to estimate the processing time (load) of the ChiMod for each candidate implementation on its respective processor (either general-purpose or special-purpose). This is then divided by an adjusted MF similar to TaskLoad on line 23. The implementation with the lowest load is selected.

3.3.3.6 Determine When to Rescale ChiPools Nodes and processors can be dynamically added and removed in Chimaera. This can be done by administrators due to routine maintenance or automatically due to a crash. However, unlike container orchestration frameworks, scaling is more complex than simply spawning a container. This is because certain services, such as extensible hashtables, may require data to be recomposed in response to scaling. To do this, ChiMods possess the *Rescale* API, which is invoked after processors are added and removed by the ChiMonitor. This API is called if the ChiOrchestrator determines the reduced load caused by expansion outweighs the data movement cost of expansion or if nodes are removed from the ChiMonitor due to failure.

3.3.4 Implementation Details

Chimaera is implemented in a total of 30K LOC. We build upon the foundations provided in LabStor [1], which was originally designed as a single-node, composable I/O stack development platform for symmetric CPUs. In addition to this, we provide a distributed and heterogeneous composable I/O stack architecture, including load balancing and multi-tenancy. Mochi Thallium is used to provide RDMA.

3.3.4.1 An Example I/O Path To demonstrate our design, we implement ChiMods for compression, average, a strongly-consistent key-value store (ChiKVS), and a filesystem (ChiFS). ChiFS is a lightweight frontend to ChiKVS, where directories are stored as key-value pairs. ChiKVS persists metadata in a transaction log that is asynchronously compacted periodically. The log stores the per-node metadata operations in a concurrent log ordered using a logical timestamp implemented using an atomic counter. The log is traversed to perform crash recovery.

3.4 Evaluations

3.4.1 Hardware

All tests were conducted on a research cluster, designed to support a hierarchical storage architecture. The cluster consists of storage and a compute rack, each having 32 nodes. The two racks are interconnected by two isolated Ethernet networks (one of 40Gb/s and the other 10Gb/s), with RoCE enabled. Each compute node has a dual Intel(R) Xeon Scalable Silver 4114 with 24 cores and 48 threads, 48 GB RAM, 128GB NVMe PCIe x8 drive, 256GB SSD drive, and 1TB HDD. In each evaluation, we only use the NVMe.

We did not have SmartNICs and computational storage devices available in our research cluster. However, we take an approach similar to LegOS [136] and DevFS [12] to emulate asymmetric processing speeds and NUMA. We emulate the Mellanox BlueField E-Series DPU [137] by dedicating 8 CPU cores on a separate NUMA node from the application. We also emulate a CSD as described in DevFS [12], which has 4 cores. For both devices, Chimaera is limited to use up to 16GB of RAM, and the core clock is lowered to 1GHz. In each evaluation, the testbed is configured to emulate the following: 12-core CPU (2.2GHz), 8-core SmartNIC (1 GHz), and 4-core CSD (1 GHz). Chimaera is configured to dedicate up to 4 cores on the CPU and all cores on the SmartNIC and CSD.

3.4.2 Evaluation Objectives

The objectives of this evaluation are to demonstrate:

1. Composing I/O stacks to workloads yields benefits to performance and resource utilization
2. Allowing I/O stacks to be dynamically offloaded to accelerators yields performance and energy benefits
3. Targeted monitoring of I/O stack operations can be done with high accuracy and little overhead
4. Intelligent scheduling can mediate contention, resulting in better overall multi-tenant performance and utilization

3.4.3 Evaluation Results

3.4.3.1 Monitoring Accuracy and Performance In this evaluation, we demonstrate the impact of sampling frequency on the performance and accuracy of monitoring ChiContainers. To do this, we define a ChiFS I/O stack that contains filesystem metadata, compression, encryption, and block storage. We use the IOR benchmark [138] to perform 4GB of I/O per process where the I/O size is randomly chosen between 4KB and 1MB. The I/O happens in parallel to node-local NVMe devices. We measure application runtime and ChiTask load model accuracy using mean absolute percentage error (MAPE). The sampling frequency is varied between 1/10 and 10/1, where 10/1 means samples are collected for 10 seconds every 1 second.

From Figure 3.5, it can be seen the choice of sampling frequency has tradeoffs between accuracy and performance. The highest frequency 10/1 results in all models attaining maximum accuracy within the first 5 seconds, while the lowest frequency

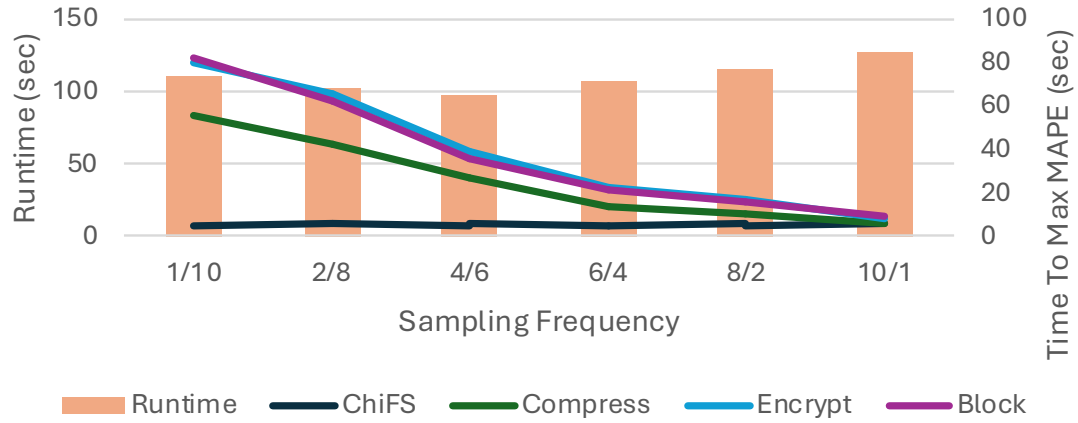


Figure 3.5. Effect of sampling frequency on runtime and monitoring accuracy. Low frequency (1/10) has high performance but takes 8x longer to reach maximum accuracy. High frequency gets to maximum accuracy fastest, but incurs an overhead of 27% compared to the 4/6 frequency. 4/6 obtains best performance since the model is more accurate than 1/10 for most of the program, leading to better scheduling decisions.

1/10 results requires 85 seconds to maximize accuracy for each ChiMod. The filesystem metadata management ChiMod reaches maximum accuracy quickly in each case since the model used is simply rolling average (metadata updates have approximately constant-time complexity due to the use of hashing). Compression, encryption, and block storage are more complex and use nonlinear least squares regression to model behavior. In each case, the MAPE becomes fairly high, where the average prediction is within 15% of the true value for each model by the end of the program. However, application runtime becomes deeply affected by the choice of sampling frequency. At the highest frequency, performance is 18% worse than the lowest frequency. Conversely, the lowest frequency is 10% slower than the moderate frequency of 4/6. This is because 4/6 converged to optimal accuracy within 25 seconds of runtime, whereas 1/10 took 85 seconds, meaning 1/10's scheduling decisions were suboptimal due to lower accuracy. Overall, it can be seen that Chimaera can efficiently and accurately model the behavior of tasks, with some tradeoff between runtime performance and accuracy.

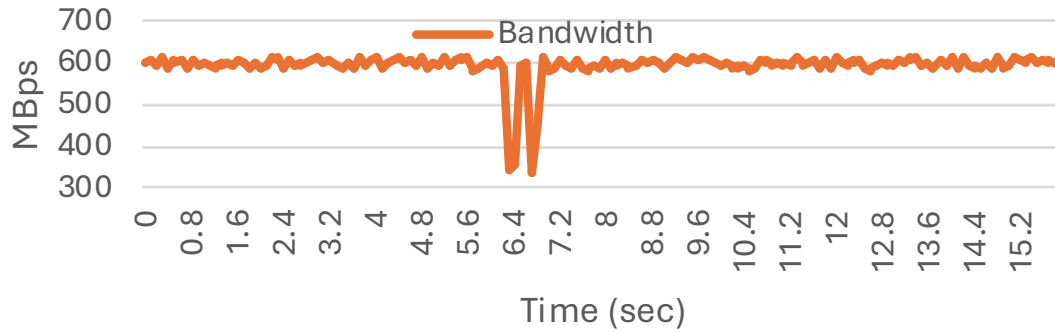


Figure 3.6. Effect of upgrade on runtime performance for a 4KB I/O benchmark on 4 nodes. The upgrade is sent at 6 seconds, resulting in a brief dip in performance for 8ms in total.

3.4.3.2 Upgrade Performance In this evaluation, we demonstrate the impact of dynamically upgrading ChiMods at runtime. To do this, we run a synthetic benchmark and perform an upgrade of ChiFS during its execution. The benchmark initially creates 1000 files per node and then writes 4KB at random offsets for a configurable duration. The upgrade changes the in-memory filesystem metadata structure from an `std::unordered_map` to a `boost::unordered_map`, requiring a copy of the metadata to occur. The benchmark is configured to run for 16 seconds, and the upgrade occurs after 6 seconds. ChiFS is deployed as a ChiPool of 16 ChiContainers running over 4 nodes – resulting in an initial distribution of 4 per node. The benchmark is 48 processes (12 per node).

From Figure 3.6, it can be seen that the cost of the upgrade affects performance minimally, resulting in a total throughput drop of 40% for .1 seconds. There is a brief lag as the update has to be distributed to each node and wait for the ChiFS to become quiescent, which requires waiting for all pending ChiFS tasks to complete across all processors on the node. Upgrades are processed independently per-node, resulting in a few different lag points. The throughput drops are due to the upgrade itself. It requires a binary to be dynamically loaded from a backend filesystem and requires

the metadata state (1000 inodes) to be copied. The binary is approximately 8MB in size (100usec) and the metadata copy completes in 3usec. The overall runtime of the application was affected by only a few milliseconds to complete this upgrade. Overall, Chimaera’s upgrade pipeline avoids the service interruptions caused by inefficient upgrade pipelines such as those for kernel-level filesystems that require the entire service to be shut down, and potentially a system reboot, in order to address.

3.4.3.3 Benefits of Composable Storage In this evaluation, we demonstrate the performance benefits of allowing storage stacks to be composed to particular applications compared to tightly coupled deployments. To do this, we run two applications with different I/O and computational behaviors and compare storage stacks tailored to each of them. We define two I/O stacks – a ChiFS deployment with Snappy [139] compression and without compression. For the applications, we first run an I/O kernel of VPIC [140], which is a large-scale space weather plasma simulation code. In VPIC-IO, each MPI process writes data related to eight million particles and each particle has eight floating point properties with a total size of 32 bytes. At each checkpoint, each process writes 256MB of data in units of 1MB. In addition, we run filebench [94] web server per node, which is latency-sensitive and creates, reads, and closes many small files (on average 16KB in size) using 100 threads per-node, where I/O happens to different directories on each node. No accelerator offloading is performed. We run the evaluation for 8 nodes, where VPIC has 12 processes per node and runs 4 steps.

From Figure 3.7, it can be seen that the two I/O paths cause different performance variations for the two applications. In the case of VPIC, performance increases by 40% when writing to NVMe. This is because the average VPIC block decreased from 1MB to 580KB – a 45% reduction in the total I/O performed by VPIC. Since VPIC is bandwidth-intensive, the reduced data size is met with lower I/O time, which outweighs the in-memory compression time and leads to a significant performance in-

	VPIC		Web Server	
	Runtime (sec)	I/O Size	Runtime (sec)	I/O Size
-Compress	345	1MB	104	16KB
+Compress	248	580KB	166	13KB

Figure 3.7. Comparison of two I/O stacks for two different workloads. Compression gives VPIC a 40% performance improvement by reducing I/O volume. Compression, however, gives significant overhead to the latency-sensitive web server.

crease. Conversely, running filebench with this compression was not optimal and resulted in an overall 65% decrease in performance. While snappy reduced the data size of the filebench workload by 23%, the I/O size was already small enough that similar latency penalties were experienced in addition to the compression overhead. Overall, the ability to compose customized I/O paths to address specific workload behaviors is an important feature that can lead to great performance gains compared to one-size-fits-all approaches.

3.4.3.4 Effectiveness of Offloading Across Many Accelerators In this evaluation, we demonstrate the effectiveness of dynamically offloading tasks across diverse accelerators within a node through vertical migration. To do this, we consider an I/O stack that transparently performs a data-intensive KMeans clustering, similar to credit fraud detection algorithms [141], and compare several hardware configurations: CPU-only (Spark), CPU + SmartNIC (SmartShuffle), CPU + CSD (SmartSSD), and all accelerators (Chimaera). Since SmartShuffle and SmartSSD were designed specifically for particular hardware, we emulate their code by running different configurations of Chimaera where lanes are affined to particular processors. The clustering dataset used is 1GB per process, totalling 256GB of data. In the case of all accelerators being used, the migration factor is varied between 1.1 and 2. We measure both application runtime and resource utilization.

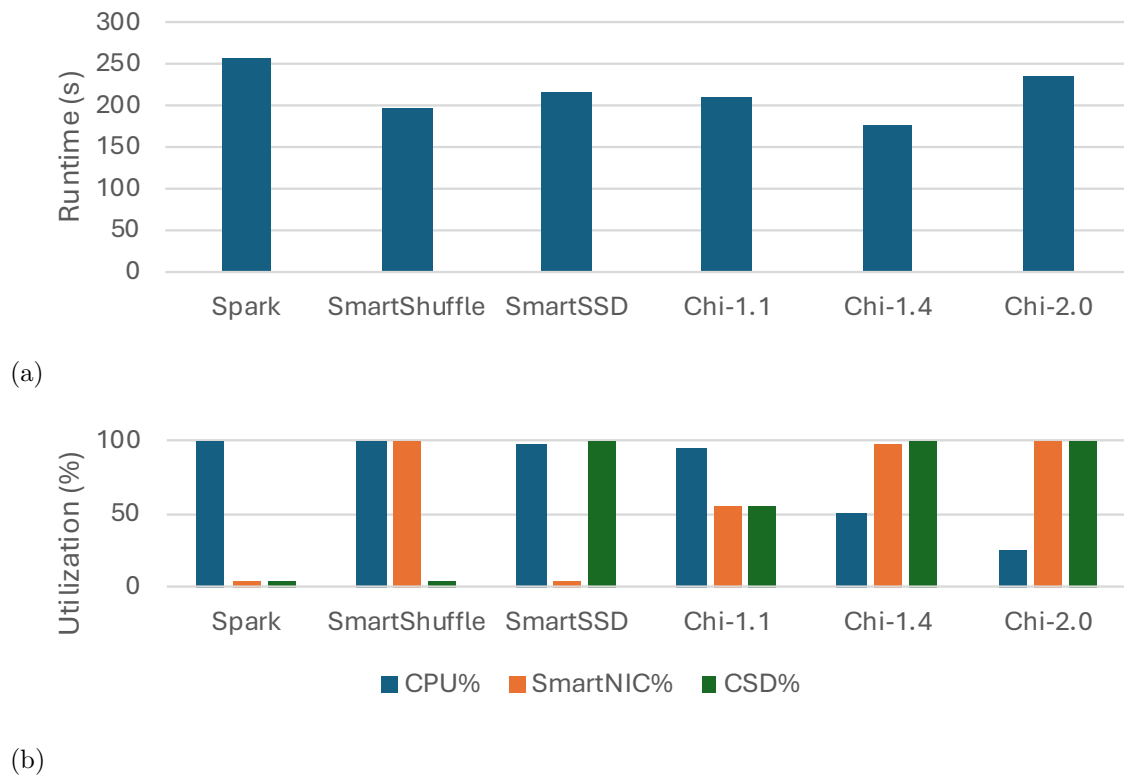


Figure 3.8. Comparison of I/O stack designs for running a transparent KMeans clustering. Chimaera, unlike all other solutions, utilizes all available processors, yielding the highest performance and resource utilization.

From Figure 3.8, it can be seen that the different offloading approaches result in significant differences in application performance and energy efficiency. When using the CPU-only version of KMeans, the runtime is 30% slower than SmartShuffle and 15% slower than SmartSSD. This is because the CPU-only KMeans does not leverage the additional processing units given by the SmartNIC to perform these calculations. Additionally, SmartShuffle outperforms SmartSSD since the SmartNIC has twice as many cores as the CSD. Lastly, Chimaera performs as much as 43% faster than the CPU-only KMeans since it leverages all available cores. When $MF = 1.1$, the performance is 36% faster since all cores receive work. However, since only a 10% performance loss is accepted per-task, the resource utilization is still suboptimal and the CPU remains overloaded. By $MF = 1.3$, the maximum resource utilization and performance is obtained, leading to the optimal 46% solution. With $MF=2$, performance is similar to the CPU-only version since the accelerators are prioritized. However, only 25% of the CPU is utilized, meaning most work is handled in the accelerators, which is more energy-efficient. Overall, it can be seen that accelerators offer great potential to improve performance and energy consumption despite having wimpier architectures. However, the orchestration of tasks across these architectures has a delicate balance. Existing solutions prioritize affining tasks to a particular accelerator type. However, Chimeara schedules operations across diverse accelerators leading to higher resource utilization, performance, and hardware-independent software solutions.

3.4.3.5 Reducing Cross-Application I/O and Processor Contention In this evaluation, we demonstrate the effectiveness of dynamic load balancing for running co-located workloads. To do this, we run two workloads and I/O stacks: filebench-varmail (latency-sensitive) and Gray Scott + snappy compression (bandwidth- and compute-intensive). Gray Scott [142] solves a partial differential equation in a cubic grid with characteristic dimension L . We set $L = 1568$ and run 4 steps, resulting in a total of $500GB$ of uncompressed data and $340GB$ after compression. We run both applications at a scale of 16 nodes, where Gray Scott has 12 processes per node. We compare three migration policies: random mapping (Rand), migration factor (MF), and migration factor + horizontal scheduling ($MF-HZ$). A migration factor of 1.2 is used for write operations and 1.0 for reads for MF and $MF-HZ$. This ensures that writes have the potential to offload to slower processors, while reads always remain on the fastest processor to minimize latency. We measure application runtime, resource utilization, p99 latency (varmail), and bandwidth (Gray Scott).

From Figure 3.9, it can be seen the choice of scheduling policy has significant impact on application performance and utilization. For Gray Scott, the rand results in a 13% performance decrease compared to the MF scheduling. This is because 4 / 16 CPU cores are dedicated to Chimaera for random, while MF only has 2 – meaning the applications get two cores back. For rand, tasks are assigned to all running workers roughly equally, limiting the room for elasticity. This results in all cores being only partially utilized. Since Gray-Scott performs only asynchronous writes, MF offloads most of Gray-Scott’s work to the SmartNIC and CSD, resulting in CPU cores being released back to applications. In addition, this asynchronicity leads MF and $MF-HZ$ to have similar runtime performance. However, $MF-HZ$ has 8% higher bandwidth than MF . This is because $MF-HZ$ migrates ChiFS to the SmartNIC to lower NUMA latency. The SmartNIC is chosen as it has more cores and thus holds more ChiFS tasks. For varmail, both runtime and latency are



Figure 3.9. Performance and utilization of co-locating compute- and bandwidth-heavy Gray Scott and latency-sensitive varmail for different scheduling policies.

affected by the migration policy. For the random mapping, the runtime is 35% slower than MF and p99 latency is 80x worse. This is because latency-sensitive varmail tasks are mapped to the same workers as bandwidth- and computationally-expensive Gray Scott tasks, resulting in head-of-line blocking. The MF algorithm uses active monitoring to map the Gray Scott tasks to high-latency workers and varmail tasks to low latency based on I/O and CPU load estimates. The high-latency and low-latency workers are on separate cores to minimize interference. Lastly, the MF and $MF-HZ$ algorithms do not affect the varmail performance significantly since the algorithm is read-heavy. Most tasks are processed by the CPU (since the $MF = 1.0$ for reads), resulting in $MF-HZ$ never moving ChiContainer state to an accelerator. Overall, I/O stack work must be carefully mediated to ensure high performance and resource utilization. Consideration must be given to the I/O and computational characteristics of workloads and hardware.

3.5 Conclusion

In this work, we address the limitations of developing workload-specific I/O paths in heterogeneous environments with many accelerators. We present Chimaera: A Distributed Semi-Microkernel for Composing Heterogeneous Data Services. We showcase a loosely-coupled, modular design that provides the notifications and event handlers necessary to develop specialized, reliable, and evolutionary I/O features. We leverage this modularity to compose workload-specific distributed I/O services simultaneously on the same hardware set. We demonstrate how work generated on these diverse paths can be cohesively scheduled across heterogeneous processing resources in modern accelerated machines to minimize contention while respecting the limitations of accelerators. Experimental results demonstrate that I/O paths composed in Chimaera can yield performance gains of up to 46% over alternatives under various workloads.

CHAPTER 4

MEGAMMAP: BLURRING THE BOUNDARY BETWEEN MEMORY AND TIERED STORAGE FOR DATA-INTENSIVE WORKLOADS

4.1 Introduction

Traditionally, memory and I/O substrates have been considered separate entities due to their differences in terms of performance and persistence. However, modern data-intensive memory-centric workloads widespread in HPC and Cloud are challenging these distinctions. Data analytics [99], machine learning [143], and deep learning [144] codes perform large-scale computations on data which greatly exceed the bounds of memory, relying on explicit data movements to I/O systems to meet basic capacity requirements. This often leads to significantly increased development complexity [145] and suboptimal, one-off solutions where I/O and compute happen in distinct, synchronous phases [103], incurring the memory wall [146] problem in the compute phase and the notorious I/O bottleneck [3] during the I/O phase. Conversely, scientific simulation codes [100] are becoming increasingly memory-intensive and are developed assuming large memory capacities are provided to avoid out-of-core development complexity. To reduce code complexity and I/O costs, HPC and Cloud sites have been steadily increasing total DRAM capacity so that datasets can be staged primarily in main memory [147]. However, while many applications desire an effectively infinite memory to generate and analyze massive datasets, the ever-increasing size of data [148] and the extreme financial and energy costs of DRAM [149, 150] make scaling DRAM capacity unsustainable.

To provide applications the capacity and bandwidth they require, heterogeneous storage is being explored. Modern storage accelerators (e.g., NVMe, Phase Change Memory, and Compute Express Link) have made significant advances in terms of capacity, bandwidth and latency, offering performance within an order of

magnitude of DRAM while providing high density [101]. These accelerators have already been adopted in several HPC sites. ORNL’s Summit and Lawrence Livermore’s El Capitan machines, for example, come equipped with high-bandwidth, low-latency, node-local NVMe, whereas Argonne’s Aurora machine contains both NVMe and 3D XPoint modules. However, while modern storage provides great potential to improve performance, presenting this highly heterogeneous **Deep Memory and Storage Hierarchy (DMSH)** to applications remains an ongoing challenge.

In order to address the performance, programmability, and capacity limitations of distributed out-of-core algorithms, the semantic gap between memory and heterogeneous storage should be eliminated through a tiered, nonvolatile **Distributed Shared Memory (DSM)** abstraction. With this approach, datasets can be produced and analyzed in what appears as a single, coherent main memory, when in reality data is dispersed among both memory and tiered storage. This removes the burden of explicit I/O synchronization and memory management from developers, while simultaneously reducing data movement overheads by leveraging storage accelerators and allowing compute and I/O to execute concurrently. While several DSM solutions have been proposed and deployed in production Cloud centers [151], none currently provide transparent integration with tiered storage, persistence, and optimizations for HPC. In order to provide such an abstraction, several challenges must be overcome.

First, applications should be able to propagate their access pattern intention. Existing DSMs are unaware of the regions of shared memory that will be read or modified until it happens, which limits the effectiveness of prefetching and data placement algorithms [152] and causes extreme overheads to maintain memory coherence [153]. While this may appear as an imposition to users, many distributed and out-of-core algorithms are already structured to accomplish this, as they typically operate over

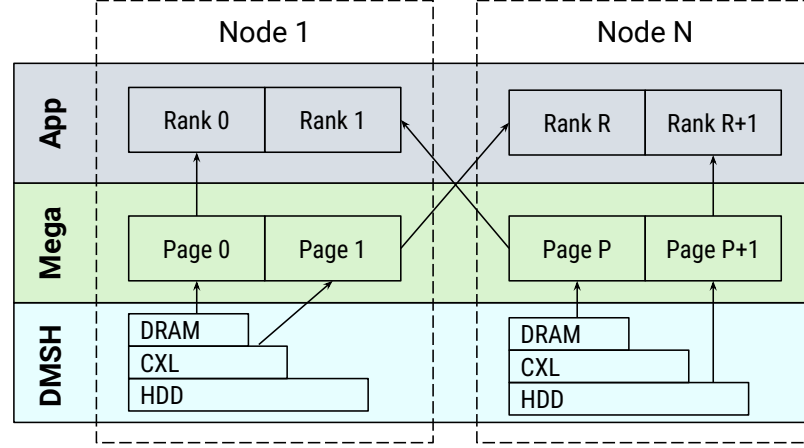


Figure 4.1. Depiction of Coherent, Distributed, and Tiered MegaMmap Memory

large, well-defined subsets of out-of-core data between synchronization points, such as barriers and locks [154].

Second, memory coherence policies must be optimized to address the characteristics of HPC environments. DSMs are primarily designed to support chaotic, unpredictable multi-tenant Cloud workloads while remaining fault-tolerant, incurring prohibitive communication overheads to locate and invalidate replicas [154]. However, HPC applications are typically executed in isolation, do not require fault tolerance, and are oftentimes coordinated, where data structures are read and modified in well-defined phases [155] (e.g., producer-consumer workflows).

Third, new data placement and prefetching policies must be developed to support the DMSH and consider memory-centric application behavior. Existing DSMs operate only over a single tier and have no indication of access pattern. However, hardware in the DMSH is highly diverse, requiring considerations to device lifetime, access pattern, capacity, latency, and bandwidth. In addition, out-of-core workloads tend to operate over large datasets iteratively, where a large subset of the dataset is processed and then ignored [156]. This behavior offers opportunity to overlap data movement with the computation.

In this work, we present the design and implementation of MegaMmap: a new tiered, nonvolatile DSM for HPC that abstracts both remote memory and storage, effectively providing infinite memory capacity without sacrificing performance. MegaMmap provides a C++ library that leverages language features to provide a uniform, byte-addressable interface to present massive datasets as if they were in memory. A transactional memory API is provided, allowing applications to specify their intent to read or modify a region of data. Internally, MegaMmap manages the complexity of data placement among the distributed memory and storage hierarchy through the use of novel online prefetching and data organization algorithms. Factors such as randomness seeds and access intent are used to guide data organization decisions and ensure that global memory accesses do not frequently have to stall for remote memory or storage. MegaMmap provides a wide variety of intent-aware cache-coherency optimizations to minimize memory access latency for a variety of workloads. The contributions of this work are as follows:

1. A durable, persistent, and intuitive **Distributed Shared Memory (DSM) system**, which significantly reduces out-of-core development complexity by allowing applications to present massive datasets as memory objects.
2. A user-driven **transactional memory access API**, which leads to improved decision-making in cache coherence and data organization policies by propagating memory access intent.
3. A comprehensive set of **intent-aware memory coherence** optimizations, which improves the latency and bandwidth of memory accesses based on workload characteristics.
4. A wide variety of **tiered data organization** policies, which minimize I/O stall times by leveraging heterogeneous storage hardware and advance knowledge of

access pattern intent.

4.2 Background and Related Work

Software DSM is being investigated as a method of reducing the programming burden of analyzing massive datasets in memory and sharing information between processes [157]. Much research has been conducted to discuss the latency considerations of cache coherency and the effectiveness of data placement and prefetching in shared memory systems.

Memory Mapped I/O (MMIO): MMIO [158] enables applications to share data across processes and present persistent files as C-style arrays. In the persistent case, data from the file is automatically loaded from the storage system during a page fault. Modifications to the file are made in the OS page cache and then evicted asynchronously during times of high memory pressure or an explicit synchronization call (e.g., `msync` or `fsync`). MMIO simplifies programming out-of-core algorithms by removing the burden of deciding which parts of the file are buffered in main memory from the developer [158]. However, this approach traditionally suffers in performance due to frequent page faulting [159], little-to-no prefetching requiring complex `mmap` tracing [160], and significant I/O overhead caused by small (typically 4KB) page sizes [161]. The default page fault path in `mmap` can be customized to change prefetching and data placement policies by either editing the kernel directly or leveraging the `userfaultfd` [162] system call. Efforts are being made to improve the performance of MMIO for single-node and DSM systems, described below.

Single-Node Shared Memory: Several works have been proposed customizations to the traditional MMIO path to improve performance and resource utilization in single-node cases. `uMMAP-IO` [161], for example, utilizes this approach to extend virtual memory to SSDs and Hard Drives for HPC workloads, effectively increasing mem-

ory capacity. However, `userfaultfd` increases the cost of an individual page fault due to increased context switching between the kernel and userspace [163]. Many works aim to provide custom page fault handlers while avoiding `userfaultfd` [158, 164–167]. Aquila [164] and FlatFlash [167], for example, make kernel modifications to reduce software overheads of the MMIO path. To avoid kernel modification, Tricache [166] uses LLVM to prepend a custom buffer cache handler before all CPU load/store operations, removing a physical page fault entirely. In their specific page handler code, NVMeS are used as a temporary buffering layer via the Storage Performance Development Kit (SPDK). These works are focused on single-node, multi-threaded applications, not fully distributed, multi-process shared-memory applications.

Distributed Shared Memory (DSM): DSM allows applications to access remote memory as if it were its own memory. These works typically extend the virtual memory management subsystems of the OS to implement custom page fault handlers to provide the shared memory abstraction, although some use language features (e.g., operator overloading). A number of works have proposed designs and improvements to DSM systems. TreadMarks [153] uses virtual memory to provide a unified global address space, but suffers from extreme cache-coherence overheads. A1 [151] is a distributed in-memory graph database used by Microsoft to support the Bing search engine. A1 focuses on optimizing DSM latency for graph problems over commodity DRAM by using lightweight RDMA networks. Concordia [157] proposes a network protocol to reduce the overhead of DSM cache-coherency on fast networks by offloading the coherency protocols to programmable network switches. Leap [168] discusses a kernel-level algorithm to prefetch remote memory using RDMA into the Linux page cache. HotPot [154] explores a DSM that replaces DRAM with Intel Optane DC Persistent Memory (DCPMM), but does not investigate tiering algorithms. Alternatively, Grappa [169] provides specific optimizations of shared memory for graph-based and SQL-like workloads. Currently, DSMs focus on single-tier approaches for Cloud

workloads, and have not yet explored the implications of expanding memory capacity to nodes with multiple tiers of storage.

Partitioned Global Address Space (PGAS) Languages: Alternative to DSMs, PGAS languages provide distributed applications the illusion of a global unified memory address space through the use of language features [170]. The PGAS library manages the complexity of unifying the underlying disjoint memory systems and provides abstractions to allocate memory and inform data movement strategies [171]. To improve performance, PGAS libraries provide explicit APIs to fetch remote data as needed. A number of PGAS languages have been proposed, including OpenSHMEM [172], Charm++ [173], and UPC(++) [174]. UPC++ [174], for example, uses C++ to overload the address operator to abstract the complexity of mapping pointer offsets to physical memory locations. Synchronization primitives such as locks and futures are also provided. From these primitives, more complex objects such as unordered maps and queues can be implemented. Generally, these libraries have better performance than traditional DSMs, as they avoid page fault overheads and have finer control over message granularity and scheduling. However, developers are responsible for manually partitioning datasets into the PGAS library and tracking what data is local and remote, which can increase programming complexity and force strict requirements on application structure [169]. Lastly, PGAS, libraries currently only consider remote DRAM, and have yet to explore expansion to other storage tiers such as local and remote NVMe and HDD.

Non-DSM Tiered and PMEM Storage Management: Multi-tiering has existed for decades. Many prefetching and data placement methods that target caching between low-level CPU caches and main memory [146,175] have been proposed. However, CPU caches have different hardware characteristics from storage such as Persistent Memory (PMEM) and NVMe [176,177]. More relevantly, there has been some

work on buffering between main memory and storage [3, 83, 178]. The Linux kernel provides the OS page cache, which enables I/O to be temporarily cached in main memory, while asynchronously shifting data to and from disk. However, the provided data placement and prefetching policies of the kernel only support two-tier caching (i.e., DRAM + HDD). Many PMEM filesystems [179, 180] have emerged that consider the interaction between DRAM and PMEM. While they abstract devices that are byte-addressable, the filesystems in general only support POSIX I/O and do not provide a DSM interface (nor MMIO) abstracting these devices. Other works have expanded buffering to N-tiers of storage. Hermes [3], for example, is an extensible distributed hierarchical buffering platform which transparently manages all local and remote storage tiers in order to provide increased bandwidth for HPC workloads. However, the existing data placement policies provided in Hermes focus on the I/O of high-bandwidth checkpoint-restart HPC workloads, not memory patterns in memory-centric workloads. In addition, these technologies are interfaced using I/O libraries such as POSIX and HDF5, and not through a global, byte-addressable shared memory abstraction.

Background Summary: Our extensive literature review indicates that no existing DSM system can effectively integrate memory and tiered storage to provide a unified, logical memory space for massive datasets. Existing DSM solutions focus only on local and remote memory, and do not consider intermediate storage. Solutions that provide byte-addressable abstractions over memory and storage simultaneously are only designed for single-node cases (e.g., MMIO) and cannot be directly applied to provide shared memory in distributed cases, as they do not consider the complexities of distributed memory coherence and metadata management.

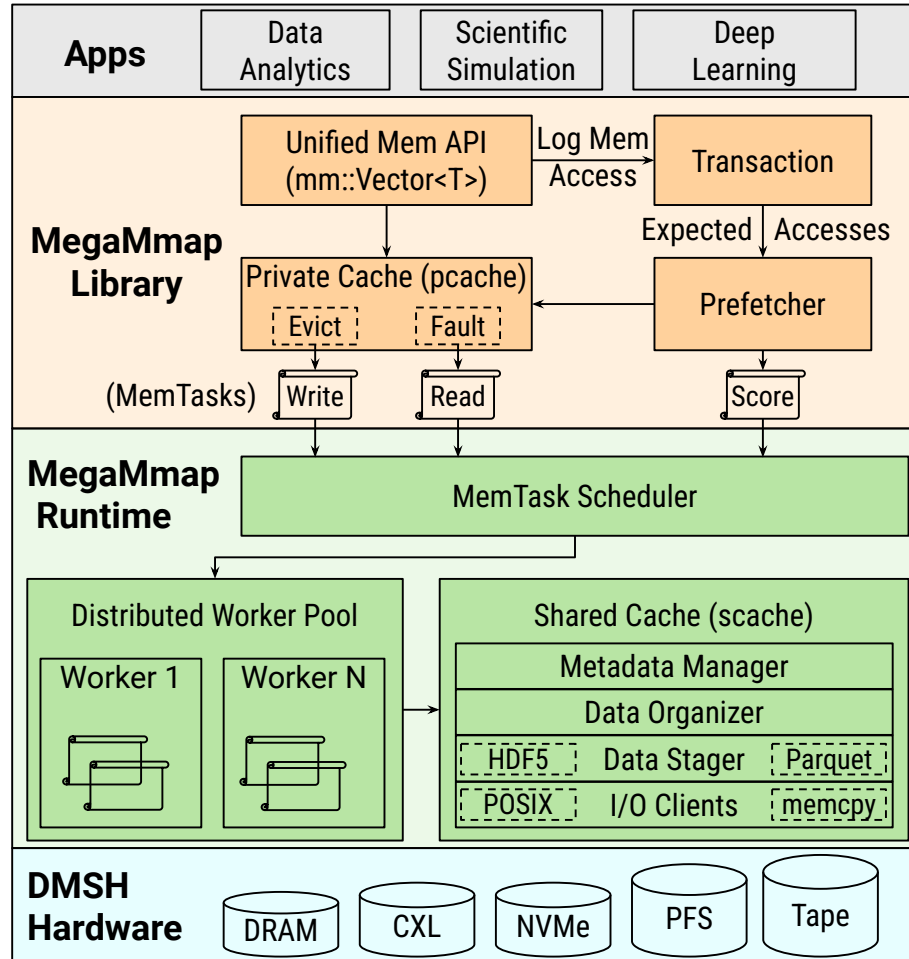


Figure 4.2. MegaMmap Architecture

4.3 Design

MegaMmap is a fully distributed and tiered shared memory designed to provide modern scientific applications an efficient, intuitive, and portable way of sharing massive amounts of data between processes while hiding the complexity of data movement, cache coherence, and resource management. MegaMmap exposes byte-addressible APIs, similar to *mmap* and *std::vector*, allowing out-of-core datasets to be produced, presented, and analyzed in what appears as an infinite main memory. These APIs internally fragment and cache application data as pages of configurable size. Applications can specify their access intentions through a transactional mem-

ory API. Several cache coherence policies are provided through the use of a hinting system, reducing communication and data movement overheads for several common access patterns experienced in data-intensive workloads. To ensure DRAM is well-utilized and I/O stalls are minimized, pages are automatically evicted, prefetched, and organized in the DMSH through extensible prefetching and data organization engines. These engines can be tuned to the specific behavior of the application to offer maximum bandwidth and latency. We provide several such policies based on the memory patterns of several common HPC machine learning and simulation algorithms. The high-level architecture of MegaMmap is depicted in Figure 4.2. MegaMmap is designed with the following objectives in mind:

1. **Intuitive:** MegaMmap should hide the complexity of data movement for memory-centric, out-of-core workloads behind an easy-to-use memory API.
2. **Heterogeneous:** MegaMmap should provide algorithms that automatically manage the organization and coherency of data in a tiered environment.
3. **Semantic:** MegaMmap should provide developers a method of describing application intent to inform data organization policies.
4. **Portable:** MegaMmap should be compatible with a variety of application structures.
5. **Persistent:** MegaMmap should ensure that data is correct and persistent after a job ends.

4.3.1 An Intuitive and Portable Nonvolatile Shared Memory Abstraction

Programming distributed out-of-core algorithms requires significant developer effort to manage the capacity constraints of main memory [181], leading many data-intensive applications to divide into distinct, alternating, and synchronous phases of

Listing 1 An example of MegaMmap for calculating inertia in KMeans. A non-volatile, read-only shared vector *pts* maps an existing file named *pts.parquet*. The vector size is the dataset size (transparently queried using the parquet library) divided by the size of *Point3D*. The vector is limited to store 1MB of data in DRAM before evictions are triggered. The elements of the vector are logically partitioned evenly among processes (*Pgas*). A read transaction is initiated, where a for loop then iterates over the dataset and calculates the sum of squared distances (inertia).

```

1  #include <mega_mmap/vector.h>
2
3  void KMeansInertia(std::vector<Point3D> &ks) {
4      int rank = mpi::get_rank();
5      int nprocs = mpi::get_comm_size();
6      mm::Vector<Point3D> pts("/points.parquet");
7      pts.BoundMemory(MEGABYTES(1));
8      pts.Pgas(rank, nprocs);
9      float distance = 0;
10     tx = pts.SeqTxBegin(
11         pts.local_off(), pts.local_size(),
12         MM_READ_ONLY);
13     for (Point3D p : tx) {
14         distance += pow(NearestCentroid(p, ks), 2);
15     }
16     pts.TxEnd();
17     return distance;
18 }
```

compute and I/O, where I/O is oftentimes the bottleneck [1,3]. This conceptual separation between memory and storage leads to the ever-increasing DRAM capacity in modern computing machines to avoid such bottlenecks [182], causing severe environmental impacts due to high energy costs [183]. MegaMmap bridges this semantic gap between memory and storage by providing unified, coherent infinite shared memory abstractions. To avoid explicit I/O phases, MegaMmap provides a nonvolatile memory abstraction that transparently synchronizes data to arbitrary storage backends, such as a file on a PFS. Applications can inform MegaMmap of expected behavioral patterns through the use of a comprehensive transactional memory hinting API to optimize cache coherence and data movement policies. This section provides details on the specifics of the abstractions provided by MegaMmap.

An Infinite Shared Memory Abstraction: To give applications the illusion of a large memory, MegaMmap implements a shared memory vector API, providing implementations of several functions and operators including array index, memory copy, acquiring current size, appending, resizing, and destroying the data container. Processes connect to the shared vector using a semantic, user-defined key common to all processes. The vector API is highly versatile and allows more complex distributed data structures, such as matrices, logs, and multi-dimensional arrays, to be developed using simple offset calculations and appends. Through C++ templating, MegaMmap can theoretically store any type of data – including complex C++ classes, so long as a serialization method is provided. Unlike standard C++ vectors, MegaMmap shared vectors are not destroyed in the destructor – users must explicitly destroy them. This is to avoid the race condition where processes finish at separate times. Shared vectors can be either volatile or nonvolatile, where volatile vectors store temporary data that is difficult to fit in memory and nonvolatile vectors represent data that should eventually be persisted to storage for future use.

Presenting Persistent Datasets as Memory: Data-intensive applications either produce or analyze a persistent dataset that cannot fit trivially in memory. To reduce the burden of out-of-core programming for persistent data, MegaMmap allows nonvolatile shared vectors to be created with a backing persistent object (e.g., a file on parallel filesystem). In this case, the key of the vector is structured as a URL (i.e., “protocol://URI:params”), where all information needed to read and write the object, including factors such as identifiers and port number are provided. This format is versatile, allowing MegaMmap to represent a variety of data formats (e.g., HDF5) and repositories (e.g., Amazon S3) without requiring the application to commit to any particular library’s semantics. For example, an HDF5 group can be represented with the URL *hdf5:://path/to/df.h5:mygroup*. Alternatively, multiple data objects, such as files produced in a file-per-process HPC simulation, can be mapped

as a single uniform vector via a regex query such as *file:///path/to/dataset.parquet**.

Enabling Resource Utilization Control: Balancing memory constraints typically requires programmers to be cognizant of the amount of data being allocated and the persistence requirements of the data, which is particularly challenging in out-of-core programs. MegaMmap provides explicit, fine-grained control over memory and storage capacity constraints. Applications can specify the maximum amount of DRAM and high-performance storage to use for caching using either the native C++ API or the MegaMmap configuration YAML file, which additionally contains settings regarding the nodes to deploy MegaMmap on, port numbers, etc. When the capacity constraints are reached, MegaMmap will automatically and transparently evict and reorganize pages in the DMSH.

Supporting Arbitrary Application Structures: There are many ways to structure distributed applications, such as parallel message passing and MapReduce. To remain compatible with any application structure, MegaMmap provides several synchronization options to ensure parallel application correctness. This includes distributed locks and barriers. These mechanisms can also be supplied by means of a higher-level library, such as MPI and UPC++.

Informing Policy with Transactional Memory: In order for memory tiering to perform well, applications must exhibit predictable behavior and convey that behavior to the underlying prefetching, eviction, and coherence algorithms. Transactions are used to inform MegaMmap of the access pattern a region of shared memory is about to incur and when it has completed within a particular process, accomplished through the transaction begin (*TxBegin*) and transaction end (*TxEnd*) methods. The *TxBegin* API includes hints on access method, such as read, write, and append. It also includes the indices that will be accessed. For sequential accesses, the offset and size of memory can be used. Modifications made between *TxBegin* and *TxEnd* are

Listing 2 The transaction base class. *GetPages* predicts the regions of a page that will be accessed based on the number of memory accesses encountered (*tail*). *head* represents the number of memory accesses acknowledged by the prefetching algorithm.

```
struct PageRegion {
    size_t page_idx, off, size;
    bool modified_;
};

class Transaction {
    bitfield32_t flags;
    size_t head, tail;

    virtual std::vector<PageRegion>
    GetPages(size_t off, size_t count) = 0;

    auto GetTouchedPages() {
        return GetPages(head, tail - head);
    }
    auto GetFuturePages(size_t count) {
        return GetPages(tail, count);
    }
};
```

not required to be visible to other processes immediately. TxEnd commits all unflushed data that was not automatically flushed. Users can develop their own custom transactions by inheriting from the transaction class (shown in Listing 2) and using a templated *TxBegin* method. With this knowledge in advance, prefetching, eviction, reorganization, and coherence algorithms can be far more effective.

Use Case - KMeans: KMeans is a read-intensive clustering algorithm, where each process is given a non-overlapping subset of a dataset. In parallel, each training iteration locates the centroid nearest to a sample point in terms of a given distance metric (e.g., euclidian) and calculates inertia by summing the calculated distances. An example of how to apply MegaMmap to this workload is shown in Listing 1.

Use Case - Random Forest (RF): RF is a mixed workload, where each iteration builds a decision tree based on a random subset of features and samples. This practice is known as out-of-order bagging. Each process of RF randomly subsets a subsample

of the whole dataset with replacement and decides the split point of the decision tree based on the entropy of each feature. The dataset is then divided into left and right samples based on the split point.

4.3.2 Actively Ensuring Data Persistence and Consistency

Storage accelerators in HPC sites are ephemeral, ending when the application's job ends. In order to ensure that data remains persistent, all cached data must be flushed out of these devices into a persistent backend, such as a parallel filesystem. In addition, data may be expected in a particular format, such as HDF5 or NetCDF. This requires data stored in cache to be converted into the application's desired format. To ensure data remains persistent after the job ends, MegaMmap actively flushes modified data to storage during periods of computation. This reduces the cost of explicit synchronization functions. MegaMmap also ensures that data is stored in its proper format by transparently and asynchronously staging data in and out of the backing storage. This section provides details on the overall lifecycle of memory pages in MegaMmap and the high-level architecture in Figure 4.2.

Distributed Heterogeneous Caching Structure: There are two page caches in MegaMmap: the *Private Cache (pcache)* and *Shared Cache (scache)*. The pcache is a DRAM-only cache of configurable maximum size that is stored per-process, while the scache is distributed, tiered, and coherent across multiple processes. Each application process is linked to the MegaMmap library, which internally stores the pcache and a queue for submitting *MemoryTasks* to the MegaMmap runtime, which is a process running separate from applications that manages the scache. The runtime can dedicate a configurable maximum number of CPU cores and dynamically adjusts the number of cores based on experienced load using an approach similar to LabStor [1]. When the application reads or writes to MegaMmap, it first goes through the pcache. If a memory page is not currently cached, a page fault will occur. If space is needed

to service the page fault, an eviction will occur. Users can also explicitly flush the content of pages to ensure modifications are visible to other processes. During page fault, eviction, and flushing operations, the MegaMmap library constructs a `MemoryTask` that contains the subset of a page to read or update from the scache. The task will be placed in the queue and polled by the runtime, which will then be scheduled to a worker and executed.

Strong Consistency through MemoryTask Scheduling: MegaMmap allows distributed processes to interact with potentially the same data concurrently. It is possible that multiple processes intend to modify the same parts of data simultaneously. To provide **strong consistency** (i.e., read-after-write guarantees), the runtime's *Scheduler* ensures `MemoryTasks` for the same page are hashed to the same worker. To improve performance, MegaMmap allows reader tasks to proceed in parallel until a writer task is encountered. Writes can be executed concurrently if they are to non-overlapping subsets of the page. To further improve performance, the MegaMmap scheduler divides workers into low-latency and high-latency groups. `MemoryTasks` containing less than 16KB of data will be sent to low-latency workers, which are scheduled on different CPU cores from high-latency workers. This is to ensure that latency-sensitive requests are not stalled by large requests, which has been shown to improve quality-of-service in several applications [1].

Lifecycle of Modified Data: MegaMmap transactions mark a period of time when modifications made to memory do not need to be visible to other processes immediately. For this reason, MegaMmap uses copy-on-write semantics for handling memory modifications. Processes write to their local pcache first and have their own view of data. During the end of a transaction, an eviction, or an explicit user-driven call to flush, the process's local modifications are made visible to other processes. Since transactions store the exact memory accesses made, only the bits of the page that

were modified during a transaction will be a part of the writer MemoryTask operation. This reduces I/O amplification and improves data correctness, since stale data will not be included in the MemoryTask operation updates. To further improve performance, writer MemoryTasks are constructed with a copy of the modifications and are then executed in a non-blocking, asynchronous fashion. During an eviction, the application will only experience the performance cost of a memory copy, instead of metadata updates, networking operations, and potentially an I/O operation. When the writer MemoryTask is scheduled on one of the Runtime’s workers, the data organization engine will be invoked to decide the placement of data in the DMSH.

Read, Page Fault, and Prefetcher Path: When reading data from MegaMmap, either the data is present in the pcache or must be fetched from the scache. Read operations first check the pcache for existence. If a page is not present in the pcache, a synchronous page fault occurs, which constructs a read MemoryTask and submits to the runtime. To reduce the frequency and cost of page faults, the MegaMmap library also provides a lightweight *prefetcher*, which constructs asynchronous score MemoryTasks based on the nature of the active memory transaction. The importance score is a number between 0 and 1 representing the priority of a memory page to a particular process. These scores are sent to the Data Organizer in the runtime, which determines the composition of data in the DMSH. Pages with higher scores will be elevated to high-performance tiers, while pages with lower scores will be evicted to lower tiers. The Data Organizer will take the maximum of scores if several processes score the same page within a configurable timeframe.

Persistently Integrating Memory with Storage: To improve code reuse and portability, MegaMmap allows arbitrary datasets to be presented as shared vectors, where the vector’s name is the URL of the data to load in storage (e.g., a path on a POSIX filesystem). This requires data to be transparently integrated with a

particular backend before a job ends. To accomplish this, the *Data Stager* is responsible for serializing, deserializing, and flushing content to the backend. The stager is an extensible component containing integrations with widely-used file formats (e.g., HDF5, Adios2, parquet) and storage services (e.g., PFS, Amazon S3). When an application initially constructs a persistent MegaMmap vector, the key of the vector will indicate the type of staging service to use through a URL format. For example, “hdf5:///path/to/df.h5:mygroup” will ensure that pages which are part of the df.h5 vector will be sent to the HDF5 stager. Periodically and during the termination of the runtime, the stager task will be scheduled to serialize pages in the scache and persist them. During a page fault, if a page is not present in the scache, the stager will be invoked to read and deserialize a subset of data from the persistent backend. These (de)serialization functions allow MegaMmap to transparently load content from storage in the format applications expect to operate on, without requiring specific linkage or semantics of the I/O library.

4.3.3 Efficient Memory Coherency

Cache coherence (i.e., two processes accessing the same memory should have equivalent data views) has been widely studied in the context of persistent distributed data structures. There are several workload patterns and use cases where optimizations can be made to reduce communication, synchronization, and data movement. Traditional DSMs focus on coherence to handle small random reads and writes to local and remote DRAM housing temporary content. These algorithms assume that applications accessing data are uncoordinated, in addition to requiring specific page sizes (4KB, 2MB, and 1GB) due to physical restrictions imposed by the CPU. This results in poor I/O access patterns, increased communications, and significant software overheads. MegaMmap supports reads and writes to both temporary and persistent data fragmented among multiple tiers of memory and storage. To offer high perfor-

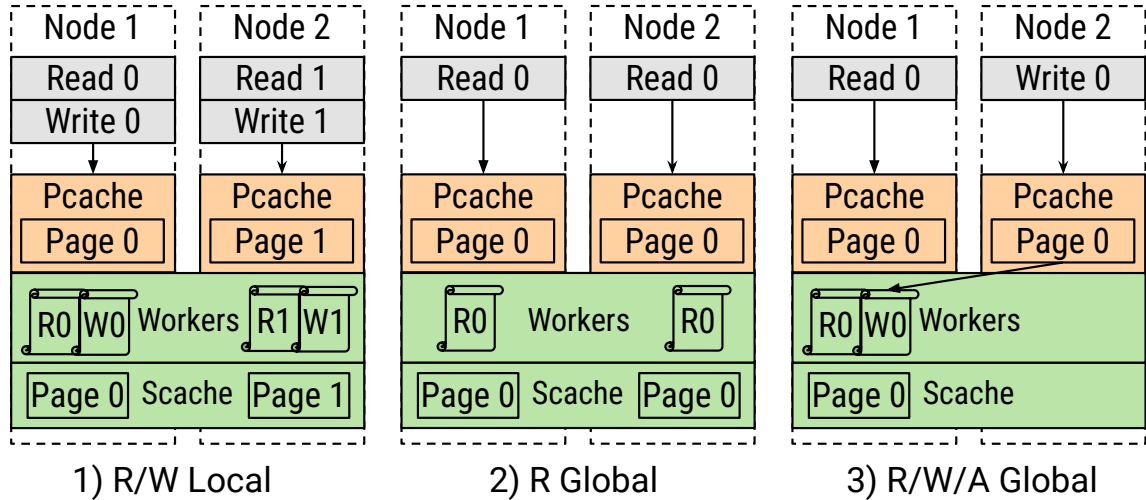


Figure 4.3. Overview of coherence policies. Read (R) / write (W) to non-overlapping regions will first place data locally on the node that produced it without synchronization. Read-only will allow for data to be replicated in the shared cache to improve availability. Read, write, append global will hash page faults and evictions to the same worker.

mance for a variety of HPC workloads, MegaMmap supports workload-specific page granularity configuration, partial paging, and configurable coherency optimizations depending on access pattern and coordination. This section discusses the coherence algorithms used for the pcache and scache.

Coherency Optimizations with Transactional Memory: Modern DSMs are built to be correct for chaotic access patterns and multi-tenancy, but offer limited optimization for applications with well-defined behavior. Shared data structures in parallel programs are oftentimes divided into distinct phases of read-only, write-only, or append-only, where each process owns a non-overlapping partition of the shared memory for its lifetime. This well-defined nature presents opportunity to eliminate communication overheads and reduce data movements. While coherence in MegaMmap is mainly the responsibility of the application programmer using synchronization points such as barriers and locks (similar to any MPI or PGAS program), transactions further allow programmers to capture various common access patterns between these

points (shown in Figure 4.3) via transactions to optimize communication and data movement costs.

Read / Write Local: In this case, all processes interact with nonoverlapping regions of the DSM. By nature, the caches will be coherent across machines for both operations. During mutable array index operations, the transaction will track the set of pages that were modified. During evictions or flushing operations, the page modifications will be copied to the runtime and processed. Only the parts of the page that were modified will be transferred, ensuring that two processes modifying different parts of the same page do not conflict. An example of this access pattern is matrix transpose and multiplication. These operations are embarrassingly parallel, but are oftentimes intermediate steps in a more complex program that may require a different access pattern in the near future.

Read Only Global: This is common for analysis programs such as machine learning and deep learning. In this case, data is never modified, and no process connected to the shared memory will modify the data. In this case, MegaMmap allows the data to be replicated in both the pcache and scache. This improves data availability and reduces I/O and networking costs by allowing data to be placed near the process requiring the data.

Write and Append Only Global: This is common in many programs, including analysis, inference, and simulation. Gray-Scott [142], for example, is a simulation that models the reaction between chemicals that iteratively modifies concentration values in a 3-D grid. Alternatively, in DBSCAN [184], a k-d tree is created by appending samples to the left and right branches based on a split point. In these cases, MegaMmap leverages properties of the runtime’s scheduling system to ensure consistency. During page eviction, MemoryTasks are constructed and scheduled in the runtime. The runtime internally ensures the tasks are ordered and that the data

operations defined in the MemoryTask run to completion before the next begins.

Read, Write, and Append Global: This case is less common in HPC, but comes up frequently in Cloud workloads. A key-value store, for example, may incur writes and reads to the same region simultaneously. MegaMmap handles this case the same as the write/append case. When the MegaMmap transaction spans only a single page, strong consistency and atomicity are guaranteed across simultaneous reads and modifications since MemoryTasks will be hashed to the same worker and processed in sequence. Transactions spanning across several pages will have to use synchronization primitives, including locks and barriers, to guarantee correctness – although in many cases this explicit synchronization can be avoided by simply setting a large enough page size.

Collective: In cases where a memory region is going to be accessed by several processes, any of the above access patterns can be marked as collective to allow MegaMmap to improve scalability. A communicator can be passed indicating the set of processes accessing the region. Memory accesses will follow a tree-based pattern to avoid overloading a single node, similar to allgather operations in MPICH [185, 186].

Changing Phases: A MegaMmap vector’s access pattern may change throughout the lifetime of a program. For example, in DBSCAN, the k-d tree construction algorithm first divides a root sample by a split point into left and right append-only samples. After all processes have finished splitting their root sample, the left and right samples are then only ever read from. MegaMmap supports phases through synchronization primitives such as barriers. If a region changes from read-only to write-only, all replicas produced during reads will be invalidated.

Reducing Data Movement Through Configurable and Partial Paging: Fixed pages sizes are restrictive, and can result in I/O amplification if the page size is too

large or poor access patterns if the page size is too small. Users can choose a custom page size for a particular MegaMmap vector. Vectors do not need to have the same page sizes, but the page size for a particular vector will be equivalent across processes and immutable after the creation of the vector. In addition, since transactions indicate the exact regions of memory that will be accessed, it is possible to know in advance the regions of the page that will be accessed. This allows MegaMmap pages to contain only the fragments of data needed during a page fault.

4.3.4 Masking I/O Stalls with Informed Tiered Data Movement Policies

Modern memory-centric HPC and Cloud workloads, including simulation and machine learning, are bottlenecked by the cost of data movement speeds resulting from both the Memory Wall [146] and I/O Bottleneck [3]. Traditional DSMs do not consider the complexity of tiering and require a large, expensive data copy in order to persist data. While DMSH tiering policies have been provided in I/O buffering platforms, they are tailored to workloads where I/O and memory are treated as separate and are typically optimized for bursty producer-only [3, 187] and repetitive read workloads [178] by leveraging statistics and machine learning to guess I/O intention. Unlike existing tiering methodologies, MegaMmap incentivizes applications to specify their access intent through a transactional memory API, offering the opportunity to provide well-informed data placement decisions. While this may appear as an imposition to users, many distributed and out-of-core algorithms are already structured to accomplish this, as they typically operate over large, well-defined subsets of out-of-core data between synchronization points, such as barriers and locks [154].

Data Prefetching: With advance knowledge of the subsets of data being accessed on each iteration, significant performance and resource utilization benefits can be made. In Algorithm 2, MegaMmap’s prefetcher is shown. The prefetcher takes as input the maximum amount of memory that the MegaMmap’d region can occupy ($Vec.Max$),

the current amount of data in the region ($Vec.Cur$), the page size ($Vec.PageSize$), the minimum score that can be assigned ($MinScore$), and the active transaction object (Tx). The transaction object represents the expected access pattern of the memory region. Whenever a memory access to a MegaMmap vector is made, the transaction tail ($Tx.Tail$) is incremented. $Tx.Head$ represents the current number of memory accesses that have been acknowledged by the prefetcher.

When invoked, the prefetcher determines the set of pages that can be evicted. This is done by determining the set of pages that will not be accessed in the near future. First, all pages that have already been touched (pages between $Tx.Head$ and $Tx.Tail$) will be marked with a score of 0. All pages that are expected to be touched and could be stored in the vector if it were empty (pages between $Tx.Tail$ and $Tx.Tail + Vec.Max/Vec.PageSize$) will be marked with a score of 1. Any page with a score of zero between $Tx.Head$ and $Tx.Tail$ will be evicted. Note that certain transactions (e.g., random) may touch a page several times. The scores between $Tx.Head$ and $Tx.Tail$ may not be 0 if a page is expected to be retouched.

After this, the prefetcher will begin scoring future pages. Pages that can fit in the current amount of space available in the pcache will be marked with a score of 1 (pages between $Tx.Tail$ and $Tx.Tail + (Vec.Max - Vec.Cur)/Vec.PageSize$). All pages that will be accessed soon, but cannot fit in the pcache, will be marked with a score proportional to the minimum amount of time before a page fault could occur. This is calculated by summing the amount of time it theoretically would take to read a page from the scache considering the bandwidth of the tier it is currently located on. When setting the score of a page, the ID of the node setting the score is also propagated to improve the locality of data.

Data Organization: The Data Organizer is responsible for interpreting the scores supplied by the prefetcher. Score updates to the same page will all be hashed to the

same worker. Periodically (configurable by the user) the Data Organizer interprets the scores and determines the node and tier where data should be placed. Each tier is assigned a score based on its performance characteristics, where tiers with a score closer to 1 have high I/O performance. The organizer will first attempt to place pages in the fastest tiers if there is available capacity. Pages with lower scores in a tier will be prioritized for eviction to make space for higher-scoring data. If a node sets a high score for a page, the organizer will store the page on that node.

4.3.5 Implementation Details

MegaMmap was implemented in 5K lines of C++ code. MegaMmap utilizes Hermes [3], which is a hierarchical buffering platform, to provide basic infrastructure for enacting data movement policies and provide metadata management to locate data in the DMSH. We extended Hermes to support low-latency I/O NVMe devices through the SPDK [57] and traditional libc *mmap* and *memcpy* for upcoming CXL devices. In addition, we augment Hermes to have an extensible data staging layer and support staging for several I/O libraries, including HDF5 1.14 [188] and parquet [189]. Hermes uses Mochi thallium [190] to support low-latency RDMA transfers.

Minimizing Indexing Overhead: To avoid hashtable lookups on every memory access, the page that was last accessed is checked first. This is because many algorithms iterate over a page in its entirety. On average, reading from MegaMmap vectors adds two integer operations and a conditional statement as overhead to a typical memory access (`std::vector`). We found that this overhead is minor ($\approx 5\%$) compared to a typical memory access in an iterative workload that multiplies a matrix by a scalar.

4.4 Evaluations

4.4.1 Experimental Methodology

Algorithm 2 Private Cache Prefetcher

```

1: function PREFETCHER(Vec, Tx, MinScore)
2:   Evict(Vec, Tx)
3:   Prefetch(Vec, Tx, MinScore)
4:   Tx.Head = Tx.Tail
5: end function
6: function EVICT(Vec, Tx)
7:    $N = Vec.Max / Vec.PageSize$ 
8:   for Page in Tx[Tx.Head, Tx.Tail) do
9:     Page.SetScore(0.0, Vec.NodeId)
10:  end for
11:  for Page in Tx[Tx.Tail, Tx.Tail + N) do
12:    Page.SetScore(1.0, Vec.NodeId)
13:  end for
14:  EvictIfZeroScore(Tx[Tx.Head, Tx.Tail))
15: end function
16: function PREFETCH(Vec, Tx, MinScore)
17:   BaseTime = 0
18:    $N = (Vec.Max - Vec.Cur) / Vec.PageSize$ 
19:   for Page in Tx[Tx.Tail, Tx.Tail + N) do
20:     T = Page.GetTier()
21:     BaseTime + = Page.GetSize() / T.BW
22:   end for
23:   EstTime = BaseTime
24:   Score = 1.0
25:   while Score > MinScore do
26:     Page = Tx[Tx.Tail + N]
27:     T = Page.GetTier()
28:     EstTime + = Page.GetSize() / T.BW
29:     Score = EstTime / BaseTime
30:     Page.SetScore(Score, Vec.NodeId)
31:     N = N + 1
32:   end while
33: end function

```

4.4.1.1 Hardware All tests were conducted on a research cluster, designed to support a hierarchical storage architecture. The cluster consists of storage and a compute rack, each having 32 nodes. The two racks are interconnected by two isolated Ethernet networks (one of 40Gb/s and the other 10Gb/s), with RoCE enabled. Each compute node has a dual Intel(R) Xeon Scalable Silver 4114 with 24 cores and 48 threads, 48 GB RAM, 128GB NVMe PCIe x8 drive, 256GB SSD drive, and 1TB HDD.

4.4.1.2 Software In several of our experiments, we use the MLlib library of Apache Spark 3.4.1 [156] as a comparison. Spark is largely designed for Cloud environments, where fault tolerance is a primary design goal. To make our comparisons fair, we disable all fault tolerance features to ensure no unnecessary data replication occurs. For MPI-based applications, we use mpich 3.4.3. We do not compare against published DSM solutions such as Grappa, HotPot, and Microsoft A1. This is because Grappa is now deprecated, A1 is not publicly available, and HotPot is designed assuming a particular hardware and software configuration incompatible with our available testbed, including a specific network fabric, drivers, and Linux 3.11.0. We run each experiment 3 times and report the average.

To demonstrate the performance and programmability of MegaMmap, we implemented and verified several real-world machine learning and simulation applications. Each algorithm was verified by comparing their outputs on several datasets to their published counterparts. We measure code volume in terms of LOC using `cloc` [191], which ignores visual spaces and comments. We also provide an overview of the behavior of these algorithms in terms of their data movement patterns. Each of these algorithms can be parallelized using an arbitrary number of processes p .

	KMeans	RF	DBSCAN	Gray-Scott
MegaMmap	589 C++	403 C++	748 C++	282 C++
Original	1009 Scala/Py	710 Scala/Py	1518 C++	407 C++

Figure 4.4. MegaMmap code 45% - 2x smaller. In each case, all I/O partitioning, I/O compatability, and most messaging is removed.

KMeans: KMeans is a foundational and widely-used clustering algorithm that groups data into k spherical clusters. We implement a custom version of KMeans|| [192], which is the same algorithm used in Apache Spark [193]. KMeans|| initially divides the dataset evenly among each process and then performs several sequential, read-only iterations over the dataset to determine the initial and final centroids, resulting

in approximately $2 * \log(n)$ iterations for the entire algorithm to converge. After this, data points are assigned to centroids. The assignments are persisted automatically using a file-backed MegaMmap.

Density-based Scanning (DBSCAN): DBSCAN is another important clustering algorithm that identifies arbitrarily-shaped clusters by separating points based on a given distance ϵ . We implement custom versions of μ DBSCAN [184]. Initially, DBSCAN constructs a k-d tree, which is a decision tree that splits the dataset by axis. At the first iteration, the dataset is split evenly among the processes. The median and entropy is estimated per-axis using a small, random subsample. The axis with the largest entropy is chosen, and each process divides the dataset into two fractions: left and right of the median. Processes are then partitioned to handle the subsets. This recursion terminates when all points in the subsample have a distance within ϵ of the median or when the sample is smaller than *min_pts*. Now that each point belongs to a μ cluster (set of points in a leaf), the μ clusters can be merged in parallel to form the full clusters, which are then persisted.

Random Forest: Random Forests (RFs) are commonly used by data scientists for modeling complex, nonlinear associations in data and providing an indication of feature importance for dimensionality reduction techniques [194]. We implement a custom version of RF. Initially, each process performs out-of-order bagging (oob) on $N/(oob * p)$ randomly-selected samples, where N is the overall dataset size and *oob* is the number of bags to generate. Each oob iteration measures the entropy (Gini impurity) of each feature in a chosen feature subset. The per-process oob results are then aggregated to find the feature maximizing entropy. A point is then randomly selected from the dataset and used as the split point. The dataset and processes are then divided into two partitions: left and right. The recursion continues until either the maximum depth (*max_depth*) of the tree is reached or the entropy difference is

below a threshold. The algorithm repeats *num_trees* times.

Gray-Scott: The Gray-Scott reaction-diffusion model [100] is a system of partial differential equations that captures the dynamic interactions between two chemical species U and V , and holds a pivotal position in the realms of biology, chemistry, and physics. We implement a custom version of Gray-Scott [100] in 300 lines of code. Initially, a grid of volume L^3 is defined and evenly subdivided among each process. Each cell in the grid contains the concentrations of U and V at time step T . At each iteration, the concentrations are updated and exchanged between processes, transferring approximately $24 * (L/p)^2$ bytes of data per process. After a certain number of iterations (*plotgap*), the grid of size $O(L^3)$ is checkpointed. The algorithm repeats *steps* times.

4.4.1.3 Datasets To evaluate the performance of KMeans and DBSCAN, we use datasets produced by Gadget 4 [195], which is a parallel cosmological N-body and SPH code that simulates cosmic structure formations and calculations relevant for galaxy evolution and galactic dynamics. Several works cluster and model the outputs produced by this simulation to locate halo formations [184, 196] and identify the importance of cosmological features. We include the exact parameters in the AD appendix. The final output data is stored in an HDF5 file containing 3D floating-point particle positions and velocities. To evaluate Random Forest, these values are taken as input and used to predict output clusters. 80% of the original dataset was used for training and 20% for testing using a stratified random sampling that takes 80% of each cluster.

4.4.1.4 Objectives This evaluation plan aims to demonstrate :

1. Memory coherence of DSMs are not a scalability bottleneck compared to leading HPC+Cloud communication solutions, such as MPI and Spark.
2. Tiering memory can increase the resolution of scientific datasets by eliminating memory constraints, allowing more detail in the final simulation data.
3. Intelligently tiering memory can bring performance benefits to out-of-core algorithms.
4. DRAM consumption can be lowered by offloading memory to tiered storage.

4.4.2 Experimental Results

4.4.2.1 Scalability of DSMs for HPC In this evaluation, we demonstrate the scalability of MegaMmap for in-memory workloads compared to alternative approaches. This exercises the basic communication, coherence, and latency overheads of MegaMmap. To do this, we perform a weak scaling study that compares MegaMmap-based algorithms to the algorithms in the original work. All tests use datasets that allow competing algorithms to maintain all data (including copies) entirely in DRAM. In these evaluations, MegaMmap is configured with no optimizations enabled and only uses memory. For KMeans, we use a dataset of size $2GB$ per node, $k = 8$ with a maximum of 4 iterations. We use the same dataset for DBSCAN and use $\epsilon = 8$ and $min_pts = 64$. For Random forest, we use a dataset of size $128M$ per node and produce 1 decision tree with $max_depth = 10$. Gray-Scott is configured to produce 16GB of data per node ($L = 784$ for 1 node, $L = 1920$ for 16 nodes) and does not perform checkpointing ($plotgap = 0$). For each algorithm, we run 48 processes (or threads) per node, for a maximum of 768 processes.

Overall, from Figure 4.5, it can be observed that MegaMmap-based algorithms

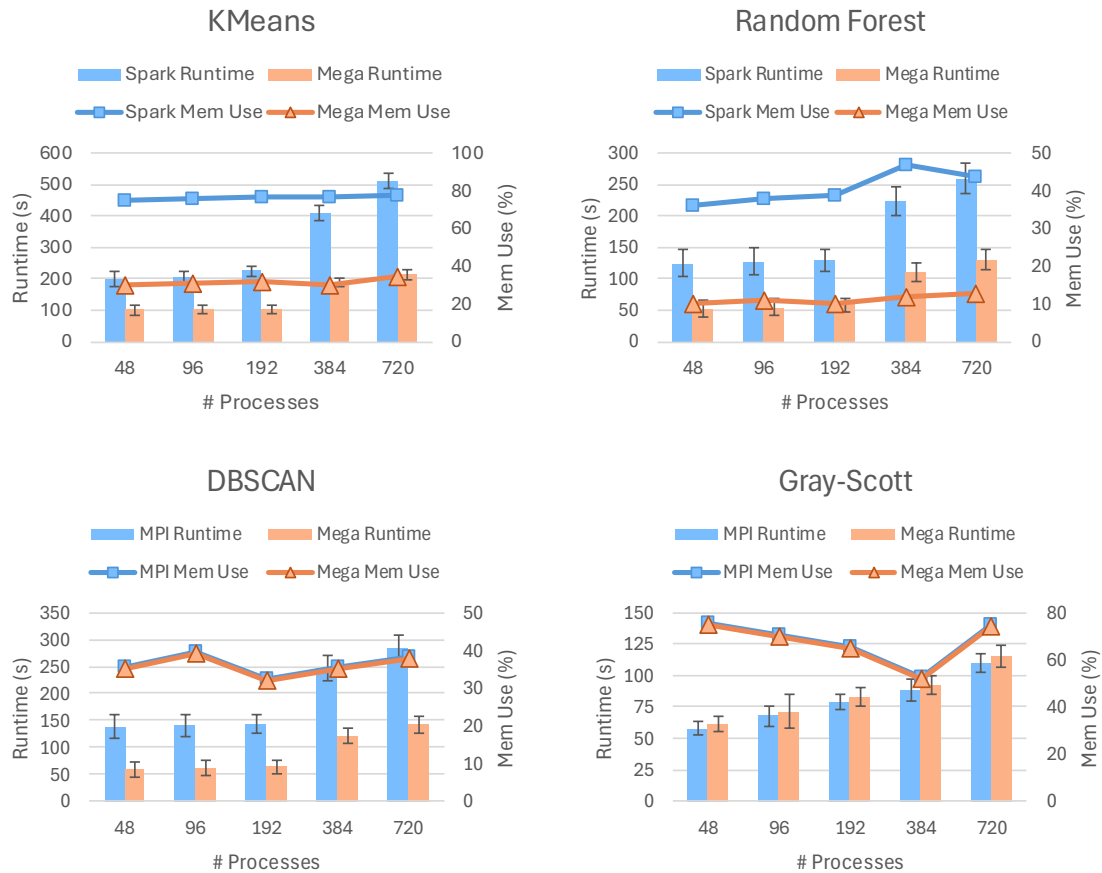


Figure 4.5. Weak scaling study of MegaMmap vs alternative application designs, where datasets fit entirely in memory. MegaMmap performs competitively to MPI and as much as 2x faster than Spark.

scale and perform competitively to state-of-practice parallel programming approaches. For Spark, performance was as much as 2x faster. This was partially due to implementation details, including its use of the slower TCP protocol and Java Runtime. However, more importantly, Spark creates several copies of the dataset when initially loading data from the backend and during the map/reduce phases. We also found that Spark used 3-4x the amount of DRAM than MegaMmap-based algorithms. For DBSCAN and Gray-Scott, MegaMmap performs similarly to the MPI-based implementation showcasing the scalability of MegaMmap's memory coherence algorithms. Both DBSCAN and MegaMmap stage the datasets entirely in DRAM at the begin-

ning of the program and largely reuse the same portions of the dataset, so there is little benefit to prefetching. Gray-Scott performs synchronous memory-to-memory communication, which MPI excels at and MegaMmap matches. This is because the coherence algorithms in MegaMmap do not spawn flurries of latency-sensitive requests for replica invalidation like traditional DSMs [153]. While DSMs have been traditionally discarded for abstracting distributed memory in HPC due to coherence overheads, it can be seen that MegaMmap’s coherence policies offer competitive performance to modern HPC and Cloud approaches for programming distributed memory systems.

4.4.2.2 Increasing Dataset Resolution

In this evaluation, we demonstrate the impact of tiering memory and storage for increasing dataset resolution, measured by the total dataset size. To do this, we run Gray-Scott to produce grids of varying size. We vary the grid size between $L = 2048$ and $L = 3456$. We compare the MPI-based implementation for various I/O backends (OrangeFS [16], tiered filesystem Assise [179], and tiered I/O buffering system Hermes [3]) vs MegaMmap in terms of dataset size and memory utilization. $L = 2048$ produces a 38GB dataset (2.5GB per node), while $L = 3456$ produces a 1.5TB dataset (96GB per node). For MegaMmap, the tiers are configured such that there is 48GB of DRAM and 128GB of NVMe per node, which fits the entire dataset at a scale of $L = 3456$.

Overall, from Figure 4.6, it can be seen that MegaMmap performs at least 20% faster than all versions of Gray Scott until $L = 2688$. This is because MegaMmap places data during the first compute phase, while all others must wait for this phase to complete. Additionally, MegaMmap’s data movement algorithms have advanced knowledge of memory access patterns, leading to improved placement decisions. After $L = 2688$, memory is fully utilized. For the MPI-based version, the default behavior of Linux is to terminate programs overutilizing memory, so $L = 2688$ is

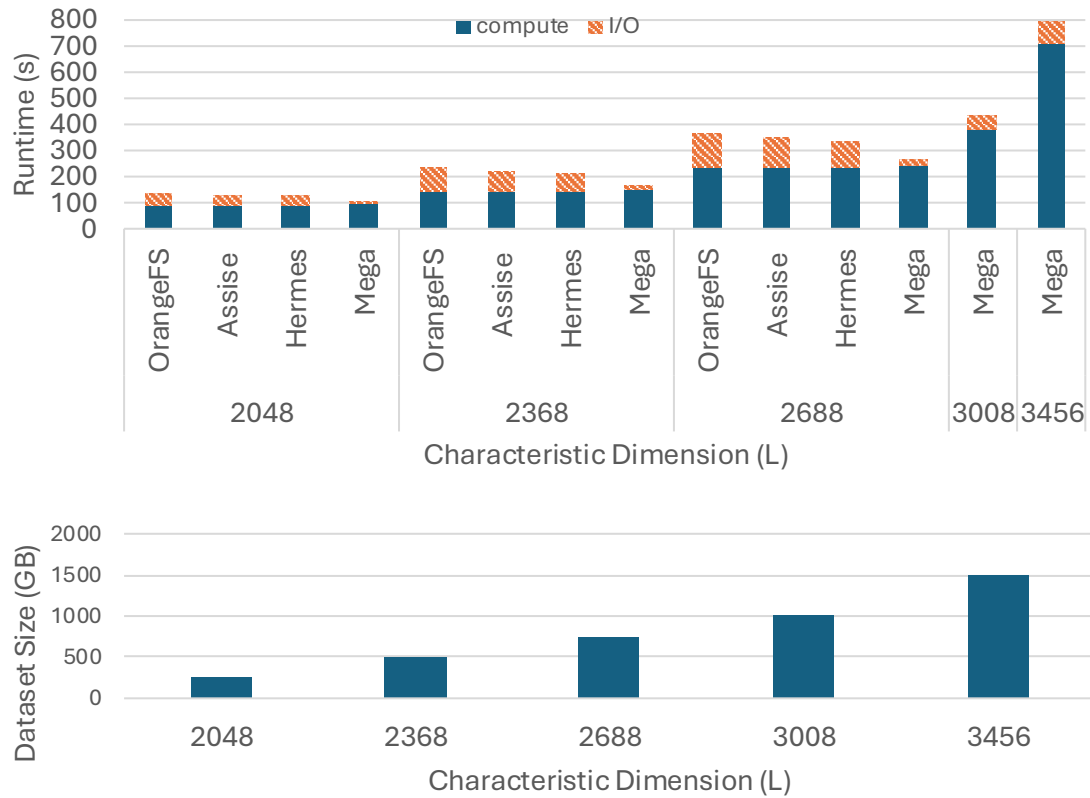


Figure 4.6. Increasing the resolution of Gray-Scott through tiering. After $L = 2688$, MPI-based Gray-Scott crashes due to memory overutilization. MegaMmap is unbounded, and can run simulations as large as $L = 3456$, producing 2x the simulation data of $L = 2688$. It's also at least 20% faster than other tiered I/O systems due to effective asynchronous data movement.

the point at which science can no longer progress. However, through MegaMmap, we are able to conduct experiments as high as $L = 3456$, resulting in over 2x the information of $L = 2688$ and double the capacity of main memory. This is because MegaMmap leverages the node-local NVMeS to enlarge effective memory capacity. Currently, state-of-practice solutions limit science to the boundaries of main memory. MegaMmap eliminates this boundary by automatically tiering memory with storage, allowing memory-constrained scientific problems to become feasible.

4.4.2.3 Performance and Cost Benefits of Persistent Tiered Memory

In this evaluation, we demonstrate the performance impacts of expanding the memory hierarchy to storage for persistent datasets that do not fit trivially in DRAM. To do this, we run Gray-Scott to generate an out-of-core grid. We set $L=3456$, the maximum size used in the previous experiment, and run 768 processes (16 nodes). The grid size is a total of 1.5TB (96GB per node) and is flushed every step (*plotgap* = 1). We run 5 steps, generating a total of 8TB of data (480GB per node). We compare various compositions of the DMSH for handling the placement of this dataset, spanning between NVMe, SSD, and HDDs. We also measure the financial cost of tiering strategies by multiplying utilized storage by \$/GB, which we estimate from retailers such as Amazon. We found that HDDs are roughly .02 \$/GB, SATA SSDs are .04\$/GB, and NVMeS are .08\$/GB.

Overall, from Figure 4.7, it can be observed that deepening the memory hierarchy to include storage tiers can greatly improve overall application performance. In this case, Gray-Scott performs a write-intensive workload, where a large grid is modified and persisted on every step. MegaMmap overlaps computation with data movement leveraging the asynchronous behavior of the write/append only cache coherence policy when modifying the grid and the asynchronous data staging engine when persisting the data during checkpoints. Individual data movements are accel-

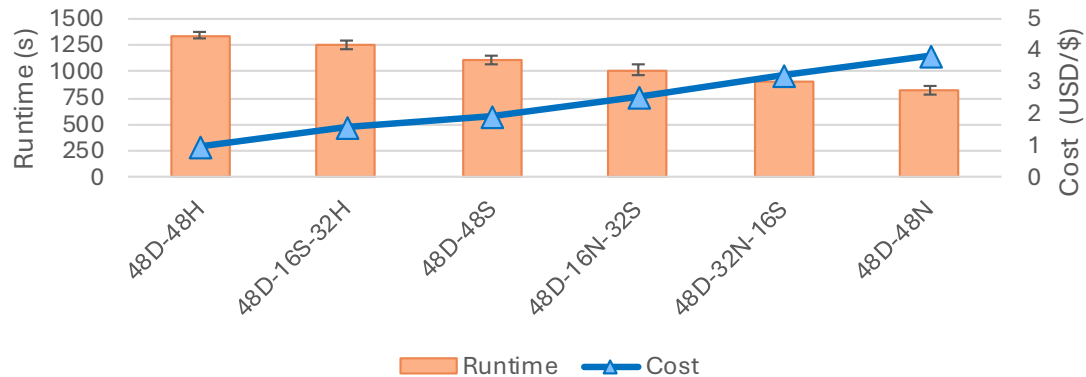


Figure 4.7. Tiering study of MegaMmap for 768-process Gray-Scott. D=DRAM, H=HDD, S=SATA SSD, N=NVMe. The number indicates the amount of storage per-node given to the program (e.g., 48D means 48GB DRAM per node). MegaMmap improves performance as much as 1.8x by using NVMe. However, performance is related closely to cost.

erated by allowing data to reside on faster storage tiers. This data is aggressively demoted to lower tiers by the data organizer to make room for future incoming data.

At the baseline, MegaMmap leverages slow, high-volume storage (HDDs) to handle memory overflow, incurring significant I/O penalties and experiencing the slowest overall performance. As we add high-performance storage, this trade-off changes. With 16GB of NVMe and 32GB of SSD per-node, performance improves 1.5x over the baseline as the dataset doesn't immediately touch the HDDs, which are 6-10x slower than the SSD and NVMe. When increasing the amount of NVMe to 32GB, performance improves an additional 20%. When using only DRAM and NVMe, performance is overall 1.8x faster than the baseline. Though, in terms of cost, 48D-48S has 1.5x improvement over 48D-48H at half the financial cost of 48D-48N. By expanding memory to high-performance storage, MegaMmap allows applications to transparently stage data in accelerated storage and avoid costly synchronous disk seeks, providing substantial performance benefits when data doesn't fit entirely in DRAM. Even cheaper hardware can result in significant performance benefits com-

pared to HDD-only solutions.

4.4.2.4 Lowering DRAM Consumption In this evaluation, we measure the performance impacts of reducing DRAM consumption using optimal configurations of MegaMmap. To do this, we generate datasets for each application and then vary the maximum DRAM capacity. The datasets are generated using Gadget and are 1TB in size (32GB per node). We run KMeans and DBSCAN to first analyze the datasets produced by Gadget. We use $k = 8$ with a $max_iter = 4$ for KMeans. For DBSCAN, we use an $\epsilon = 8$ and $min_pts = 64$. The cluster assignments are stored in a binary file. RF analyzes this data and produces 1 decision tree with a $max_depth = 10$. For Gray-Scott, we set $L = 3128$, also producing 1TB. Each application is executed with 1536 total processes (32 nodes). We vary the amount of DRAM being used for each program type. All overflowing data will fit in NVMe.

Overall, from Figure 4.8, it can be observed that each of the algorithms execute with nearly the same performance even with just half the main memory, demonstrating that MegaMmap can effectively overlap data movements with computation. KMeans can use $2.6\times$ less DRAM while DBSCAN and RF can use $2\times$ less DRAM with minimal performance overhead. This result is largely due to the effectiveness of asynchronous prefetching and eviction. KMeans, RF, and DBSCAN iterate over portions of the dataset highly predictably, where KMeans is mostly sequential while RF and DBSCAN are more pseudo-random. This allows MegaMmap to evict pages that have been touched during data-intensive computations and predict the pages that will be accessed next. In the case of Gray-Scott, which can also use $1.6\times$ less DRAM, this result is largely due to asynchronous writes and flushing. Data is initially placed in the fastest tier with available capacity, and then aggressively flushed down to lower tiers. After a certain point, each of the programs incur significant overheads due to frequent synchronous page faults and I/O stalls caused by frequent spills to NVMe, resulting in

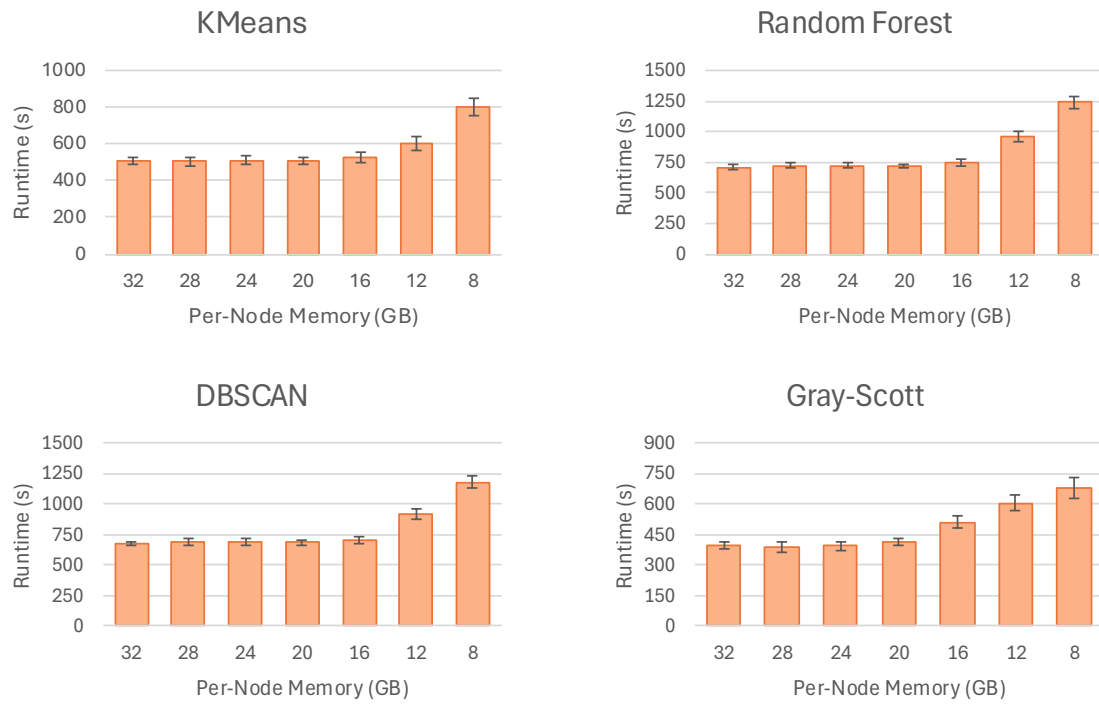


Figure 4.8. Memory scaling study of MegaMmap. Each program has 1536 processes over 32 nodes. Through intelligent tiering, DRAM can be lowered as much as 2.6x while maintaining competitive (within 10%) performance of full DRAM capacity.

performance degradation of as much as $2.5\times$. Overall, this evaluation demonstrates that achieving high performance for memory-centric, data-intensive problems can be accomplished without increasing DRAM capacity through intelligent use of storage tiering and asynchronous behavior.

4.5 Discussion & Limitations

Node Failure: Currently, MegaMmap assumes that the nodes are reliable and that the application would fail anyway if a node were to go down. If a node were to fail, the entire DSM would be corrupted. However, the MegaMmap runtime could be extended to support reliability and fault tolerance by implementing replication [134].

Security: Utilizing persistent storage to store temporary information can result in security concerns. If the data being analyzed is classified, the DSM must buffer data with the same level of access as the original content. This protection can be accomplished by changing the access control permissions of data within the MegaMmap runtime. [197].

Memory Corruption: Applications may encounter a situation where hardware stores data incorrectly. Bit flips in DRAM are not uncommon [198], and there are algorithms such as error correcting codes [198] that MegaMmap could implement to ensure that data remains correct.

Accelerator APIs: As the variety of accelerators such as FPGAs and GPUs continue to rise, software support must be provided. We designed MegaMmap with such abstraction that it could be easily extended to offer coherent access to distributed memory and storage from within accelerators.

Multi-Tenancy: MegaMmap is designed to run within an HPC job – where a single phase of a workflow runs. For this reason, we do not explore the effects of multi-tenancy in this work, as only one application accesses the DSM at a time. In the

future, we intend to expand the shared cache design to consider multi-tenant DSMs and contention mediation.

Legacy Applications and Language Support: MegaMmap is a library (similar to MPI and PGAS) designed to leverage language features, specifically operator overloading, to accomplish a friendly DSM interface. Any language supporting this (e.g., Python, Julia, C#, Rust) can implement a similar API. C programs can typically switch to a C++ compiler to use MegaMmap. To adapt an existing program to MegaMmap, only the data structures that are too large to fit in memory need to be converted. Accesses to these vectors would also have to be updated to use transactions. This can vary in time and complexity based on the volume and complexity of the original code. As future work, we intend to explore compiler extensions to allow MegaMmap to be embedded automatically into applications, which would enable support for legacy programs.

4.6 Conclusion In this work, we address the issues facing distributed, memory-centric out-of-core algorithms through the design and implementation of MegaMmap, a software DSM that enlarges effective memory capacity through intelligent tiered DRAM and storage management. We demonstrate that the complexity of developing out-of-core algorithms can be reduced by enabling massive persistent datasets to be presented as memory objects while transparently managing data placement and space utilization. We showcase how expanding memory tiering to high-performance storage can be used to both improve performance and dramatically reduce the need for DRAM in parallel workloads. We demonstrate how leveraging a transactional memory API can be used to reap substantial benefits in tiering performance. Evaluations showcase that developing algorithms with MegaMmap reduces peak memory utilization by as much as $2.6\times$ compared to leading solutions while accomplishing similar or better performance.

CHAPTER 5

ETERNIA: AN EFFECTIVELY INFINITE DISTRIBUTED SHARED MEMORY
FOR GPU WORKLOADS**5.1 Introduction**

Data-driven, memory-intensive GPU workloads have become widespread in HPC and Cloud. Scientific simulations [199–201], data analytics [99], machine learning [143], and deep learning [202, 203] codes oftentimes perform computations on data that stress or greatly exceed the bounds of GPU memory. Large Language Models (LLMs), for example, train on terabytes of data [204], model weights for inferencing are terabytes (e.g., Llama 3.1 405B) [205], and scientific simulations generate as much as petabytes [206, 207]. Yet, abstractions for GPUs to produce and analyze these massive datasets are limited. Oftentimes programmers must implement out-of-core algorithms, where GPU and main memory constraints are managed manually and data is explicitly moved between memory and storage. Making this process efficient is notoriously difficult [208] and often leads to suboptimal solutions plagued by the I/O bottleneck where data is synchronously moved between a disk-based PFS, main memory, and GPU memory in a separate phase from compute. For some algorithms, such as AI inference [209], manual out-of-core methods are unavailable and sufficient memory capacity is expected to complete the calculation. While many applications would benefit from an effectively infinite memory to process and persist massive datasets, simply purchasing more GPUs and integrating more HBM is not scalable due to extreme energy and financial cost [210].

To provide GPU workloads with the performance, energy, and capacity they require, heterogeneous storage is being investigated. NVMe SSDs have made significant advances in terms of capacity and performance [211] while consuming significantly less energy [212]. Many works have emerged that enable data to be transferred

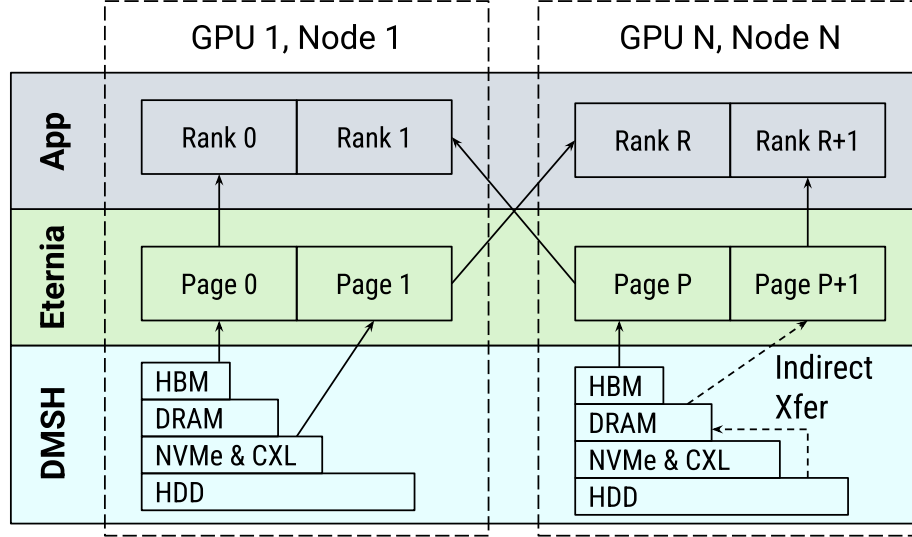


Figure 5.1. Depiction of Eternia for Tiered Storage

directly between GPU and NVMe SSDs. Persistent memory (PMEM) devices such as Intel Optane [101] and Compute Express Link (CXL) SSDs [213] allow storage to be mapped into GPUs like memory. Many HPC and Cloud sites have deployed these hardware in production. ORNL’s Summit [5], LLNL’s El Capitan [6], and NCSA’s DeltaAI [214] machines, for example, come equipped with high-bandwidth, low-latency, node-local NVMe, whereas Argonne’s Aurora [7] machine contains both NVMe and 3D XPoint modules. However, while modern storage provides great potential to improve capacity performance, presenting this highly heterogeneous **DMSH** to GPU applications is an ongoing challenge.

To address the performance, programmability, and capacity challenges facing GPU workloads, we must present the DMSH as a single logical **DSM** (shown in Figure 5.1). In this design, datasets can be accessed and modified within the GPU kernel in what appears as a single infinite memory, while data placement through distributed GPU memory and tiered storage is managed transparently. While some work has been done to expand GPU memory with NVMe [215, 216], none of them are distributed, work with arbitrary tiers of storage (e.g., SATA SSD and HDD), and support persis-

tence. In order to provide such an abstraction, several challenges must be overcome.

First, developers must be able to capture GPU kernel I/O pattern intention. Existing tiered memories are unaware of the kernel’s access behavior, limiting the effectiveness of data organization and caching policies. However, GPU access patterns are oftentimes deterministic (e.g., matrix and tensor operations). For more complex workloads, such as graph problems, more advanced techniques such as short-term memory models can be used.

Second, the overall lifecycle and persistence of data must be considered. Many libraries, such as HDF5 [188], ADIOS [217] and Parquet [218], are used to disseminate data to broader communities. However, at this time, these libraries do not support direct access from the GPU. In addition, high-performance resources like NVMe are often ephemeral (e.g., in supercomputing centers), requiring data to be written in its proper format to a long-term, oftentimes disk-based, distributed backend by the end of a job. Lastly, not all GPU (e.g., AMD) and storage (e.g., HDD) resources support direct access from the GPU. Storage and processing resources must be carefully managed to ensure data is correctly and efficiently stored.

Third, we must ensure efficient access to distributed metadata by the GPU. To communicate with directly accessible storage (e.g., NVMe), the GPU must first know the location of data on the storage hardware. However, metadata management is complex and oftentimes difficult to scale even for CPU-driven workloads [219, 220]. In a system where data placement may be frequently changing and where thousands of threads may be querying the DSM at a time, ensuring relevant metadata is cached on the GPU when it is needed is a critical priority.

Fourth, we must provide new data placement and prefetching algorithms to manage the DMSH for GPU-centric workloads. Existing GPU

memory expansion techniques assume simple binary datasets that are limited to the single node and are stored on NVMe and CXL. However, a full solution must consider all tiers of storage, the cost of serializing/deserializing from complex data formats, and the specific data access pattern of the kernel.

In this work, we present Eternia: a new DSM for distributed, data-driven GPU workloads that automatically manages the placement of data in diverse, tiered storage. Eternia provides a portable C++ library that leverages language features (e.g., operator overloading) to present massive datasets as if in memory. The system is designed to be highly portable in terms of both data and hardware. Popular data formats (e.g., HDF5, parquet) can be mapped into GPU memory, and most GPU types are supported (e.g., AMD and NVIDIA, commercial or non-commercial). Various optimizations are made to ensure Eternia can balance the extreme concurrency and speed of GPUs and High-Bandwidth Memory (HBM) with the far slower I/O substrate through effective metadata caching, prefetching, and data organization policies. We demonstrate that Eternia allows complex data-intensive applications such as scientific simulations to run with unbounded memory feasibly while also enabling greater control over memory capacity, allowing up to 4x reduced memory in certain workloads with minimal end-to-end performance loss. The contributions of this work are as follows:

1. A GPU-centric, intuitive, and flexible **Distributed Shared Memory system**, which reduces out-of-core development complexity by presenting diverse storage as a single memory
2. A **parallel transactional memory API** that informs data and metadata policies by capturing and predicting GPU’s access to out-of-core data
3. An elastic **Cross-Processor I/O Runtime** that balances CPU and GPU re-

sources, ensures data persistence, and enables access to diverse storage hardware

4. A scalable, tiered, and predictive **Metadata Cache** that minimizes latency by leveraging access pattern to preallocate and prefetch metadata into memory
5. An informed **Data Organizer** that minimizes I/O stall times by leveraging access pattern and hardware characteristics

5.2 Background and Related Work

	GPU Mem	CPU Mem	NVMe	SATA SCSI	Tiering	API	Persistent	Distributed	GPUDirect	Data Access	Access Pattern
MegaMmap [2]	No	Yes	Yes	Yes	Yes	Memory	+Data Format	Yes	None	CPU-only	Any
BaM [215]	Yes	No	Yes	No	No	Memory	No	No	Required	GPU-only	Any
GPUfs [221]	Yes	No	Yes	No	No	POSIX	Binary file	No	None	CPU + GPU	Read-only write-once
GPU4FS [222]	Yes	No	Yes	No	No	POSIX	Binary file	No	Required	GPU-only	Any
GeminiFS [216]	Yes	Yes	Yes	No	Partial	POSIX	Binary file	No	Required	GPU + CPU	Read-only Append
CUDA UVM	Yes	Yes	No	No	No	Memory	No	No	None	CPU + GPU	Any
Dragon [223]	Yes	Yes	Yes	No	Partial	Memory	No	No	None	CPU + GPU	Any
ActivePointers [224]	Yes	No	Yes	No	No	POSIX	Binary file	No	None	CPU + GPU	Read-only Write-once
GMT [225]	Yes	Yes	Yes	No	Partial	Memory	No	No	Required	GPU-only	Any
MPI (GPU) [226], NCCL [227]	Yes	No	No	No	No	Message	No	Yes	None	GPU-only	Coordinated messages
WekaFS [228], BeeGFS [229]	Yes	Yes	Yes	Yes	No	POSIX	Binary file	Yes	Flexible	CPU-only	Any
Eternia	Yes	Yes	Yes	Yes	Yes	Memory	+Data Format	Yes	Flexible	CPU + GPU	Any

Table 5.1. Comparison between Eternia and Alternatives

Storage and memory tiering is being investigated as a method of increasing memory capacity and I/O performance. APIs are emerging to improve the programmability of GPUs for data-intensive workloads.

5.2.1 CPU-Only Memory Expansion

Many works have explored the expansion of main memory in CPU-based applications. uMMAP-IO [161], for example, uses Linux’s userfaultfd to extend virtual memory to SSDs and Hard Drives for HPC workloads, effectively increasing memory capacity. Alternatively, MegaMmap [2] implements a tiered, distributed shared memory using a vector abstraction to mask storage and memory technologies. However, these works were designed only for CPU-based workloads and do not consider the

characteristics of highly concurrent, distributed GPU workloads. This work expands the MegaMmap design to support efficient invocation from GPUs.

5.2.2 Direct Storage Access from GPU

Work has been done to allow GPUs to interact with storage directly from GPU. CuFiles [230] provide a POSIX-like file abstraction that enables data to be transferred between GPU and SSD from a local filesystem (e.g., EXT4) without first copying into DRAM. CuFile code, however, is initiated and first executed on the CPU. Conversely, BaM [215] enables GPUs to interact with NVMe SSDs directly using a C++ array abstraction that hides the complexities of GPU-initiated I/O calls. Recent work enhanced the driver [216] to share NVMe queues on both CPU and GPU. These works offer performance and resource utilization optimizations and are helpful building blocks to building a tiered GPU memory. However, drivers do not inherently provide semantic interfaces to data (e.g., files) or discuss the complexities of managing distributed metadata to locate data on storage.

5.2.3 Direct Storage Access with CXL

Compute Express Link (CXL) has been proposed to unify diverse memory devices. CXL memories based on NAND flash SSDs have been developed that can be used to expand memory capacity at the expense of latency and bandwidth compared to DRAM and HBM. Many works [231–233] have integrated prototype CXL SSDs into the GPU memory system to improve memory capacity and have analyzed various performance optimizations. Since CXL is highly specialized and not widely available, we do not focus on CXL optimizations in this work. Instead, we focus more on providing a distributed abstraction portable across a wide variety of hardware.

5.2.4 Single-Node GPU Filesystems

Many GPU filesystems have been proposed exposing an interface similar to POSIX. GPUfs [221] implements a filesystem where I/O operations are partially coordinated by the CPU. The work additionally assumes data is only written at most once to a file. GPU4fs [222] implements a library filesystem that runs only on the GPU and assumes that CPU-driven codes will not access the data. GeminiFS [216] implements a filesystem designed for ML workloads that efficiently coordinates meta-data access between GPU and CPU to allow data generated on GPUs. However, these works focus only on placing data in local NVMe SSDs. They do not discuss the complexities of distributed storage and N tiers of storage. In addition, it is often the case that datasets are stored in a distributed system due to size, so expensive data copies would have to be incurred between the local system and the distributed to retrieve and persist data.

5.2.5 Distributed GPU Filesystems

Many distributed filesystems have emerged with support for GPUDirect storage, such as WekaFS [228], BeeGFS [229], and IBM Spectrum Scale [234]. This enables direct data transfers between local and remote NVMe to GPU memory. However, these works require that I/O is initiated at the side of the CPU, cannot be invoked from the GPU, do not offer GPU memory expansion, and do not provide storage tiering capabilities.

5.2.6 Single-Node Tiered GPU Memory

Some work has been done to explore the expansion of main memory on GPUs. CUDA unified virtual memory allows memory pages to be swapped between host memory and GPU memory, effectively increasing GPU memory. Dragon [223] additionally integrates SSDs into unified memory, at the expense of high latency due to page fault overheads and CPU involvement [225]. ActivePointers [224] moves the page fault handler to the GPU to improve concurrency, but relies on GPUfs [221] which is partly CPU-driven and mainly read-only. Lastly, GMT [225] implements a library orchestrated entirely by the GPU for storing data in SSD, main memory, and GPU memory. However, these approaches suffer the same limitations as the single-node filesystems. They do not consider N tiers of storage and the full lifecycle of data.

5.2.7 Distributed GPU Memory, but not Tiered

Several solutions exist to enhance the programmability of distributed GPUs. The Message Passing Interface (MPI) [226], NCCL [227], and Kokkos [235] provides a distributed memory abstraction that can be used to pass data between GPUs. Alternatively, NVSHMEM [236] provides an interface similar to OpenSHMEM [172]. However, these works do not discuss leveraging storage tiers to increase capacity and leave the persistence of data to the user.

5.2.8 Summary

Based on our extensive literature survey, we observe that there is no existing solution to enable GPUs to seamlessly interact with massive, potentially complex, datasets as if in logical GPU memory. Existing technologies for GPU memory expansion focus solely on single-node cases and do not consider how data is transferred to/from persistent storage. This limits memory capacity and data sharing techniques.

5.3 Design

Eternia is a DSM designed to provide a portable, intuitive, and efficient method of accessing and persisting massive datasets for highly concurrent and distributed data-intensive GPU workloads while masking the complexity of memory capacity constraints and data placement in diverse tiered storage from developers. Eternia exposes a byte-addressable API similar to a *std::vector*, allowing massive and potentially complex datasets (e.g., HDF5 and Parquet) to be produced and queried in what logically appears as a single infinite memory. Developers can propagate the GPU kernel’s I/O pattern intention through a parallel transactional memory API. In the backend, data is automatically prefetched into and evicted from GPU memory in pages of configurable size to the DMSH through extensible data organization algorithms, some of which are provided to optimize I/O time for common out-of-core behaviors in GPU-centric science and AI workloads. To further improve performance, Eternia automatically preallocates and caches metadata necessary for GPUs to perform I/O directly to compatible hardware (e.g., DRAM, CXL, NVMe). When direct I/O is not possible, I/O requests are batched to minimize the burden to the CPU. The high-level architecture of Eternia is shown in Figure 5.2. Below are the overall design objectives of Eternia.

1. **GPU-Centric:** Eternia should allow GPUs to perform I/O directly without CPU involvement when possible.
2. **Portable:** Eternia should allow data to be placed in any type of storage (including SATA) and should support diverse GPUs (e.g., AMD).
3. **Persistent:** Eternia should ensure that data is correct and stored in its proper format (e.g., binary, HDF5) after a job ends.
4. **Intuitive:** Eternia should hide the complexity of data movement for memory-

constrained GPU workloads behind an easy-to-use memory API.

5. **Heterogeneous:** Eternia should provide algorithms that automatically manage the organization and coherency of data in a tiered environment.
6. **Semantic:** Eternia should provide APIs to describe access patterns to inform data organization policies.

5.3.1 An Intuitive Infinite Memory API for GPUs

Listing 3 An example of Eternia for vector addition. In lines 15-17, three big vectors are created. *x* maps an HDF5 file named *vec.h5*, which Eternia will transparently decode as it is accessed and prefetched. *x* stores up to 1MB of data per GPU thread before evictions occur, and can be accessed on any GPU (*ET_ALL_GPU*). *y* and *z* are similar, but are volatile. In 7-9, the GPU kernel defines sequential transactions for *x*, *y*, and *z* and describe the offset and size of data about to be iterated over.

```

1  #include <eternia/vector.h>
2  __global__ void vectorAdd(et::Vector<float> v[3],
3                          int N) {
4      int size = (N / grid.x);
5      int off = threadIdx.x * size;
6      et::SeqTx a(v[0], off, size, ET_RDONLY);
7      et::SeqTx b(v[1], off, size, ET_RDONLY);
8      et::SeqTx c(v[2], off, size, ET_WONLY);
9      for (int i = 0; i < size; ++i) {
10         c[i] = a[i] + b[i];
11     }
12 }
13 int main() {
14     et::Vector<float> x("vec.h5", MB(1), ET_ALL_GPU);
15     et::Vector<float> y(x.size(), MB(1), ET_ALL_GPU);
16     et::Vector<float> z(x.size(), MB(1), ET_ALL_GPU);
17     VectorAdd<<<1, 64>>>({x, y, z}, x.size());
18 }
```

Developers of out-of-core GPU algorithms typically must manage GPU memory, main memory, and data placement in storage manually. As many datasets are initially stored in a massive persistent backend, such as a disk-based Parallel Filesystem (PFS), parts of these datasets must be explicitly loaded from the backend and

Listing 4 The base class of all Eternia transactions.

```

1  template<typename T>
2  class Transaction {
3      Vector<T> vec_;
4      et::Comm agg_;
5      // Implement per-transaction
6      Transaction(Vector<T> &vec);
7      ~Transaction();
8      inline int& operator[](size_t off);
9      inline const int& operator[](size_t off) const;
10     inline void Log(const int &off);
11     Iterator<Page> GetTouchedPages();
12     Iterator<Page> GetFuturePages(int lookahead);
13     // Common across transactions
14     INHERITED int Prefetch(int lookahead);
15     INHERITED Iterator<Page> Aggregate(int lookahead);
16     INHERITED void Fault(int off);
17     INHERITED void Evict(size_t size);
18 };

```

cycled between main and GPU memory. Making this process efficient is complex and oftentimes leads to suboptimal solutions where I/O is the bottleneck [2]. This separation between storage and memory leads to increasing DRAM and HBM capacity, which is becoming unsustainable due to high energy and financial cost. Eternia bridges the gap between storage and compute through an infinite memory abstraction that allows for arbitrary datasets to be presented as if entirely in GPU memory. To improve the efficiency of caching and data organization algorithms, a parallel transaction API is provided that allows GPU threads to describe their access pattern, while reducing the potential number of queries to backend storage through I/O aggregation. This section details the Eternia API and its differences from existing work.

An Infinite Shared Memory Abstraction: To give applications the illusion of infinite memory, Eternia implements a shared memory vector API similar to MegaMmap [2], providing implementations of several key functions and operators including array index, memory copy, acquiring current size, appending, resizing, and destroying the data container. Processes running on the CPU initially connect to the

shared vector using a semantic, user-defined key common to all processes. Developers can then specify the set of GPUs on a node the vector should support. Internally, this allows the vector to allocate virtual memory for the supported GPUs with APIs like *cudaMalloc* and *hipMalloc*. After initialization, the vector can be passed directly to the GPU kernels as a parameter. The vector constructor optionally takes as input a URL to a persistent dataset. When indexing the vector, the dataset will be transparently loaded and prefetched into the DMSH. Modifications to the vector will be flushed to the backend before the vector is destroyed unless otherwise configured. If no URL is provided, the vector is considered volatile and its data will not be retained. The page size of the vector can be set to instruct caches the best way to lay out data.

Presenting Persistent Datasets as Memory: Many GPU applications must either interact with existing data (e.g., LLM training) or store produced data for future analysis (e.g., scientific simulation). Eternia’s vector abstraction supports the ability to map these complex datasets as memory objects and transparently (de)serialize them. To do this, we adopt the same interface as MegaMmap – a flexible URL scheme that describes the location and format of the dataset (i.e., “protocol://URI:params”). The dataset does not need to be in local storage, and could exist in an external repository such as Amazon S3, GitHub, Globus, Google Cloud, etc. The “params” of the URL could include information such as port numbers and account names. For example, a vector representing column x of a table named *particles* stored in a parquet file on Globus could be: *globus:///home/data.parquet:username, particle.x*.

Expressing Storage and Memory Constraints: Traditionally, developers are responsible for designing programs that respect the capacity constraints of GPU and main memory, which imposes a significant programming burden – especially when considering multiple tiers of storage. In addition, it is not always clear when a storage is ephemeral. Many supercomputing sites, for example, use node-local burst

buffers as only temporary storage and cannot be accessed after a job ends. To define these constraints, Eternia provides a configuration YAML file, which contains various settings relating to deployment, including capacity constraints, port numbers, thread counts and affinity hints, etc.

Informing Policy Through Parallel Transactions: Out-of-core algorithms must balance the high capacity of storage with its reduced performance and concurrency compared to on-chip memory. This is accomplished by predicting the access pattern an out-of-core dataset is going to incur. In Eternia, the access pattern a region will incur is captured through parallel transactions. A common scenario is describing a for-loop that iterates over a subset of a dataset. Figure 4 shows the transaction base class and Figure 3 demonstrates an example GPU kernel with a sequential transaction for vector addition. The constructor marks the beginning of an access and includes fields such as access method (e.g., read, write, append), offset, and size of the iteration. To reduce the potential number of potential I/O queries to less scalable hardware, transactions contain an Eternia communicator, which provide warp-level barrier operations to aggregate I/O requests and reduce contention. The destructor marks the end of the transaction and calls cleanup operations (e.g., triggering the flush of dirty data to persistent storage). Transactions are templated, enabling custom transactions to be developed and ensure maximum flexibility to a variety of workloads. Eternia natively provides transactions to describe sequential, pseudorandom, and append-only workloads.

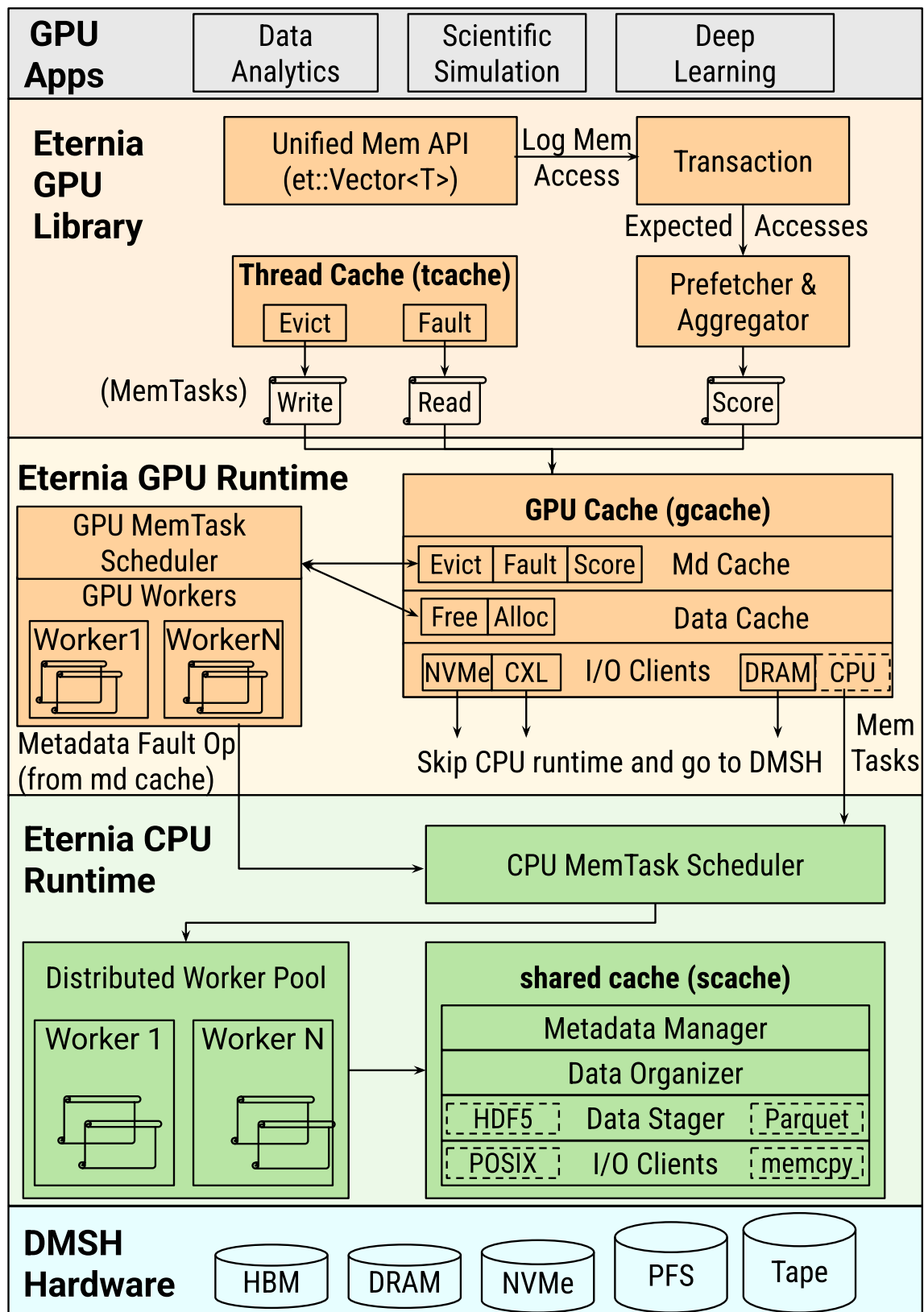


Figure 5.2. High-level Architecture of Eternia

5.3.2 Ensuring Data Persistence and Portability

Existing GPU memory expansion technologies focus purely on ensuring the GPU has enough memory to handle out-of-core workloads. However, they do not consider the broader lifecycle of data – particularly how data is loaded into memory in the first place, or how data is exported to long-term storage. There are many circumstances, such as with scientific simulation, where data is required to be retained, potentially in complex serialized formats (e.g., HDF5), long past the lifetime of a single job. Existing solutions also assume that node-local NVMe is the final resting place of data, which is oftentimes not the case – particularly in HPC where burst buffers are ephemeral [3]. Despite advances, GPU workflows still rely on CPU-mediated I/O paths even in cutting-edge systems [237–239] as no popular I/O libraries (e.g., MPI-IO [240], HDF5 [188], Adios2 [217], Parquet [218], NetCDF [241], SIONlib [242]) can be called directly within the GPU and long-term storage is often HDD-based due to superior price-per-gb [243]., we must ensure efficient cooperation between CPU and GPU tasks. This section provides details on the overall lifecycle of data in Eternia, how CPU and GPU resources are managed, and the high-level architecture in Figure 5.2.

Distributed Heterogeneous Caching Structure: There are three levels of cache in Eternia: *Thread Cache (tcache)*, *GPU Cache (gcache)*, and *Shared Cache (scache)*. The thread cache stores only the data for a particular GPU thread. Whenever the index operator (e.g., `vec[10]`) is called, it goes to the tcache. If data is not present in the tcache, a fault occurs. If space is needed in the tcache to serve a fault, an evict occurs. In either case, a *MemoryTask* containing the offset and size of data being accessed is created and sent to the next level of cache, which is the gcache. The gcache stores data and metadata locally on a particular GPU. There is only one gcache instance across all kernels, threads, and processes on a particular GPU. The gcache is

stored in shared device memory (using APIs such as *cudaIpcOpenMemHandle*), and each tcache holds a pointer to it. This ensures that data and metadata is not copied on precious GPU memory across processes, which is important to workloads coordinated with frameworks like MPI. The gcache has a metadata cache that points to the location of data on directly-accessible memory and storage, such as HBM, DRAM, and NVMe. This allows for the gcache to evict and prefetch data without CPU involvement in many cases. Lastly, the shared cache is a distributed, tiered cache that manages the global placement of data and metadata in the overall Eternia instance. The scache is the primary owner of metadata and is responsible for ensuring that the GPU caches remain correct after asynchronous operations such as data reorganization.

Read and Page Fault Path: In Eternia, reads first go to the tcache. If present, the data is accessed directly. Otherwise, a synchronous page fault occurs that constructs a Read *MemoryTask* containing the offset and size of data to retrieve. The task is first sent to the gcache, which has two general paths. **First, if the metadata for the ReadTask (i.e., file id and offsets) is cached**, the local DataCache is checked. If present, the data is copied directly from the gcache to the tcache. Otherwise, a Allocate MemTask is spawned and scheduled with the GPU runtime to fetch data from the I/O clients. The use of asynchronicity here allows for prefetching to occur outside of the critical path and reduce contention to the metadata table. If space is needed in the gcache to handle faults, evictions will occur first, which are also processed by . **Otherwise, if the metadata for the ReadTask is not cached**, it is forwarded to the distributed MemTask scheduler in the runtime through a queue stored in pinned host memory. The scheduler will assign the task to a worker, which will determine the location of data using the scache’s Metadata Manager. Then data and metadata will be copied from the scache to the local gcache, which will then be copied to the tcache.

Prefetcher Path: To avoid synchronous page faults or at least reduce their cost, the Prefetcher preemptively spawns tasks to score the importance of data and asynchronously bring critical data into the GPU caches and directly-accessible storage. The importance score is a number between 0 and 1 where 0 is unimportant and 1 is critical. Unimportant data will be evicted from high-performance storage and moved to slower tiers, while more important data is added. Since prefetching aims to be asynchronous, MemTasks are sent to workers running on the GPU. These workers are spawned periodically by the runtime at a configurable rate and parallelism. These prefetching operations are processed in the Data Organizer, which interprets these scores and enacts data reorganization algorithms. To reduce the burden of scoring from the CPU, the transactions aggregate the expected access pattern of a set of threads based on communicator. If multiple threads set the score of a memory page within a configurable time frame, the organizer will take the maximum of the scores.

Lifecycle of Modified Data and Consistency: In Eternia, data in the scache is replicated in the GPU and thread caches. However, ensuring that replicated data is correct enough without sacrificing scalability is a critical priority. In Eternia, developers are primarily responsible for ensuring the consistency of data using transactions, which capture access pattern. This includes tracking the set of modifications made to a region of memory. During each memory write operation, the transaction's Log function is called to track regions of modified memory in the tcache. In most circumstances, this logging is simply the increment of a counter. For example, in a sequential writer transaction, the set of modified pages is determined statically by the last offset evicted and the last index accessed. Either at the end of a transaction or during an eviction, modifications are committed to the gcache and scache. Instead of flushing entire memory pages, only the modifications made during the transaction are propagated. This ensures that if multiple tcaches interact with the same memory page, their results will be merged correctly. To ensure consistency across all caches, the

scache is strongly consistent and does not store replicas. MemoryTasks for read and write operations will be mapped to the same worker and processed in sequence. As GPU workloads are naturally coordinated and typically data-parallel, Eternia aims to be correct in the following general circumstances:

1. **Read-Only:** This is common in training and data analysis workloads. For example, datasets for LLM training are never modified. Cache consistency is automatic as no data in the vector is being modified.
2. **Write to non-overlapping subsets:** Caches across nodes do not need to be synchronized.
3. **Alternating Phases of Write then Read:** This is common in both scientific workloads and analysis workloads. For example, Gray-Scott [244] simulates the reaction between two fluids. In the first phase, it calculates the density of the fluids in each cell of a 3-D grid. Then it performs an exchange operation, which requires all data modifications to be visible. During write phases, gcache is prioritized and the scache is lazily updated. At the end of the transaction, the gcache pages touched during the write can be invalidated, which will force all future reads to reacquire pages.

Integrating Memory with Diverse Data Formats: Oftentimes, data is stored in a persistent format such as HDF5, Adios, or Parquet to ensure portability to other environments for future use and analysis. Currently, GPU applications must explicitly load data from these formats and pass to the GPU kernel. However, this requires developers to manually ensure capacity constraints of main memory, which introduces complexity. In Eternia, complex datasets can be represented as memory objects and transparently (de)serialized or imported from diverse data repositories (e.g., Globus, Google Drive). This is handled by the **Data Stager**. The Data Stager

is an extensible factory that exposes an API to parse URLs. For example, a vector representing column x of a table named *particles* stored in a parquet file on Globus could be: *globus:///home/data.parquet:username, particle.x*. Data is not imported until prefetched or accessed during a tcache fault. When data that has not been staged is first requested, the Metadata Manager will allocate space in the scache for holding the data. This information will be given to the stager which will then import the content in main memory and store in the scache. The fault (or prefetch) will then be serviced as usual. With this approach, Eternia can encompass the semantics of diverse I/O libraries without requiring developers to know and understand the specific APIs of each library, which oftentimes are complex in and of themselves (e.g., HDF5 has over 1000 APIs).

Managing CPU Resources: While Eternia can skip the CPU in many cases through aggressive, application-aware metadata and data prefetching and direct storage access capabilities, the CPU is primarily responsible for metadata management, indirect storage access, data reorganization, data (de)serialization, and ensuring data is persistent. To efficiently utilize CPU resources, Eternia separates tasks into two groups: low-latency and high-latency. Low-latency workers are responsible for metadata-related tasks, while high-latency workers are responsible for data-related tasks like serialization and reorganization. Low-latency workers are given dedicated CPU cores, while high-latency workers may overlap. The number of CPU cores scales dynamically depending on access pattern, and users can set a maximum number of CPU cores the Eternia runtime can use per NUMA node. For scaling, we use the approach described in LabStor [1], which estimates the amount of CPU time a task will require by training user-defined models. A model for synchronous POSIX I/O, for example, could be a latency-bandwidth calculation since the CPU will be stalled waiting for I/O. For async, non-blocking I/O that simply checks whether or not I/O is complete or a metadata table lookup, a constant would suffice.

5.3.3 Scalable, Heterogenous Metadata Management and Direct Access

GPUs are thousands of times more concurrent than CPUs. However, metadata operations are a notorious bottleneck in I/O systems [1] even in CPU-driven workloads, especially in cases of small file I/O common in AI training [86]. Due to the increased parallelism that GPUs offer, ensuring that metadata is efficiently placed is critical to the overall performance of the system. To accomplish this, Eternia implements an aggressive, predictive GPU-facing metadata cache. In cases where metadata is not cached locally, the metadata is placed in the DMSH using a scalable hashing scheme. This section details the placement and updating of metadata to ensure high metadata performance from the GPU.

Global Metadata Placement and Content: To ensure high performance, metadata must be efficiently placed and easily queried in the distributed system. As GPU workloads do not typically require high performance from metadata operations like locating the set of files in a directory, we use an object storage model for Eternia where the scache is the primary owner of metadata. Initially, when creating a vector on the CPU, the scache is invoked. Eternia’s data model has two concepts: vector and page. Vector metadata stores information that allows CPU processes to connect to the vector, including: vector name, permissions, and the set of pages belonging to the vector. Page metadata contains information to access data on physical storage, including: the identifier of a storage device (64-bit), the offset and size of data in the storage (128 bits), an atomic counter to help manage concurrent access (64-bit), and a pointer to data in memory (64-bit) if data is cached. The vector metadata is arbitrarily sized, while page metadata is a fixed 40 bytes. For both types of metadata, the scache uses a hashing scheme to map vectors and pages to nodes. For vectors, the hash is on the URL of the vector (e.g., “globus://hi.txt”) and for pages it is based on the page ID. In the gcache, only page metadata is cached. Since vector creation

and permissions checking are handled by the scache before kernels launch, there is no need to repeat this information.

Direct Access to Storage with I/O Clients: I/O clients on each GPU are initialized at the start of the Eternia runtime. The NVMe I/O client stores queue pairs, while CXL and DRAM clients store pointers. The number of queue pairs to assign the GPU and the size of CXL and DRAM clients can be configured as settings in the Eternia configuration file. When launched, the runtime automatically assigns each device a unique integer id which is used in page metadata for locating the I/O clients during faults and evictions.

GPU Metadata Cache Concurrency Control: Eternia has many asynchronous operations and threads that can modify the gcache’s metadata cache simultaneously. Ensuring correctness while maintaining scalability and low latency is a critical concern. To satisfy these objectives, the cache uses an unordered map where the pair (page id, vector id) maps to a metadata entry. In general, if a metadata entry is present in the gcache, no synchronization is required to read from it. The main cases are when new entries are added to the table (Fault) and when entries are removed (Evict). However, these operations typically occur during page faults and data reorganization, which are generally avoided on the critical path due to prefetching. For this reason, we adopt an approach that sacrifices some performance to table updates for the sake of maximizing query scalability through a combination of queueing and locking. Operations that evict and emplace metadata are queued and asynchronously polled by workers running on the GPU. These threads are launched by the runtime and periodically rescheduled. Modification operations only progress if the reader count on a bucket in the table is zero.

GPU Data Cache Concurrency Control: The data cache is only modified either during a fault (which allocates data) or an eviction (which frees data). Since vectors

store data in page granularities, the memory access pattern of this cache is more simplistic than a general allocator like malloc. While vectors may have different page granularities depending on their access pattern, in general it can be assumed that there will only be a few well-defined page sizes used throughout the entire Eternia instance. For this reason, we adopt a slab allocator strategy. We assume a fixed, configurable set of slabs. Slabs are allocated ad-hoc from a heap as-needed and then reused as much as possible. The allocator will only lock during defragmentation. To improve performance, warp-local storage is used to cache slabs.

Preemptively Caching, Evicting, and Allocating Metadata with Transactions: To minimize the cost of metadata access, Eternia preallocates and caches metadata that GPUs expect to access in the near future. While some existing work such as GeminiFS [216] have proposed preallocation of entire files on GPUs, Eternia adopts a finer-grain approach where only the portions of the file that a GPU is expected to access will be cached based on the prefetch operation of transactions. As distributed datasets are massive, caching the metadata for entire files on a GPU can be prohibitively expensive. E.g., if a dataset is 1PB, disk blocks are allocated every 1MB, and the metadata for storing a block is 40 bytes, the resulting metadata size would be 40GB.

5.3.4 Minimizing I/O Stalls With GPU-Aware Prefetching + Eviction

Data movement cost is a major concern for out-of-core GPU workloads since storage is far less concurrent than HBM [245]. Existing tiering methodologies for GPU memory are not aware of the specific access pattern that a region of memory is about to incur and rely on general algorithms, such as LRU, LFU, and sequential prefetch [216, 225] to place data. While oftentimes better than nothing, these algorithms do not capture every access pattern optimally – for example, pseudorandom used for shuffling datasets to improve model generality during ML training [86]. To account for a broad spectrum of workloads, Eternia incentivizes GPU kernel developers to specify their access pattern intention through a parallel transaction API. While this may seem intrusive, many algorithms are naturally structured to support this and predictably iterate over large portions of out-of-core data between application phases [2].

Data Prefetching: With detailed knowledge of each kernel’s access pattern, significant performance and resource utilization benefits can be made. In Algorithm 3, Eternia’s prefetcher is shown. The prefetcher’s inputs are the vector being accessed, the transaction being used, and the maximum number of pages to score. The prefetcher algorithm aims to score the importance of data based on the order of access. Higher scores will be placed closer to the gcache, while lower scores will be mapped to lower tiers of storage (e.g., NVMe). Data that is no longer in use is scored 0, indicating it can be demoted at any time.

When invoked, the prefetcher determines the set of pages that were already touched and initially assigns them a score of 0. Next, the algorithm will score the next *lookahead* pages predicted to be accessed with a score that decreases linearly in the range 1.0 to *minScore* (by default .5). This default is chosen since a score of .5 will typically be mapped no lower than middle-tier storage (e.g., NVMe). This ensures

that relevant data will be brought out of slower backend storage asynchronously. It is possible that pages that were previously touched will be accessed again in the future (e.g., pseudorandom access). In this case, the score will be updated from 0.0 to the higher score. After the scores for touched and future pages have been set, they are sent to the gcache and processed asynchronously.

Data Organization: Page scores are interpreted by the Metadata Cache and Data Organizer. Each storage tier is assigned a score range based on performance characteristics, which are used to inform page eviction and promotion decisions. Scores are always a number between 0 and 1. The Metadata Cache will evict data with scores below the HBM’s minimum score when space is needed. Alternatively, data with high scores will be faulted in if they are not already existing. The scache’s Data Organizer manages data placement for every other tier. It will place a page in the tier that most closely matches its score. If the most suited tier is out of space, pages with a lower score are targeted for eviction to make room. If no such pages exist, then the page is demoted to a lower tier until it is placed. Since the prefetcher also includes the node ID that is setting the score, the organizer also aims to place the data on the node that sets the highest score to improve locality. While the tier scoring policy is extensible, the default scoring algorithm for Eternia is relative bandwidth logarithm: $\log(tierBW)/\log(maxBW)$. Since storage devices have such wide variety in bandwidth (typically in orders of magnitude), logarithm ensures the scoring is more uniform and gives each storage device a fair chance of data placement.

Algorithm 3 GPU Lib Prefetcher

```

1: function PREFETCHER(Vec, Tx, lookahead, minScore)
2:   FlagEvicts(Vec, Tx)
3:   PredictScores(Vec, Tx, lookahead, minScore)
4:   Vec.gcache.Score(Tx.Aggregate(lookahead))
5: end function
6: function FLAGEVICTS(Vec, Tx)
7:   for Page in Tx.GetTouchedPages() do
8:     Page.SetScore(0.0, Vec.NodeId)
9:   end for
10: end function
11: function PREDICTSCORES(Vec, Tx, lookahead)
12:   i = lookahead
13:   for Page in Tx.GetFuturePages(lookahead) do
14:     Score = (1 - minScore) * i / lookahead + minScore
15:     i - = 1
16:     Page.SetScore(Score, Vec.NodeId)
17:   end for
18: end function

```

5.3.5 Implementation Details

Eternia is implemented in 4K lines of C++ code. We build Eternia using the foundations of MegaMmap, which is a CPU-only distributed shared memory. We augmented all of MegaMmap’s core components to support GPU workloads and provide optimizations to ensure efficiency. We utilize the storage driver described in BaM [215] to support direct access to NVMe by the GPU. A configurable amount of pinned host memory can be defined to allow the GPU to interact with host memory.

5.4 Evaluations

5.4.1 Hardware

Most evaluations were conducted on the NCSA Delta system [246], utilizing A40 GPU compute nodes. Each node has 4x A40s (48 GB GDDR6 each) and built around AMD’s Milan processor architecture, with 64 cores (SMT disabled, 64 hardware threads per node) at 2.45 GHz with 256 GB of DRAM. The CPUs are configured with four NUMA domains per socket (NPS=4). A disk-based PFS is given with 10TB of storage. An NVMe-based PFS is given with 500GB of storage and GPUDirect enabled. A 1.5TB node-local NVMe is also equipped, but does not support GPUDirect storage. We also use the Chameleon Cloud [247] compute liquid nodes to run the SOTA evaluation, which come equipped with 3.5TB of node-local NVMe and 40GB A100 GPUs

5.4.2 Workloads

Vector / Matrix Addition: Widely used in scientific programs and AI models. Addition is a simple sequential workload, where two vectors or matrices a and b are read from while c is written to.

KMeans: This is a widely-used clustering algorithm that groups data into k spherical clusters. We implement a version of the traditional KMeans algorithm where the initial vector and number of iterations is a parameter to the algorithm. KMeans iterates over large portions of a dataset sequentially in non-overlapping subsets across threads. At each iteration, KMeans maps points to its nearest cluster (a large sequential write) and then calculates the inertia of each cluster (a large sequential read).

Random Forest (RF): These are commonly used for modeling complex, nonlinear associations in data and can be applied to feature reduction techniques. RF has a largely pseudorandom read pattern to improve the generality and accuracy of the

model. At each iteration, samples are randomly chosen from the dataset to calculate the entropy of a feature. If the entropy is above a certain threshold, then a point will be chosen to split the dataset into two subgroups, which will then be iterated recursively.

Gray Scott: This is a scientific simulation that models the reaction between two chemicals U and V . We implement a the Gray-Scott [100] algorithm on GPU. Initially, a grid of volume L^3 is defined and evenly subdivided among each thread. Each cell in the grid contains the concentrations of U and V at time step T . At each iteration, the concentrations are updated and then exchanged, resulting in $O(24 * (L/threads)^2)$ bytes of transfer. This is a deterministic, write-heavy workload. Due to the exchange operation, it is not fully sequential.

5.4.3 Comparison Against State-of-the-Art

We compare the performance of state-of-the-art methods for out-of-core GPU programming in single-node cases. This is because the existing tools do not support distributed I/O. To do this, we implement and run a microbenchmarks using GPUfs [221], ActivePointers [224], cuFile [230], and Eternia. Due to hardware and permissions limitations of the available testbeds, we did not compare against BaM [215]. GeminiFs [216] has open-sourced a CPU emulator of their code, but does not provide the original implementation, so we do not compare this either. The benchmark is a small random I/O (4KB transfers, 10GB total). For GPUfs and ActivePointers, we use 4 cores and allow cache size to be 1GB. Eternia is configured similarly, where the maximum number of cores is 4 and gcache is 1GB. Data is stored entirely on a GPUDirect-Enabled NVMe filesystem in a simple binary format that does not need any processing. The results presented were collected on Chameleon.

It can be seen that Eternia scales and performs similarly to the alterna-

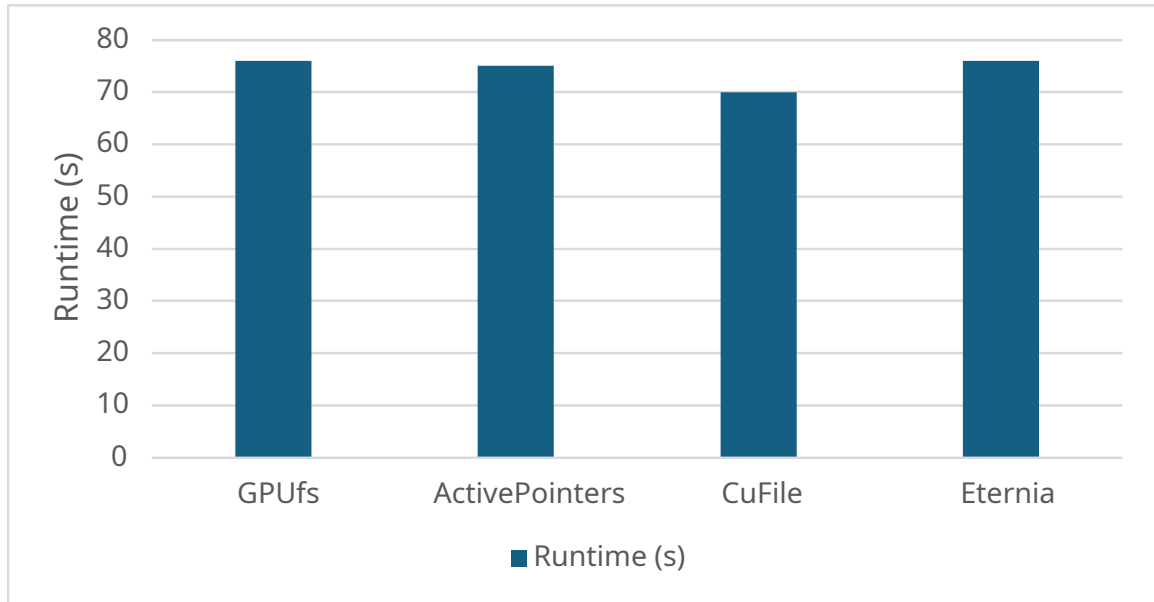


Figure 5.3. Comparison against state-of-the-art solutions. In this case where the data fits entirely in memory, Eternia has minimal overhead to applications.

tives. Surprisingly, cuFile did not provide a significant benefit. Despite support from NVIDIA engineers, we found the GPU_DIRECT path did not give significant benefit. We verified the logs did not fallback to traditional POSIX. We also verified the NUMA affinity of the GPU and application processes were the same. The negligible performance difference could be due to misconfiguration or limitations of the physical hardware. We also attempted to run this evaluation on the NCSA Delta machine over the DDN filesystem, which also has GDS support. However, we found that basic benchmark tools like gdsio failed to execute with errors on this system. In either case, we find that the GDS technology has very strict hardware requirements, has complex configuration, and is difficult to deploy, making the case for a portable I/O solution like Eternia important. Overall, Eternia does not introduce a significant overhead in cases where data fits in memory.

5.4.4 Impact of Gcache Page Granulairty and Partial Paging

This evaluation demonstrates the impact of gcache page size and partial paging on performance. We use a synthetic workload which randomly reads a dataset in 1MB chunks. We vary the gcache page size between 4KB and 64MB. We compare Eternia with and without partial paging support. We use 2 nodes for evaluation.

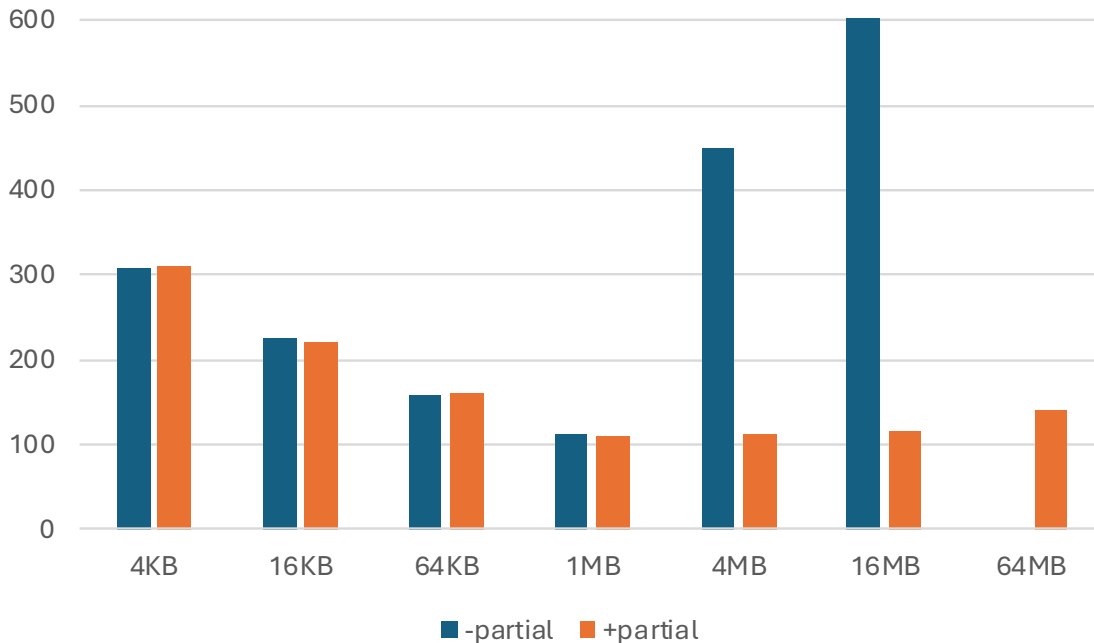


Figure 5.4. Comparison of different page size granularities on performance.

Overall, it can be seen from Figure 5.4 that setting the gcache page size too low results in significant performance degradation. This is because with lower page sizes, more I/O requests are generated and become more bounded by high I/O latency. Partial paging has no effect here as the page sizes are too small to divide further. At around 1MB the workload maximizes I/O bandwidth. Partial paging still has no effect, as the I/O is made in chunks of 1MB. At 4MB, the I/O time is nearly 4x slower without partial paging. This is because I/O amplification will fault unneeded data into the gcache. At 64MB + partial paging, I/O performance degrades 22% compared to the 1MB case. This is because a massive page size can reduce concurrency since

MemTasks regarding the same page are mapped to the same runtime thread. Overall, Eternia’s custom page granularity and partial paging settings offer significant performance opportunities for developers to improve performance depending on workload and hardware characteristics.

5.4.5 Efficiently Providing Persistence and Portability

We aim to measure the runtime’s impact on CPU utilization and performance. To do this, we compare the act of analyzing a 128GB dataset stored in several formats (binary, HDF5, and parquet) in a vector addition workload. These formats stress the runtime differently as they have different decoding operations. We set the maximum number of CPU cores for the Eternia runtime to be up to 16. The datasets are stored as 16 separate 8GB files to maximize I/O concurrency.

	Runtime	Cores
Binary	255	4
HDF5	300	10
Parquet	192	16

Figure 5.5. Comparison of performance and resource utilization of mapping different data formats with Eternia.

Overall, it can be seen from Table 5.5 that the runtime adapts to the different dataset formats differently. Binary results in the lowest CPU utilization since the data is already decoded. Only I/O tasks for reading the data from the HDD-based PFS are generated, so only 4 cores are allocated by the runtime. Parquet ends up being the fastest in overall performance. This is because parquet compresses the dataset, making its footprint half the size and thus cutting expensive disk reads in half. However, this results in all 16 cores being allocated for the workload. HDF5, while no compression is performed, also has expense for deserialization and 10 cores were used. Overall, it can be seen that the runtime effectively considers the performance

characteristics of MemTasks to improve energy efficiency by scaling CPU resources as-needed.

5.4.6 Increasing Dataset Resolution

In this evaluation, we demonstrate the impact of tiering memory and storage for increasing dataset resolution, measured by the total dataset size. To do this, we run Gray-Scott to produce grids of varying size. We vary the grid size to generate between $32GB$ and $256GB$ of data in total. We compare DDN Exascaler (NVMe, posix) and tiered I/O buffering system Hermes [3]) vs Eternia in terms of dataset size and memory utilization. Hermes uses the node-local NVMe, while DDN is a parallel filesystem operating over shared NVMe. Eternia is configured to allow the entire dataset into DRAM + NVMe. We run this evaluation over 4 nodes.

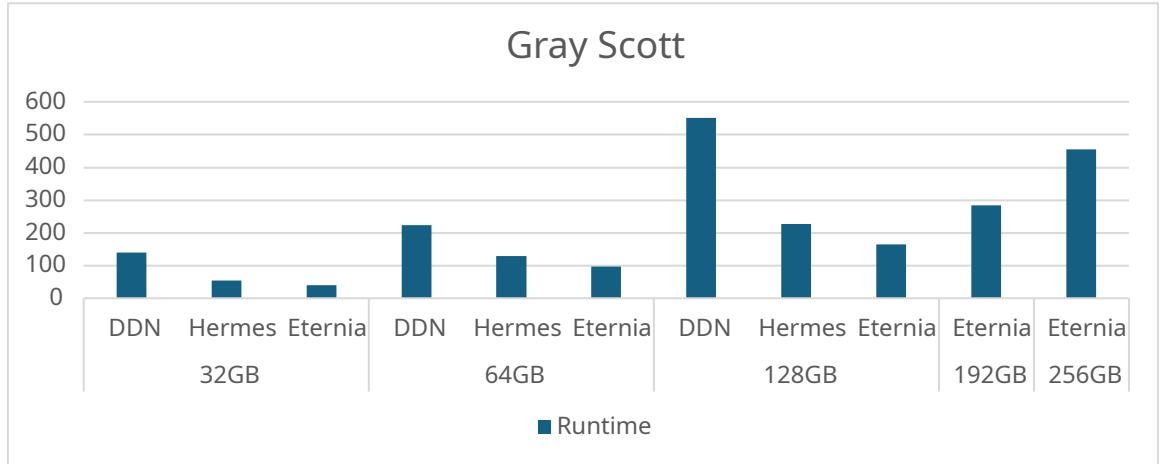


Figure 5.6. Increase resolution of GPU Gray Scott through tiered memory. After $128GB$, cuFile and DDN Gray Scott crash due to memory overutilization. Eternia is unbounded and can produce 2x the simulation data. It’s also at least 18% faster than other tiered I/O systems due to effective asynchronous data movement.

Overall, from Figure 5.6, it can be seen that Eternia performs at least 18% faster than the other implementations of Gray Scott. This is because Eternia places data during the first compute phase, while all others must wait for this phase to complete. Due to transactions, Eternia’s data organization algorithms are well-informed

which leads to improved placement decisions. When memory is fully utilized at 128GB, science can no longer progress for the other implementations. However, through Eternia, we are able to conduct experiments up to 2x larger by leveraging the node-local NVMeS to enlarge effective memory capacity. Currently, state-of-practice solutions limit science to the boundaries of main memory. Eternia eliminates this boundary by automatically tiering memory with storage enabling memory-constrained scientific problems to become feasible.

5.4.7 Impact of Tiering on Performance

We aim to measure the impact of different tiering strategies on end-to-end performance. To do this, we use KMeans as the application. The dataset for KMeans is randomly generated directly in Eternia right before KMeans is executed. We vary the tiers to use HBM, DRAM, NVMe, and PFS differently. We vary whether or not Eternia uses aggregation and prefetching which are used to inform data movement decisions. We also measure the cost of storage in the tiering strategy. A 4GB DRAM bounce-buffer is implicitly used per-node in each tiering strategy to handle I/O from NVMe or HDD to GPU.

Overall, it can be seen from Figure 5.7 that with each additional storage tier, performance improves as expected. 32G-32D is 25x faster than 32G-32H, for example. However, with this benefit comes a financial cost – 32G-32D is about 80x more expensive. Additionally, it can be seen that aggregation dramatically improves performance. Without aggregation, the average I/O size became approximately 4KB in size and the number of I/O requests multiplied by 32 (the number of threads per block in our case). This pattern has significantly worse performance on all devices (up to 10x), especially on HDDs which terminated before completion. Lastly, prefetching adds a modest performance improvement of up to 30% (e.g., 16D-16N). Since the access pattern is deterministic for KMeans, the prefetcher is able to make well-informed



Figure 5.7. Tiering study of Eternia for 4-node KMeans. G=GPU (HBM), D=DRAM, H=HDD, N=NVMe. The number indicates the amount of storage per-node given to the program (e.g., 32D means 32GB DRAM per node). Aggregation improves performance as much as 8x by reducing number of I/O ops and increasing I/O sizes. Prefetching improves performance as much as 30% by informing data placement. Introducing tiers always improves performance, but benefits related to cost.

decisions on the data to bring in. Overall, Eternia’s aggregation and prefetching optimizations are able to efficiently utilize arbitrarily-structured storage compositions and minimize the cost of expensive data transfers.

5.4.8 Effect of Lowering Memory Consumption

This evaluation measures the impact of reducing HBM consumption using optimal configurations of Eternia. We generate datasets for each application that exceed memory boundaries (64GB per node). We then progressively lower the amount of HBM given to each application between 32GB (maximum HBM) and 4GB per node. A 4GB DRAM bounce-buffer is used per-node to handle I/O from NVMe to GPU. We run each app with 4096 threads per node and for 4 nodes. For read workloads, datasets are initially stored in NVMe and loaded asynchronously into HBM as the program runs. For writes, datasets will eventually migrate from HBM to NVMe.



Figure 5.8. Memory reduction study of Eternia. 4096 threads per node, 4 nodes. Through performance matching, DRAM can be lowered as much as 4x while maintaining competitive (within 10%) performance of full DRAM capacity.

Overall, it can be seen from Figure 5.8 that when running out-of-core workloads, memory utilization can be dramatically reduced without significant impacts

to performance in each workload. At minimum, a 3x reduction was achieved and at best 4x. This result is mainly due to the fact that storage is many times slower than GPU. In Delta, the NVMe cluster achieves approximately 4-5GBps at optimal. However, with HBM, the compute kernels for each application run several times faster. It becomes wasteful to use this space while being stalled for heterogeneous data movements. After 4x, contention starts to occur between gcache and tcache. By default, tcache is 1MB. At 8GB, each thread gets a maximum of 2MB of gcache space, so there is little room for prefetching and there will be frequent stalls for data. Overall, Eternia can improve the energy-efficiency of data-intensive computations while maintaining competitive performance through active storage tiering.

5.5 Conclusion

In this work, we address the limitations of developing out-of-core, data-centric GPU algorithms through the design and implementation of Eternia: a distributed shared memory system that enlarges effective GPU memory capacity by intelligently managing tiered storage and memory resources. Eternia demonstrates that the development complexity of out-of-core GPU algorithms can be significantly reduced by allowing persistent datasets to be presented as memory while transparently placing data in complex, diverse storage. Eternia considers the full lifecycle of data, enabling GPUs to persistently store data in any desired format and ensure that important data is moved to long-term storage before temporary resources are revoked. When possible, Eternia offers GPUs direct paths to data to minimize CPU involvement on the critical path, boosting performance. We demonstrate how using a parallel transaction API can be used to gain substantial benefits in I/O performance by reducing I/O stalls and the number of I/O queries through batching. Evaluations show that Eternia can reduce HBM + DRAM consumption by as much as 4 $\times$ when compared to existing distributed programming approaches.

CHAPTER 6

RELATED WORKS

6.1 I/O Development Platforms

There are several approaches for developing I/O stacks. However, these methodologies are generally confined to specific hardware (either storage and processors), have limited extensibility, and do not consider the broad scope of AI application I/O requirements – such as the ability to program custom data transformations.

6.1.1 Filesystem Development Platforms

Tools have emerged to develop filesystems. FUSE [71] enables users to implement kernel-level filesystems in userspace. Several subsequent works have emerged extending FUSE and addressing its various performance issues [38,40]. Alternatively, Bento [15] is a platform that allows filesystems to be developed in userspace and interpreted in the kernel, also enabling incremental upgrades to filesystem code. These development platforms differ from Recombinant in several ways. First, fundamentally, filesystem development platforms are confined to the UNIX file abstraction, ignoring other representations such as object storage. These platforms also do not allow for the development of custom data operations (e.g., classification operators for fraud detection), limiting their flexibility to modern AI applications. Lastly, these toolkits do not enable customization of the entire I/O stack, such as I/O schedulers, limiting innovation and performance for AI workloads. Recombinant provides customization of all layers of the I/O stack, including I/O interfaces and active storage semantics.

6.1.2 Library I/O Stacks

Library I/O stacks are developed in userspace as shared libraries and linked directly into applications. These libraries are developed using modern interfaces to certain specialized storage devices. Drivers such as the Storage Performance Devel-

opment Kit (SPDK) [57] and hardware interfaces such as memory-mapped I/O for persistent memory [248] and compute express link (CXL) [115] offer more opportunity to develop specialized I/O stacks than what OS kernels currently expose. Several works exist that develop custom filesystems [35] and key-value stores [63]. However, these library I/O stacks are limited to particular types of hardware and developed as one-off solutions for particular programs. They are not designed to support a variety of programs and address diverse I/O requirements. However, Recombinant aims to provide a platform that can host several workload-specific, heterogeneous I/O stacks simultaneously to address diverse workload requirements while automatically balancing the contention between them.

6.1.3 Microkernels

A microkernel, such as MINIX3 [74] or SeL4 [75], is a minimalistic kernel architecture where the majority of OS services are provided as libraries developed in userspace, including device drivers and I/O systems. This provides enhanced development speed when compared to monolithic kernels (e.g., Linux), as most programming is done outside the kernel. However, microkernels are not used in production computing centers. As of November 2021, all Top500 supercomputing centers run Linux [76]. This is because Linux has a massive community of developers and is easily installed on any machine architecture. To utilize microkernels, computing centers would have to uproot their infrastructure and manually determine the set of modules required for their systems to run, including drivers, which is both difficult and expensive. For these reasons, Recombinant maintains compatibility with a variety of operating systems.

6.1.4 Flexible I/O Stacks

Work has been done to explore I/O stacks that can be adjusted to particular workload and hardware types. Labios [84] is a distributed label-based I/O system that proposes a flexible storage stack design that maps conflicting data representations into a unified label representation that encompasses the data and unary operations (e.g., compression) to perform on that data. Alternatively, Alluxio [249] provides a platform that unifies various I/O interfaces and storage systems to automatically tier data for a variety of storage systems and AI applications. While these works have some ability to customize I/O paths and offer portability across storage hardware, they do not explore multi-tenancy, processor heterogeneity, and dynamic load balancing.

6.1.5 Memory-Centric I/O Stacks

Memory Mapped I/O (MMIO) [158] enables applications to share data across processes and present persistent files as C-style arrays. MMIO simplifies programming out-of-core algorithms by removing the burden of deciding which parts of the file are buffered in main memory from the developer [158]. On POSIX-compatible OSes (e.g., Linux), the default page fault path can be customized to change prefetching and data placement policies by either editing the kernel directly or leveraging the `userfaultfd` [162] system call. However, this approach imposes several limitations onto developers. First, only the page fault path can be customized, not other layers of the I/O stack such as filesystems, I/O schedulers, and data transformations. Second, CPUs impose physical restrictions on page size granularity for MMIO, resulting in severe, unavoidable performance degradation due to poor I/O access patterns and I/O amplification [159].

6.2 Heterogeneous Storage and Memory Management

There are many works that discuss the matching of data to storage hardware. However, these works are generally built under the assumption of particular machine architectures and hardware devices and do not consider a variety of AI application requirements.

6.2.1 Heterogeneous I/O

Many works discuss methods to automatically match data to heterogeneous storage hardware. Strata [67] proposes a multi-tiered filesystem for single-node database applications. Assise [179] extends strata to form a distributed tiered filesystem prioritizing traditional cloud workloads, such as video streaming. Alternatively, Hermes [3] is a multi-tiered and extensible I/O buffering system that provides several prefetching and data placement policies to reduce the I/O bottleneck for scientific applications. Several works have emerged from its flexible design to improve I/O performance in various workloads. HCompress [83] describes optimizations made to Hermes to leverage hierarchical compression to optimize data movement among different tiers of storage. HFetch [178] describes a prefetching algorithm for Hermes for particular scientific data analysis codes. Hades [19] extends Hermes to transparently apply mathematical operators during data production to accelerate future analytics in producer-consumer workflows common in scientific environments. However, while these work provide efficient policies to automatically manage heterogeneous storage resources, Recombinant differs from them in several ways. First, the existing work focuses on CPU-centric database and scientific workloads, whereas Recombinant aims to leverage heterogeneous processing resources for modern AI applications. Furthermore, these works generally do not consider the contention caused in multi-tenant environments executing a variety of workloads simultaneously. During the course of this dissertation, the Hermes project was rebuilt as a specific plugin to the Recombi-

nant platform to provide it these enhanced features.

6.2.2 Heterogeneous Single-Node Memory

Several works have been proposed to leverage heterogeneous storage as extensions to main memory. uMMAP-IO [161], for example, utilizes this approach to extend virtual memory to SSDs and Hard Drives for HPC workloads, effectively increasing memory capacity. However, `userfaultfd` increases the cost of an individual page fault due to increased context switching between the kernel and userspace [163]. Many works aim to provide custom page fault handlers while avoiding `userfaultfd` [158, 164–167]. Aquila [164] and FlatFlash [167], for example, make kernel modifications to reduce software overheads of the MMIO path. To avoid kernel modification, Tri-cache [166] uses LLVM to prepend a custom buffer cache handler before all CPU load/store operations, removing a physical page fault entirely. In their specific page handler code, NVMeS are used as a temporary buffering layer via the Storage Performance Development Kit (SPDK). These works are focused on single-node, multi-threaded applications, not fully distributed, multi-process shared-memory applications.

6.2.3 Distributed Shared Memory (DSM)

In distributed memory environments, DSM allows applications to access remote memory as if it were its own memory. These works typically extend the virtual memory management subsystems of the OS to implement custom page fault handlers to provide the shared memory abstraction, although some use language features (e.g., operator overloading). A number of works have proposed designs and improvements to DSM systems. TreadMarks [153] uses virtual memory to provide a unified global address space, but suffers from extreme cache-coherence overheads. A1 [151] is a distributed in-memory graph database used by Microsoft to support the Bing search engine. A1 focuses on optimizing DSM latency for graph problems over commodity DRAM by using lightweight RDMA networks. Concordia [157] proposes a network protocol to reduce the overhead of DSM cache-coherency on fast networks by offloading the coherency protocols to programmable network switches. Leap [168] discusses a kernel-level algorithm to prefetch remote memory using RDMA into the Linux page cache. HotPot [154] explores a DSM that replaces DRAM with Intel Optane DC Persistent Memory (DCPMM), but does not investigate tiering algorithms. Alternatively, Grappa [169] provides specific optimizations of shared memory for graph-based and SQL-like workloads. Currently, DSMs focus on single-tier approaches primarily for Cloud workloads, and have not yet explored the implications of expanding memory capacity to nodes with multiple tiers of storage.

6.2.4 Partitioned Global Address Space (PGAS) Languages

Alternative to DSMs, PGAS languages provide distributed applications the illusion of a global unified memory address space through the use of language features [170]. The PGAS library manages the complexity of unifying the underlying disjoint memory systems and provides abstractions to allocate memory and inform data movement strategies [171]. To improve performance, PGAS libraries provide explicit APIs to fetch remote data as needed. A number of PGAS languages have been proposed, including OpenSHMEM [172], Charm++ [173], and UPC(++) [174]. UPC++ [174], for example, uses C++ to overload the address operator to abstract the complexity of mapping pointer offsets to physical memory locations. Synchronization primitives such as locks and futures are also provided. From these primitives, more complex objects such as unordered maps and queues can be implemented. Generally, these libraries have better performance than traditional DSMs, as they avoid page fault overheads and have finer control over message granularity and scheduling. However, developers are responsible for manually partitioning datasets into the PGAS library and tracking what data is local and remote, which can increase programming complexity and force strict requirements on application structure [169]. Lastly, PGAS libraries currently only consider remote DRAM, and have yet to explore expansion to other storage tiers such as local and remote NVMe and HDD. In addition, PGAS libraries do not directly offer data persistence and require the use of an external service to copy data into storage.

6.3 Mediating Contention on Storage and Processors

6.3.1 Symmetric Processor Contention

Load balancing algorithms are applied in many data-centric applications to improve resource utilization and reliability [122–125]. Container orchestration frameworks, such as Kubernetes [126], are often used to make reliable and elastic web services [127, 128]. Work has also been done to use these frameworks to load balance distributed I/O systems [129] such as CephFS [130]. Labios [84] presents a distributed I/O system that allows custom unary data operators (e.g., compression) to be defined and applied transparently in the I/O stack. This work provides configurations to allow administrators to determine the amount of CPU resources that can be expended to handle data operations. Alternatively, Hades [19] describes a data streaming services that transparently applies CPU-based data operators as background work and balances the contention caused by them with application threads by considering current resource utilization. These works differ from Recombinant because they focus on the contention caused on symmetric CPU resources. They do not provide methods to automatically schedule computations on heterogeneous processors while considering contention.

6.3.2 Heterogeneous Processor Scheduling and Contention

Alternatively, modern computing centers are adopting accelerators to offload I/O-related tasks from the CPU to improve energy efficiency, cost, and overall application performance [113]. To accomplish this, I/O stacks are being developed for particular accelerator technologies [9–12, 108, 110, 111, 113, 117–120]. DevFS [12] describes a single-node PMEM filesystem that executes on processors coupled with the PMEM device itself, offloading metadata management, crash consistency, and security to the storage device. LineFS [10] describes a distributed SmartNIC PMEM filesystem which additionally offloads networking operations, replication, and compression. Alternatively, works such as SmartSSD [110] and SmartShuffle [11] move data streaming pipelines to SmartNICs and CSDs, allowing stream-specific computations to occur as background tasks instead directly on the host CPU. However, these works differ from Recombinant in several ways. First, they are specialized solutions designed to solve particular problem types over particular hardware types and come with a rigid set of tightly coupled features. Unlike Recombinant, they do not explore extensible and evolving I/O stack designs to improve code reuse, development pace, and portability. In addition, they focus on offloading tasks to a particular accelerator type, such as SmartNICs, and do not currently consider heterogeneous machines containing various processing and storage hardware. Lastly, these works have limited consideration to cross-application I/O- and compute- interference in multi-tenant deployments. They investigate only relieving contention from the CPU, but not on the accelerators themselves.

6.3.3 Storage Contention

Several works discuss methods to mediate contention on storage resources. Harmonia [87] demonstrates the impact of contention on burst buffer resources in multi-tenant HPC environments. It demonstrates that I/O performance fluctuates dramatically depending on the I/O behavior of the running applications, which is largely due to the hardware characteristics of the underlying storage. It describes several heuristic policies to handle different potential objectives over burst buffers, such as maximizing resource utilization, priority queueing, and round-robin (fairness). This work leverages the bursty nature of scientific applications to improve coordination by monitoring the length of compute phases across programs. blk-switch [44] describes kernel-level optimizations for managing contention on both network and storage. In this work, applications are partitioned into two groups: low latency and high throughput. This provides an I/O scheduler that dynamically maps latency-sensitive requests (e.g., 4KB) to separate hardware queues from throughput-intensive requests (e.g., 64KB). The determination of class is static based on I/O size. Split Level I/O Scheduling [250] demonstrates several methods to mediate contention for Cloud workloads, including databases, hypervisors, and distributed filesystems. This work finds that storage contention is caused at several layers of the Linux I/O stack and proposes several optimizations to system call, page cache, and block I/O layers to maximize I/O performance. While each of these works demonstrate various angles and sources of storage contention in particular workloads, they differ from Recombinant in various ways. First, their contention policies do not consider the global utilization of resources. I/O operations are confined to the storage device they are made to. In Recombinant, data is containerized and can be migrated across machines in response to contention. Second, these works are not designed to account for the patterns of diverse AI workloads. The policies discussed are heuristic based primarily on I/O size and user-defined priority. Recombinant provides methods to dynamically

understand diverse application behavior through reinforcement learning and dynamic application and resource monitoring.

CHAPTER 7

CONCLUSION

7.1 Summary

In this dissertation, we first present a platform for building single-node I/O stacks. Traditionally, I/O systems have been developed within the confines of a centralized OS kernel. This led to monolithic and rigid storage systems that are limited by low development speed, expressiveness, and performance. Various assumptions are imposed including reliance on the UNIX-file abstraction, the POSIX standard, and a narrow set of I/O policies. However, this monolithic design philosophy makes it difficult to develop and deploy new I/O approaches to satisfy the rapidly-evolving I/O requirements of modern data-intensive applications. To address these issues, we propose LabStor: a modular and extensible platform for developing high-performance, customized I/O stacks outside of the kernel. LabStor enables single-purpose I/O modules (e.g, I/O schedulers) to be developed in the comfort of userspace and released as plug-ins, while end-users can compose these modules to form workload- and hardware-specific I/O stacks. Evaluations show that by switching to a fully modular design, tailored I/O stacks can yield performance improvements of up to 60% in various applications while allowing I/O code to be upgraded in milliseconds.

Next we present a platform for building distributed I/O stacks for heterogeneous storage and processing resources. Large-scale, data-intensive applications spanning HPC and Cloud have highly diverse I/O and computational requirements, resulting in the adoption of heterogeneous storage hardware and the development of specialized data-centric accelerators, such as SmartNICs, Processing In Memory (PIM), and computational storage. This has led to the development of many one-off storage solutions that are developed in the confines of a particular machine architecture and workload, resulting in costly redevelopment efforts, limited portability,

lack of customization, and suboptimal resource utilization. In this work, we extend the abstractions described in LabStor to be distributed and encompass both heterogeneous processors and storage to enable distributed, portable I/O stacks to be rapidly developed. Chimaera introduces a novel algorithm that matches computations to heterogeneous processing hardware based on live application monitoring and specific processor characteristics. Chimaera also introduces a novel intelligent process scheduling algorithm that automatically mediates resource contention across heterogeneous processors and storage by globally considering the highly dynamic I/O and computational characteristics across co-located I/O stacks when matching them to hardware to ensure high performance and resource utilization. This policy uses reinforcement learning techniques to automatically adapt to diverse application behaviors. Experimental results demonstrate that I/O paths composed in Chimaera can yield performance gains of up to 46% over alternatives under various workloads by utilizing heterogeneous processors. Chimaera is currently used as the foundations for the implementation of several funded projects, including Hermes, IoWarp, DTIO, and COEUS, totalling nearly \$10 million of funding.

Next, we leverage the I/O platforms discussed in this dissertation to develop specific I/O stacks to address the characteristics of data-intensive workloads. One such work regards leveraging I/O to improve memory capacity for CPU-based applications. Large-scale data analytics, scientific simulation, and deep learning codes perform massive computations on data greatly exceeding the bounds of main memory. These out-of-core algorithms suffer from severe data movement penalties, programming complexity, and limited code reuse. To solve this, computing sites have steadily increased DRAM capacity. However, this is not sustainable due to financial and environmental costs. A more elegant, low-cost, and portable solution is to expand memory to distributed multi-tiered storage. MegaMmap, which demonstrates a specific I/O stack built in Chimaera for memory-constrained data-intensive work-

loads, provides a new, intuitive memory abstraction that allows massive datasets to be presented as if they were in memory while automatically matching memory pages across heterogeneous memory and storage through novel, intelligent prefetching and data movement policies that factor dynamic access pattern and storage characteristics. Applications inform MegaMmap of its access pattern intent through a novel transactional memory API, allowing MegaMmap to generally to a diversity of applications. Evaluations showcase that MegaMmap reduces peak memory utilization by as much as $2.6\times$ compared to leading solutions while accomplishing similar or better performance.

Finally, we extend the memory abstraction paradigm to address the unique challenges of GPU-centric applications through Eternia. GPUs have become integral to many science and applications, but dataset sizes are now straining the bounds of GPU memory, and simply increasing HBM capacity is no longer sustainable due to extreme energy costs. Building upon the foundations established by MegaMmap, Eternia provides a distributed shared memory abstraction that enlarges effective GPU memory capacity through intelligent tiered DRAM and storage management. Eternia allows massive datasets to be presented as if in GPU memory while automatically managing data placement across distributed GPU memory and tiered storage. Through a parallel transactional memory API, GPU kernels can inform data placement, prefetching, and metadata caching algorithms to ensure critical data is efficiently accessible. Eternia addresses the full lifecycle of GPU data, enabling direct storage access when possible while ensuring persistence across diverse storage tiers. Evaluations demonstrate that Eternia can reduce HBM consumption for various workloads up to $4\times$ without sacrificing performance through effective storage tiering.

According to the achievements and findings in this work, we are able to make

the following conclusions:

1. The traditional, individualistic methodology for developing I/O stacks must be re-envisioned to address diverse data-intensive application I/O requirements. Switching to a modular I/O stack design allows workload-specific and hardware-optimized I/O paths to be constructed rapidly and easily maintained without sacrificing performance. We demonstrate up to 10x fewer lines of code due to efficient code reuse in addition to up to 60% performance improvements from using specially-composed I/O services.
2. Methods for utilizing the increasing storage and processor heterogeneity of modern computing environments must be improved to ensure the sustainability and performance of data-intensive workloads. Automatically matching I/O and computations to hardware allows improvements to both energy-efficiency and performance while leaving applications unburdened by these decisions. We demonstrate up to 4x peak memory utilization reduction by efficient hardware matching while maintaining comparable performance.
3. With the rise of diverse applications and the multi-tenant nature of modern computing environments, methods for sharing resources must be improved to minimize cross-application interference. Scheduling policies that factor dynamic and diverse application behavior and hardware characteristics improve performance and resource utilization in multi-tenant environments. We demonstrate that multi-tenant workloads benefit up to 80x lower p99 latency by intelligent orchestration of I/O operations.

The achievements we have accomplished to date have laid the foundations for developing, maintaining, and continuously evolving the next generation of large-scale I/O systems needed to handle the recent breakthroughs in AI techniques.

7.2 Future Work

Having successfully developed and evaluated our software tools across diverse applications spanning common machine learning algorithms (e.g., clustering, random forests), scientific simulation codes (e.g., Gray Scott, VPIC), and GPU-centric workloads through Eternia, we have demonstrated the effectiveness of our composable I/O approach across both CPU and GPU environments. The completion of Eternia represents a significant milestone in extending memory abstractions to GPU workloads, addressing the critical need for efficient distributed shared memory systems in GPU-accelerated applications.

Looking forward, several promising research directions emerge from this foundation. First, with the rapid advancement of large language models and computer vision applications, there is an opportunity to further optimize our GPU memory management techniques for transformer architectures and neural network training pipelines that exhibit unique access patterns. Additionally, emerging hardware technologies such as advanced CXL devices, computational storage, and next-generation interconnects present opportunities to extend our composable I/O framework to leverage these new capabilities automatically.

In addition, Chimaera is currently being used as the foundation for our recent \$5 million IoWarp project. With IoWarp, we aim to build a comprehensive I/O platform for scientific computing environments that leverages large language models to provide an intuitive interface to massive storage systems, job schedulers, and datasets, while optimizing data movement throughout complex, distributed machine architectures integrating diverse storage and processing resources. We intend to add new data operators to Chimaera to specifically support this system architecture. First, we intend to add custom data operators that transparently tag data with descriptive metadata to allow the LLM interface to determine the location of data based on user

prompts. For example, let's say a scientist puts the prompt, "Describe the data for the most recent weather simulation output." In this case, our Recombinant platform may transparently calculate quantities, such as the average temperature and precipitation, while the simulation runs. The LLM would then be able to read the metadata for the simulation's output and respond, "This is an HDF5 file. It was found that the average temperature and precipitation in the region was 84f and 2.5mm". In addition to improving data quality, we intend to add new data movement algorithms to improve performance for dynamic, multi-tenant AI workflows – particularly deep learning. Lastly, we intend to improve the overall evaluation and implementation of Chimaera to leverage real storage and processing accelerators using our community testbed: StoreHub.

BIBLIOGRAPHY

- [1] L. Logan, J. C. Garcia, J. Lofstead, X.-H. Sun, and A. Kougkas, “Labstor: A modular and extensible platform for developing high-performance, customized i/o stacks in userspace,” in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–15, IEEE, 2022. © 2022 IEEE. Reprinted, with permission, from Luke Logan; Jaime Cernuda Garcia; Jay Lofstead; Xian-He Sun; Anthony Kougkas, LabStor: A Modular and Extensible Platform for Developing High-Performance, Customized I/O Stacks in Userspace, 13-18 November 2022.
- [2] L. Logan, A. Kougkas, and X.-H. Sun, “Megammap: Blurring the boundary between memory and storage for data-intensive workloads,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*, SC ’24, IEEE Press, 2024. © 2024 IEEE. Reprinted, with permission, from Luke Logan; Anthony Kougkas; Xian-He Sun, MegaMmap: Blurring the Boundary Between Memory and Storage for Data-Intensive Workloads, 17-22 November 2024.
- [3] A. Kougkas, H. Devarajan, and X.-H. Sun, “Hermes: a heterogeneous-aware multi-tiered distributed I/O buffering system,” in *Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing*, pp. 219–230, 2018.
- [4] X.-H. Sun, G. Heber, and A. Kougkas, “IOWarp: A next-generation data management platform for scientific workflows.” <https://github.com/iowarp>, 2024. NSF Award 2411318.
- [5] Oak Ridge Leadership Computing Facility, *Summit User Guide*, 2024. https://docs.olcf.ornl.gov/systems/summit_user_guide.html.
- [6] Lawrence Livermore National Laboratory, *El Capitan*, 2024. <https://asc.llnl.gov/exascale/el-capitan>.
- [7] Argonne Leadership Computing Facility, *Aurora*, 2024. <https://www.alcf.anl.gov/aurora>.
- [8] Google Cloud, *Local SSD*, 2024. <https://cloud.google.com/compute/docs/disks/local-ssd>.
- [9] A. Psistakis, F. Chaix, and J. Torrellas, “Minos: Distributed consistency and persistency protocol implementation & offloading to smartnics,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 1–17, IEEE, 2024.
- [10] J. Kim, I. Jang, W. Reda, J. Im, M. Canini, D. Kostić, Y. Kwon, S. Peter, and E. Witchel, “Linefs: Efficient smartnic offload of a distributed file system with pipeline parallelism,” in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pp. 756–771, 2021.
- [11] J. Lin, T. Ji, X. Hao, H. Cha, Y. Le, X. Yu, and A. Akella, “Towards accelerating data intensive application’s shuffle process using smartnics,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 7, no. 2, pp. 1–23, 2023.

- [12] S. Kannan, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, Y. Wang, J. Xu, and G. Palani, “Designing a true {Direct-Access} file system with {DevFS},” in *16th USENIX Conference on File and Storage Technologies (FAST 18)*, pp. 241–256, 2018.
- [13] M. Torabzadehkashi, S. Rezaei, A. HeydariGorji, H. Bobarshad, V. Alves, and N. Bagherzadeh, “Computational storage: an efficient and scalable platform for big data and hpc applications,” *Journal of Big Data*, vol. 6, no. 1, p. 100, 2019.
- [14] R. Macedo, Y. Tanimura, J. Haga, V. Chidambaram, J. Pereira, and J. Paulo, “{PAIO}: General, portable {I/O} optimizations with minor application modifications,” in *20th USENIX Conference on File and Storage Technologies (FAST 22)*, pp. 413–428, 2022.
- [15] S. Miller, K. Zhang, M. Chen, R. Jennings, A. Chen, D. Zhuo, and T. Anderson, “High velocity kernel file systems with bento,” in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pp. 65–79, USENIX Association, Feb. 2021.
- [16] M. M. D. Bonnie, B. Ligon, M. Marshall, W. Ligon, N. Mills, E. Q. S. Sampson, S. Yang, and B. Wilson, “Orangefs: advancing pvfs,” in *USENIX Conference on File and Storage Technologies (FAST)*, 2011.
- [17] M. Cao, S. Bhattacharya, and T. Ts’o, “Ext4: The next generation of ext2/3 filesystem,” in *LSF*, 2007.
- [18] O. Rodeh, J. Bacik, and C. Mason, “Btrfs: The linux b-tree filesystem,” *ACM Transactions on Storage (TOS)*, vol. 9, no. 3, pp. 1–32, 2013.
- [19] J. Cernuda, L. Logan, A. Gainaru, S. Klasky, J. Lofstead, A. Kougkas, and X.-H. Sun, “Hades: A context-aware active storage framework for accelerating large-scale data analysis,” in *The 24th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing*, (Philadelphia), pp. 577–586, May 6-9 2024.
- [20] J. Cernuda, H. Devarajan, L. Logan, K. Bateman, N. Rajesh, J. Ye, A. Kougkas, and X.-H. Sun, “Hflow: A dynamic and elastic multi-layered i/o forwarder,” in *2021 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 114–124, IEEE, 2021.
- [21] C. Axenie, R. Tudoran, S. Bortoli, M. A. H. Hassan, C. S. Sánchez, and G. Brasche, “Dimensionality reduction for low-latency high-throughput fraud detection on datastreams,” in *2019 18th IEEE International Conference On Machine Learning And Applications (ICMLA)*, pp. 1170–1177, IEEE, 2019.
- [22] A. Harlap, H. Cui, W. Dai, J. Wei, G. R. Ganger, P. B. Gibbons, G. A. Gibson, and E. P. Xing, “Addressing the straggler problem for iterative convergent parallel ml,” in *Proceedings of the seventh ACM symposium on cloud computing*, pp. 98–111, 2016.
- [23] J. Fried, Z. Ruan, A. Ousterhout, and A. Belay, “Caladan: Mitigating interference at microsecond timescales,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pp. 281–297, 2020.

- [24] C. Iorgulescu, R. Azimi, Y. Kwon, S. Elnikety, M. Syamala, V. Narasayya, H. Herodotou, P. Tomita, A. Chen, J. Zhang, *et al.*, “{PerfIso}: Performance isolation for commercial {Latency-Sensitive} services,” in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pp. 519–532, 2018.
- [25] J. Hwang, M. Vuppalapati, S. Peter, and R. Agarwal, “Rearchitecting linux storage stack for μ s latency and high throughput,” in *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, pp. 113–128, 2021.
- [26] C. Chen, Y. Chen, and P. C. Roth, “Dosas: Mitigating the resource contention in active storage systems,” in *2012 IEEE International Conference on Cluster Computing*, pp. 164–172, IEEE, 2012.
- [27] L. Logan, J. Lofstead, S. Levy, P. Widener, X.-H. Sun, and A. Kougkas, “pmemcpy: a simple, lightweight, and portable i/o library for storing data in persistent memory,” in *2021 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 664–670, IEEE, 2021.
- [28] F. Wang, S. Oral, G. Shipman, O. Drokin, T. Wang, and I. Huang, “Understanding lustre filesystem internals,” *Oak Ridge National Laboratory, National Center for Computational Sciences, Tech. Rep*, vol. 120, 2009.
- [29] A. Aghayev, S. Weil, M. Kuchnik, M. Nelson, G. R. Ganger, and G. Amvrosiadis, “File systems unfit as distributed storage backends: Lessons from 10 years of ceph evolution,” in *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP ’19*, (New York, NY, USA), p. 353–369, Association for Computing Machinery, 2019.
- [30] J. Xu and S. Swanson, “{NOVA}: A log-structured file system for hybrid {Volatile/Non-volatile} main memories,” in *14th USENIX Conference on File and Storage Technologies (FAST 16)*, pp. 323–338, 2016.
- [31] H.-J. Kim, Y.-S. Lee, and J.-S. Kim, “{NVMeDirect}: A user-space {I/O} framework for application-specific optimization on {NVMe}{SSDs},” in *8th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 16)*, 2016.
- [32] F. Hupfeld, T. Cortes, B. Kolbeck, J. Stender, E. Focht, M. Hess, J. Malo, J. Marti, and E. Cesario, “The xtremfs architecture—a case for object-based file systems in grids,” *Concurrency and computation: Practice and experience*, vol. 20, no. 17, pp. 2049–2060, 2008.
- [33] B. Welch, M. Unangst, Z. Abbasi, G. A. Gibson, B. Mueller, J. Small, J. Zelenka, and B. Zhou, “Scalable performance of the panasas parallel file system.,” in *FAST*, vol. 8, pp. 1–17, 2008.
- [34] S. Zheng, M. Hoseinzadeh, and S. Swanson, “Ziggurat: A tiered file system for {Non-Volatile} main memories and disks,” in *17th USENIX Conference on File and Storage Technologies (FAST 19)*, pp. 207–219, 2019.
- [35] R. Kadekodi, S. K. Lee, S. Kashyap, T. Kim, A. Kolli, and V. Chidambaram, “Splitfs: Reducing software overhead in file systems for persistent memory,” in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pp. 494–508, 2019.

- [36] S. Guignani and X. Lu, “Dstore: A fast, tailless, and quiescent-free object store for pmem,” in *Proceedings of the 30th International Symposium on High-Performance Parallel and Distributed Computing*, pp. 31–43, 2021.
- [37] G. Lee, S. Shin, W. Song, T. J. Ham, J. W. Lee, and J. Jeong, “Asynchronous i/o stack: A low-latency kernel i/o stack for ultra-low latency ssds,” in *2019 {USENIX} Annual Technical Conference ({USENIX}{ATC} 19)*, pp. 603–616, 2019.
- [38] Q. Huai, W. Hsu, J. Lu, H. Liang, H. Xu, and W. Chen, “{XFUSE}: An infrastructure for running filesystem services in user space,” in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pp. 863–875, 2021.
- [39] J. Liu, A. Rebello, Y. Dai, C. Ye, S. Kannan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “Scale and performance in a filesystem semi-microkernel,” in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, SOSP ’21*, (New York, NY, USA), p. 819–835, Association for Computing Machinery, 2021.
- [40] Y. Zhu, T. Wang, K. Mohror, A. Moody, K. Sato, M. Khan, and W. Yu, “Direct-fuse: Removing the middleman for high-performance fuse file system support,” in *Proceedings of the 8th International Workshop on Runtime and Operating Systems for Supercomputers*, pp. 1–8, 2018.
- [41] M. Marty, M. de Kruijf, J. Adriaens, C. Alfeld, S. Bauer, C. Contavalli, M. Dalton, N. Dukkupati, W. C. Evans, S. Gribble, *et al.*, “Snap: A microkernel approach to host networking,” in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pp. 399–413, 2019.
- [42] I. Zhang, J. Liu, A. Austin, M. L. Roberts, and A. Badam, “I’m not dead yet! the role of the operating system in a kernel-bypass era,” in *Proceedings of the Workshop on Hot Topics in Operating Systems, HotOS ’19*, (New York, NY, USA), p. 73–80, Association for Computing Machinery, 2019.
- [43] D. Ji, Q. Zhang, S. Zhao, Z. Shi, and Y. Guan, “Microtee: designing tee os based on the microkernel architecture,” in *2019 18th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/13th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)*, pp. 26–33, IEEE, 2019.
- [44] J. Hwang, M. Vuppapapati, S. Peter, and R. Agarwal, “Rearchitecting linux storage stack for μ s latency and high throughput,” in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pp. 113–128, USENIX Association, July 2021.
- [45] M. Hedayati, K. Shen, M. L. Scott, and M. Marty, “{Multi-Queue} fair queuing,” in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pp. 301–314, 2019.
- [46] M. Yi, M. Lee, and Y. I. Eom, “Cffq: I/o scheduler for providing fairness and high performance in ssd devices,” in *Proceedings of the 11th International Conference on Ubiquitous Information Management and Communication*, pp. 1–6, 2017.

- [47] M. Lee, D. H. Kang, M. Lee, and Y. I. Eom, “Improving read performance by isolating multiple queues in nvme ssds,” in *Proceedings of the 11th International Conference on Ubiquitous Information Management and Communication*, pp. 1–6, 2017.
- [48] J. Woo, M. Ahn, G. Lee, and J. Jeong, “{D2FQ}:{Device-Direct} fair queueing for {NVMe}{SSDs},” in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pp. 403–415, 2021.
- [49] P. Mishra and A. K. Somani, “Host managed contention avoidance storage solutions for big data,” *Journal of Big Data*, vol. 4, no. 1, pp. 1–42, 2017.
- [50] Y. Chen, Y. Lu, F. Yang, Q. Wang, Y. Wang, and J. Shu, “Flatstore: An efficient log-structured key-value storage engine for persistent memory,” in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 1077–1091, 2020.
- [51] B. Peng, H. Zhang, J. Yao, Y. Dong, Y. Xu, and H. Guan, “{MDev-NVMe}: A {NVMe} storage virtualization solution with mediated {Pass-Through},” in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pp. 665–676, 2018.
- [52] B. Peng, J. Yao, Y. Dong, and H. Guan, “Mdev-nvme: Mediated pass-through nvme virtualization solution with adaptive polling,” *IEEE Transactions on Computers*, 2020.
- [53] S. Peter, J. Li, I. Zhang, D. R. K. Ports, D. Woos, A. Krishnamurthy, T. Anderson, and T. Roscoe, “Arrakis: The operating system is the control plane,” *ACM Trans. Comput. Syst.*, vol. 33, nov 2015.
- [54] S. Kannan, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, Y. Wang, J. Xu, and G. Palani, “Designing a true Direct-Access file system with DevFS,” in *16th USENIX Conference on File and Storage Technologies (FAST 18)*, (Oakland, CA), pp. 241–256, USENIX Association, Feb. 2018.
- [55] M. Dong, H. Bu, J. Yi, B. Dong, and H. Chen, “Performance and protection in the zofs user-space nvm file system,” in *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP ’19*, (New York, NY, USA), p. 478–493, Association for Computing Machinery, 2019.
- [56] “Direct access for files,” 2014.
- [57] Z. Yang, J. R. Harris, B. Walker, D. Verkamp, C. Liu, C. Chang, G. Cao, J. Stern, V. Verma, and L. E. Paul, “Spdk: A development kit to build high performance storage applications,” in *2017 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, pp. 154–161, 2017.
- [58] M. Bjørling, J. Gonzalez, and P. Bonnet, “{LightNVM}: The linux {Open-Channel}{SSD} subsystem,” in *15th USENIX Conference on File and Storage Technologies (FAST 17)*, pp. 359–374, 2017.
- [59] N. Moti, F. Schimmelpfennig, R. Salkhordeh, D. Klopp, T. Cortes, U. Rückert, and A. Brinkmann, “Simurgh: A fully decentralized and secure nvmm user space file system,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC ’21*, (New York, NY, USA), Association for Computing Machinery, 2021.

- [60] Y. Chen, Y. Lu, B. Zhu, and J. Shu, “Kernel/user-level collaborative persistent memory file system with efficiency and protection,” *arXiv preprint arXiv:1908.10740*, 2019.
- [61] Y. Lu, J. Shu, Y. Chen, and T. Li, “Octopus: an {RDMA-enabled} distributed persistent memory file system,” in *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, pp. 773–785, 2017.
- [62] Y. Chen, Y. Lu, B. Zhu, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, and J. Shu, “Scalable persistent memory file system with {Kernel-Userspace} collaboration,” in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pp. 81–95, 2021.
- [63] W. Zhang, X. Zhao, S. Jiang, and H. Jiang, “Chameleondb: a key-value store for optane persistent memory,” in *Proceedings of the Sixteenth European Conference on Computer Systems*, pp. 194–209, 2021.
- [64] T. Vinçon, A. Bernhardt, I. Petrov, L. Weber, and A. Koch, “nkx: near-data processing with kv-stores on native computational storage,” in *Proceedings of the 16th International Workshop on Data Management on New Hardware*, pp. 1–11, 2020.
- [65] B. Zhang and D. H. Du, “Nvlsm: A persistent memory key-value store using log-structured merge tree with accumulative compaction,” *ACM Transactions on Storage (TOS)*, vol. 17, no. 3, pp. 1–26, 2021.
- [66] T. Bisson, K. Chen, C. Choi, V. Balakrishnan, and Y.-s. Kee, “Crail-kv: A high-performance distributed key-value store leveraging native kv-ssds over nvme-of,” in *2018 IEEE 37th International Performance Computing and Communications Conference (IPCCC)*, pp. 1–8, IEEE, 2018.
- [67] Y. Kwon, H. Fingler, T. Hunt, S. Peter, E. Witchel, and T. Anderson, “Strata: A cross media file system,” in *Proceedings of the 26th Symposium on Operating Systems Principles*, pp. 460–477, 2017.
- [68] W. Wang and S. Diestelhorst, “Quantify the performance overheads of pmdk,” in *Proceedings of the International Symposium on Memory Systems*, pp. 50–52, 2018.
- [69] L. Zhang and S. Swanson, “Pangolin: A {Fault-Tolerant} persistent memory programming library,” in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pp. 897–912, 2019.
- [70] Y. Ren, C. Min, and S. Kannan, “{CrossFS}: A cross-layered {Direct-Access} file system,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pp. 137–154, 2020.
- [71] T. L. Kernel, “Fuse.”
- [72] J. Lofstead and A. Younge, “Gestalt computing: Hybrid traditional hpc and cloud hardware and software support,” *CLOUD COMPUTING 2022*, p. 63, 2022.
- [73] J. Yang, J. Kim, M. Hoseinzadeh, J. Izraelevitz, and S. Swanson, “An empirical guide to the behavior and use of scalable persistent memory,” in *18th {USENIX} Conference on File and Storage Technologies ({FAST} 20)*, pp. 169–182, 2020.

- [74] J. N. Herder, H. Bos, B. Gras, P. Homburg, and A. S. Tanenbaum, “Minix 3: A highly reliable, self-repairing operating system,” *ACM SIGOPS Operating Systems Review*, vol. 40, no. 3, pp. 80–89, 2006.
- [75] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, *et al.*, “sel4: Formal verification of an os kernel,” in *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, pp. 207–220, 2009.
- [76] Top500, “Operating system family / linux,” November 2021.
- [77] S. McClure, A. Ousterhout, S. Shenker, and S. Ratnasamy, “Efficient scheduling policies for {Microsecond-Scale} tasks,” in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pp. 1–18, 2022.
- [78] J. Liu, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, and S. Kannan, “File systems as processes,” in *11th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 19)*, (Renton, WA), USENIX Association, July 2019.
- [79] I. Zhang, A. Raybuck, P. Patel, K. Olynyk, J. Nelson, O. S. N. Leija, A. Martinez, J. Liu, A. K. Simpson, S. Jayakar, P. H. Penna, M. Demoulin, P. Choudhury, and A. Badam, “The demikernel datapath os architecture for microsecond-scale datacenter systems,” in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, SOSP ’21*, (New York, NY, USA), p. 195–211, Association for Computing Machinery, 2021.
- [80] N. Moti, F. Schimmelpfennig, R. Salkhordeh, D. Klopp, T. Cortes, U. Rückert, and A. Brinkmann, “Simurgh: a fully decentralized and secure nvmm user space file system,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–14, 2021.
- [81] R. R. Curtin, J. R. Cline, N. P. Slagle, W. B. March, P. Ram, N. A. Mehta, and A. G. Gray, “Mlpack: A scalable c++ machine learning library,” *Journal of Machine Learning Research*, vol. 14, no. Mar, pp. 801–805, 2013.
- [82] R. B. Ross, G. Amvrosiadis, P. Carns, C. D. Cranor, M. Dorier, K. Harms, G. Ganger, G. Gibson, S. K. Gutierrez, R. Latham, *et al.*, “Mochi: Composing data services for high-performance computing environments,” *Journal of Computer Science and Technology*, vol. 35, no. 1, pp. 121–144, 2020.
- [83] H. Devarajan, A. Kougkas, L. Logan, and X.-H. Sun, “Hcompress: Hierarchical data compression for multi-tiered storage environments,” in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 557–566, IEEE, 2020.
- [84] A. Kougkas, H. Devarajan, J. Lofstead, and X.-H. Sun, “Labios: A distributed label-based i/o system,” in *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*, pp. 13–24, 2019.
- [85] A. Kougkas, H. Devarajan, and X.-H. Sun, “Iris: I/o redirection via integrated storage,” in *Proceedings of the 2018 International Conference on Supercomputing*, pp. 33–42, 2018.

- [86] H. Devarajan, H. Zheng, A. Kougkas, X.-H. Sun, and V. Vishwanath, “Dlio: A data-centric benchmark for scientific deep learning applications,” in *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, pp. 81–91, IEEE, 2021.
- [87] A. Kougkas, H. Devarajan, X.-H. Sun, and J. Lofstead, “Harmonia: An interference-aware dynamic i/o scheduler for shared non-volatile burst buffers,” in *2018 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 290–301, IEEE, 2018.
- [88] L. Soares and M. Stumm, “{FlexSC}: Flexible system call scheduling with {Exception-Less} system calls,” in *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*, 2010.
- [89] W. J. Stewart, *Probability, Markov chains, queues, and simulation*. Princeton university press, 2009.
- [90] J. U. Farley and L. W. Ring, “A stochastic model of supermarket traffic flow,” *Operations Research*, vol. 14, no. 4, pp. 555–567, 1966.
- [91] L. Torvalds, “Linux kernel github,” 2016. commit hash: 1f5bd336b9150560458b03460cbcfcbcf8995b1.
- [92] K. Keahey, J. Anderson, Z. Zhen, P. Riteau, P. Ruth, D. Stanzione, M. Cevik, J. Colleran, H. S. Gunawi, C. Hammock, J. Mambretti, A. Barnes, F. Halbach, A. Rocha, and J. Stubbs, “Lessons learned from the chameleon testbed,” in *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC ’20)*, USENIX Association, July 2020.
- [93] C. Min, S. Kashyap, S. Maass, and T. Kim, “Understanding manycore scalability of file systems,” in *2016 USENIX Annual Technical Conference (USENIX ATC 16)*, pp. 71–85, 2016.
- [94] R. McDougall and J. Mauro, “Filebench,” URL: <http://www.nfsv4bat.org/Documents/nasconf/2004/filebench.pdf> (Cited on page 56.), 2005.
- [95] X. Ji, B. Yang, T. Zhang, X. Ma, X. Zhu, X. Wang, N. El-Sayed, J. Zhai, W. Liu, and W. Xue, “Automatic,{Application-Aware}{I/O} forwarding resource allocation,” in *17th USENIX Conference on File and Storage Technologies (FAST 19)*, pp. 265–279, 2019.
- [96] A. K. Paul, B. Wang, N. Rutman, C. Spitz, and A. R. Butt, “Efficient metadata indexing for hpc storage systems,” in *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*, pp. 162–171, IEEE, 2020.
- [97] S. Patil, K. Ren, and G. Gibson, “A case for scaling hpc metadata performance through de-specialization,” in *2012 SC Companion: High Performance Computing, Networking Storage and Analysis*, pp. 30–35, IEEE, 2012.
- [98] S. Byna, J. Chou, O. Rubel, H. Karimabadi, W. S. Daught, V. Roytershteyn, E. W. Bethel, M. Howison, K.-J. Hsu, K.-W. Lin, *et al.*, “Parallel i/o, analysis, and visualization of a trillion particle simulation,” in *SC’12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pp. 1–12, IEEE, 2012.

- [99] M. M. A. Patwary, S. Byna, N. R. Satish, N. Sundaram, Z. Lukić, V. Roytershteyn, M. J. Anderson, Y. Yao, and P. Dubey, “Bd-cats: big data clustering at trillion particle scale,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–12, 2015.
- [100] J. E. Pearson, “Complex patterns in a simple system,” *Science*, vol. 261, no. 5118, pp. 189–192, 1993.
- [101] A. Shanbhag, N. Tatbul, D. Cohen, and S. Madden, “Large-scale in-memory analytics on intel® optane™ dc persistent memory,” in *Proceedings of the 16th International Workshop on Data Management on New Hardware*, DaMoN ’20, (New York, NY, USA), Association for Computing Machinery, 2020.
- [102] D. D. Sharma, “Compute express link®: An open industry-standard interconnect enabling heterogeneous data-centric computing,” in *2022 IEEE Symposium on High-Performance Interconnects (HOTI)*, pp. 5–12, IEEE, 2022.
- [103] J. Lüttgau, S. Snyder, P. Carns, J. M. Wozniak, J. Kunkel, and T. Ludwig, “Toward understanding i/o behavior in hpc workflows,” in *2018 IEEE/ACM 3rd International Workshop on Parallel Data Storage & Data Intensive Scalable Computing Systems (PDSW-DISCS)*, pp. 64–75, 2018.
- [104] P. Schwan *et al.*, “Lustre: Building a file system for 1000-node clusters,” in *Proceedings of the 2003 Linux symposium*, vol. 2003, pp. 380–386, 2003.
- [105] M. J. Brim, A. T. Moody, S.-H. Lim, R. Miller, S. Boehm, C. Stanavige, K. M. Mohror, and S. Oral, “Unifyfs: A user-level shared file system for unified access to distributed local storage,” in *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 290–300, IEEE, 2023.
- [106] L. Logan, J. Lofstead, S. Levy, P. Widener, X.-H. Sun, and A. Kougkas, “pmem-cpy: a simple, lightweight, and portable i/o library for storing data in persistent memory,” in *2021 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 664–670, IEEE, 2021.
- [107] X. Yang, Y. Zhang, H. Chen, C. Sun, F. Li, and W. Zhou, “Polardb-scc: A cloud-native database ensuring low latency for strongly consistent reads,” *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3754–3767, 2023.
- [108] W. Cao, Y. Liu, Z. Cheng, N. Zheng, W. Li, W. Wu, L. Ouyang, P. Wang, Y. Wang, R. Kuan, *et al.*, “{POLARDB} meets computational storage: Efficiently support analytical workloads in {Cloud-Native} relational database,” in *18th USENIX conference on file and storage technologies (FAST 20)*, pp. 29–41, 2020.
- [109] H. N. Schuh, W. Liang, M. Liu, J. Nelson, and A. Krishnamurthy, “Xenic: Smartnic-accelerated distributed transactions,” in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pp. 740–755, 2021.
- [110] J. H. Lee, H. Zhang, V. Lagrange, P. Krishnamoorthy, X. Zhao, and Y. S. Ki, “Smartssd: Fpga accelerated near-storage data analytics on ssd,” *IEEE Computer architecture letters*, vol. 19, no. 2, pp. 110–113, 2020.
- [111] G. Lettieri, A. Fais, G. Antichi, and G. Procissi, “Smartnic-accelerated stream processing analytics,” in *2023 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, pp. 135–140, IEEE, 2023.

- [112] NVIDIA, “Nvidia bluefield dpu,” 2020. Accessed: 2024-08-02.
- [113] M. Liu, S. Peter, A. Krishnamurthy, and P. M. Phothilimthana, “E3: {Energy-Efficient} microservices on {SmartNIC-Accelerated} servers,” in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pp. 363–378, 2019.
- [114] TOP500, “June 2024 top500 list,” 2024. Accessed: 2024-08-02.
- [115] D. D. Sharma, “Compute express link,” *CXL Consortium White Paper*, 2019.
- [116] Z. Wang, Y. Guo, K. Lu, J. Wan, D. Wang, T. Yao, and H. Wu, “Rcmp: Reconstructing rdma-based memory disaggregation via cxl,” *ACM Transactions on Architecture and Code Optimization*, vol. 21, no. 1, pp. 1–26, 2024.
- [117] D. Gouk, M. Kwon, H. Bae, and M. Jung, “Dockerssd: Containerized in-storage processing and hardware acceleration for computational ssds,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 379–394, IEEE, 2024.
- [118] D. Bansal, G. DeGrace, R. Tewari, M. Zygmunt, J. Grantham, S. Gai, M. Baldi, K. Doddapaneni, A. Selvarajan, A. Arumugam, *et al.*, “Disaggregating stateful network functions,” in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pp. 1469–1487, 2023.
- [119] W. Lin, Y. Shan, R. Kosta, A. Krishnamurthy, and Y. Zhang, “Supernic: An fpga-based, cloud-oriented smartnic,” in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, pp. 130–141, 2024.
- [120] L.-C. Chen, C.-C. Ho, and Y.-H. Chang, “Uppipe: A novel pipeline management on in-memory processors for rna-seq quantification,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6, IEEE, 2023.
- [121] T. L. K. D. Team, “Filesystem in userspace (fuse),” 2024. Accessed: 2024-08-02.
- [122] S. K. Mishra, B. Sahoo, and P. P. Parida, “Load balancing in cloud computing: a big picture,” *Journal of King Saud University-Computer and Information Sciences*, vol. 32, no. 2, pp. 149–158, 2020.
- [123] K. Al Nuaimi, N. Mohamed, M. Al Nuaimi, and J. Al-Jaroodi, “A survey of load balancing in cloud computing: Challenges and algorithms,” in *2012 second symposium on network cloud computing and applications*, pp. 137–142, IEEE, 2012.
- [124] K. Bateman, N. Rajesh, J. C. Garcia, L. Logan, J. Ye, S. Herbein, A. Kougkas, and X.-H. Sun, “Luxio: Intelligent resource provisioning and auto-configuration for storage services,” in *2022 IEEE 29th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, pp. 246–255, IEEE, 2022.
- [125] N. Rajesh, H. Devarajan, J. C. Garcia, K. Bateman, L. Logan, J. Ye, A. Kougkas, and X.-H. Sun, “Apollo: An ml-assisted real-time storage resource observer,” in *Proceedings of the 30th International Symposium on High-Performance Parallel and Distributed Computing*, pp. 147–159, 2021.
- [126] Kubernetes, “Kubernetes documentation,” 2024. Accessed: 2024-08-02.

- [127] M. Imran, V. Kuznetsov, K. M. Dziedziniwicz-Wojcik, A. Pfeiffer, P. Parrigopoulos, S. Trigazis, T. Tedeschi, and D. Ciangottini, “Migration of cmsweb cluster at cern to kubernetes: a comprehensive study,” *Cluster Computing*, vol. 24, no. 4, pp. 3085–3099, 2021.
- [128] K. Kaur, F. Guillemin, and F. Sailhan, “Live migration of containerized microservices between remote kubernetes clusters,” in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications Workshops (INFOCOM WK-SHPS)*, pp. 1–6, IEEE, 2023.
- [129] Rook, “Rook documentation,” 2024. Accessed: 2024-08-02.
- [130] A. Aghayev, S. Weil, M. Kuchnik, M. Nelson, G. R. Ganger, and G. Amvrosiadis, “File systems unfit as distributed storage backends: lessons from 10 years of ceph evolution,” in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pp. 353–369, 2019.
- [131] C. Wang, K. Mohror, and M. Snir, “File system semantics requirements of hpc applications,” in *Proceedings of the 30th International Symposium on High-Performance Parallel and Distributed Computing*, pp. 19–30, 2021.
- [132] J. Liu, Q. Koziol, G. F. Butler, N. Fortner, M. Chaarawi, H. Tang, S. Byna, G. K. Lockwood, R. Cheema, K. A. Kallback-Rose, *et al.*, “Evaluation of hpc application i/o on object storage systems,” in *2018 IEEE/ACM 3rd International Workshop on Parallel Data Storage & Data Intensive Scalable Computing Systems (PDSW-DISCS)*, pp. 24–34, IEEE, 2018.
- [133] C. Engelmann, G. R. Vallee, T. Naughton, and S. L. Scott, “Proactive fault tolerance using preemptive migration,” in *2009 17th Euromicro International Conference on Parallel, Distributed and Network-based Processing*, pp. 252–257, IEEE, 2009.
- [134] H. Devarajan, A. Kougkas, and X.-H. Sun, “Hreplica: a dynamic data replication engine with adaptive compression for multi-tiered storage,” in *2020 IEEE International Conference on Big Data (Big Data)*, pp. 256–265, IEEE, 2020.
- [135] T. Chajed, J. Tassarotti, M. Theng, R. Jung, M. F. Kaashoek, and N. Zeldovich, “Gojournal: a verified, concurrent, crash-safe journaling system,” in *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, pp. 423–439, 2021.
- [136] Y. Shan, Y. Huang, Y. Chen, and Y. Zhang, “{LegoOS}: A disseminated, distributed {OS} for hardware resource disaggregation,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pp. 69–87, 2018.
- [137] Mellanox, “Nvidia mellanox bluefield data processing unit,” tech. rep., NVIDIA, 2020.
- [138] IOR, “Ior documentation,” 2024. Accessed: 2024-08-02.
- [139] Google, “Snappy — a fast compressor/decompressor,” 2019.
- [140] S. Byna, J. Chou, O. Rubel, H. Karimabadi, W. S. Daughter, *et al.*, “Parallel I/O, analysis, and visualization of a trillion particle simulation,” in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pp. 1–12, IEEE, 2012.

- [141] P. Chougule, A. Thakare, P. Kale, M. Gole, and P. Nanekar, “Genetic k-means algorithm for credit card fraud detection,” *International Journal of Computer Science and Information Technologies (IJCSIT)*, vol. 6, no. 2, pp. 1724–1727, 2015.
- [142] “Gray-scott simulation code.”
- [143] M. I. Jordan and T. M. Mitchell, “Machine learning: Trends, perspectives, and prospects,” *Science*, vol. 349, no. 6245, pp. 255–260, 2015.
- [144] C. Adams, “Neutrino and cosmic tagging with unet,” 2015.
- [145] W. F. Godoy, N. Podhorszki, R. Wang, C. Atkins, G. Eisenhauer, J. Gu, P. Davis, J. Choi, K. Germaschewski, K. Huck, A. Huebl, M. Kim, J. Kress, T. Kurc, Q. Liu, J. Logan, K. Mehta, G. Ostrouchov, M. Parashar, F. Poeschel, D. Pugmire, E. Suchyta, K. Takahashi, N. Thompson, S. Tsutsumi, L. Wan, M. Wolf, K. Wu, and S. Klasky, “Adios 2: The adaptable input output system. a framework for high-performance data management,” *SoftwareX*, vol. 12, p. 100561, 2020.
- [146] Y.-H. Liu and X.-H. Sun, “Lpm: concurrency-driven layered performance matching,” in *2015 44th International Conference on Parallel Processing*, pp. 879–888, IEEE, 2015.
- [147] A. Pupykina and G. Agosta, “Survey of memory management techniques for hpc and cloud computing,” *IEEE Access*, vol. 7, pp. 167351–167373, 2019.
- [148] Q. He, S. Zhou, B. Kobler, D. Duffy, and T. McGlynn, “Case study for running hpc applications in public clouds,” in *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing*, pp. 395–401, 2010.
- [149] K. H. Lee *et al.*, *A strategic analysis of the DRAM industry after the year 2000*. PhD thesis, Massachusetts Institute of Technology, 2013.
- [150] S. Mittal, “A survey of architectural techniques for dram power management,” *International Journal of High Performance Systems Architecture*, vol. 4, no. 2, pp. 110–119, 2012.
- [151] C. Buragohain, K. M. Risvik, P. Brett, M. Castro, W. Cho, J. Cowhig, N. Gloy, K. Kalyanaraman, R. Khanna, J. Pao, *et al.*, “A1: A distributed in-memory graph database,” in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pp. 329–344, 2020.
- [152] J. Lee, H. Kim, and R. Vuduc, “When prefetching works, when it doesn’t, and why,” *ACM Trans. Archit. Code Optim.*, vol. 9, mar 2012.
- [153] C. Amza, A. L. Cox, S. Dwarkadas, P. Keleher, H. Lu, R. Rajamony, W. Yu, and W. Zwaenepoel, “Treadmarks: Shared memory computing on networks of workstations,” *Computer*, vol. 29, no. 2, pp. 18–28, 1996.
- [154] Y. Shan, S.-Y. Tsai, and Y. Zhang, “Distributed shared persistent memory,” in *Proceedings of the 2017 Symposium on Cloud Computing*, pp. 323–337, 2017.
- [155] B. Behzad, S. Byna, Prabhat, and M. Snir, “Optimizing i/o performance of hpc applications with autotuning,” *ACM Trans. Parallel Comput.*, vol. 5, mar 2019.

- [156] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, “Spark: cluster computing with working sets,” in *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing*, HotCloud’10, (USA), p. 10, USENIX Association, 2010.
- [157] Q. Wang, Y. Lu, E. Xu, J. Li, Y. Chen, and J. Shu, “Concordia: Distributed shared memory with in-network cache coherence,” in *FAST*, pp. 277–292, 2021.
- [158] N. Y. Song, Y. Son, H. Han, and H. Y. Yeom, “Efficient memory-mapped i/o on fast storage device,” *ACM Transactions on Storage (TOS)*, vol. 12, no. 4, pp. 1–27, 2016.
- [159] I. Malliotakis, A. Papagiannis, M. Marazakis, and A. Bilas, “Hugemap: Optimizing memory-mapped i/o with huge pages for fast storage,” in *Euro-Par 2020: Parallel Processing Workshops: Euro-Par 2020 International Workshops, Warsaw, Poland, August 24–25, 2020, Revised Selected Papers 26*, pp. 344–355, Springer, 2021.
- [160] J. Won, S. Yun, A. Jemin, J.-C. Kim, and K. Kang, “Spidermine: Low overhead user-level prefetching,” in *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing*, SAC ’23, (New York, NY, USA), p. 1332–1341, Association for Computing Machinery, 2023.
- [161] S. Rivas-Gomez, A. Fanfarillo, S. Valat, C. Laferriere, P. Couvee, S. Narasimhamurthy, and S. Markidis, “ummap-io: User-level memory-mapped i/o for hpc,” in *2019 IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, pp. 363–372, IEEE, 2019.
- [162] W. Liu and A.-I. A. Wang, “Lfuzz: Exploiting locality-enabled techniques for file-system fuzzing,” in *European Symposium on Research in Computer Security*, pp. 507–525, Springer, 2023.
- [163] S. Bitchebe and A. Tchana, “Out of hypervisor (ooh): When nested virtualization becomes practical,” 2022.
- [164] A. Papagiannis, M. Marazakis, and A. Bilas, “Memory-mapped i/o on steroids,” in *Proceedings of the Sixteenth European Conference on Computer Systems*, pp. 277–293, 2021.
- [165] A. Papagiannis, G. Xanthakis, G. Saloustros, M. Marazakis, and A. Bilas, “Optimizing memory-mapped i/o for fast storage devices,” in *Proceedings of the 2020 USENIX Conference on Usenix Annual Technical Conference*, pp. 813–827, 2020.
- [166] G. Feng, H. Cao, X. Zhu, B. Yu, Y. Wang, Z. Ma, S. Chen, and W. Chen, “Tri-cache: A user-transparent block cache enabling high-performance out-of-core processing with in-memory programs,” *ACM Transactions on Storage*, vol. 19, no. 2, pp. 1–30, 2023.
- [167] A. Abulila, V. S. Mailthody, Z. Qureshi, J. Huang, N. S. Kim, J. Xiong, and W.-m. Hwu, “Flatflash: Exploiting the byte-accessibility of ssds within a unified memory-storage hierarchy,” in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 971–985, 2019.

- [168] H. Al Maruf and M. Chowdhury, “Effectively prefetching remote memory with leap,” in *Proceedings of the 2020 USENIX Conference on Usenix Annual Technical Conference*, pp. 843–857, 2020.
- [169] J. Nelson, B. Holt, B. Myers, P. Briggs, L. Ceze, S. Kahan, and M. Oskin, “Latency-tolerant software distributed shared memory,” in *2015 {USENIX} Annual Technical Conference ({USENIX}{ATC} 15)*, pp. 291–305, 2015.
- [170] G. Almasi, “Pgas (partitioned global address space) languages,” 2011.
- [171] A. M. Aji, L. S. Panwar, F. Ji, K. Murthy, M. Chabbi, P. Balaji, K. R. Bisset, J. Dinan, W.-c. Feng, J. Mellor-Crummey, *et al.*, “Mpi-acc: accelerator-aware mpi for scientific applications,” *IEEE transactions on parallel and distributed systems*, vol. 27, no. 5, pp. 1401–1414, 2015.
- [172] B. Chapman, T. Curtis, S. Pophale, S. Poole, J. Kuehn, C. Koelbel, and L. Smith, “Introducing openshmem: Shmem for the pgas community,” in *Proceedings of the Fourth Conference on Partitioned Global Address Space Programming Model*, pp. 1–3, 2010.
- [173] L. V. Kale and S. Krishnan, “Charm++ a portable concurrent object oriented system based on c++,” in *Proceedings of the eighth annual conference on Object-oriented programming systems, languages, and applications*, pp. 91–108, 1993.
- [174] J. Bachan, S. B. Baden, S. Hofmeyr, M. Jacquelin, A. Kamil, D. Bonachea, P. H. Hargrove, and H. Ahmed, “Upc++: A high-performance communication framework for asynchronous computation,” in *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 963–973, IEEE, 2019.
- [175] X. Lu, R. Wang, and X.-H. Sun, “Care: A concurrency-aware enhanced lightweight cache management framework,” in *The 29th IEEE International Symposium on High-Performance Computer Architecture (HPCA-29)*, IEEE Computer Society, 2023.
- [176] M. Hoseinzadeh, “A survey on tiering and caching in high-performance storage systems,” *arXiv preprint arXiv:1904.11560*, 2019.
- [177] Y. T. Jin, S. Ahn, and S. Lee, “Performance analysis of nvme ssd-based all-flash array systems,” in *2018 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pp. 12–21, IEEE, 2018.
- [178] H. Devarajan, A. Kougkas, and X.-H. Sun, “Hfetch: Hierarchical data prefetching for scientific workflows in multi-tiered storage environments,” in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 62–72, IEEE, 2020.
- [179] T. E. Anderson, M. Canini, J. Kim, D. Kostić, Y. Kwon, S. Peter, W. Reda, H. N. Schuh, and E. Witchel, “Assise: Performance and availability via client-local {NVM} in a distributed file system,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pp. 1011–1027, 2020.
- [180] J. Kim, Y. J. Soh, J. Izraelevitz, J. Zhao, and S. Swanson, “Subzero: Zero-copy io for persistent main memory file systems,” in *Proceedings of the 11th ACM SIGOPS Asia-Pacific Workshop on Systems*, pp. 1–8, 2020.

- [181] G. Quintana-Ortí, F. D. Igual, M. Marqués, E. S. Quintana-Ortí, and R. A. Van de Geijn, “A runtime system for programming out-of-core matrix algorithms-by-tiles on multithreaded architectures,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 38, no. 4, pp. 1–25, 2012.
- [182] J. Lüttgau, M. Kuhn, K. Duwe, Y. Alforov, E. Betke, J. Kunkel, and T. Ludwig, “Survey of storage systems for high-performance computing,” *Supercomputing Frontiers and Innovations*, vol. 5, no. 1, 2018.
- [183] T. Vogelsang, “Understanding the energy consumption of dynamic random access memories,” in *2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 363–374, 2010.
- [184] A. Sarma, P. Goyal, S. Kumari, A. Wani, J. S. Challa, S. Islam, and N. Goyal, “ μ dbscan: an exact scalable dbscan algorithm for big data exploiting spatial locality,” in *2019 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 1–11, IEEE, 2019.
- [185] R. Thakur and W. D. Gropp, “Improving the performance of collective operations in mpich,” in *European Parallel Virtual Machine/Message Passing Interface Users’ Group Meeting*, pp. 257–267, Springer, 2003.
- [186] Z. Cai, Z. Liu, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, and O. Saarikivi, “Synthesizing optimal collective algorithms,” in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pp. 62–75, 2021.
- [187] N. Rajesh, H. Devarajan, J. C. Garcia, K. Bateman, L. Logan, J. Ye, A. Kougkas, and X.-H. Sun, “Apollo: An ml-assisted real-time storage resource observer,” in *Proceedings of the 30th International Symposium on High-Performance Parallel and Distributed Computing, HPDC ’21*, (New York, NY, USA), p. 147–159, Association for Computing Machinery, 2020.
- [188] M. Folk, G. Heber, Q. Koziol, E. Pourmal, and D. Robinson, “An overview of the hdf5 technology suite and its applications,” in *Proceedings of the EDBT/ICDT 2011 Workshop on Array Databases, AD ’11*, (New York, NY, USA), p. 36–47, Association for Computing Machinery, 2011.
- [189] D. Vohra, *Apache Parquet*, pp. 325–335. Berkeley, CA: Apress, 2016.
- [190] R. B. Ross, G. Amvrosiadis, P. Carns, C. D. Cranor, M. Dorier, K. Harms, G. Ganger, G. Gibson, S. K. Gutierrez, R. Latham, B. Robey, D. Robinson, B. Settlemeyer, G. Shipman, S. Snyder, J. Soumagne, and Q. Zheng, “Mochi: Composing data services for high-performance computing environments,” *Journal of Computer Science and Technology*, vol. 35, no. 1, pp. 121–144, 2020.
- [191] A. Danial, “cloc: v1.92,” Dec. 2021.
- [192] B. Bahmani, B. Moseley, A. Vattani, R. Kumar, and S. Vassilvitskii, “Scalable k-means++,” *arXiv preprint arXiv:1203.6402*, 2012.
- [193] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, *et al.*, “Mllib: Machine learning in apache spark,” *The journal of machine learning research*, vol. 17, no. 1, pp. 1235–1241, 2016.
- [194] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

- [195] V. Springel, R. Pakmor, O. Zier, and M. Reinecke, “Simulating cosmic structure formation with the gadget-4 code,” *Monthly Notices of the Royal Astronomical Society*, vol. 506, no. 2, pp. 2871–2949, 2021.
- [196] L. Lucie-Smith, H. V. Peiris, A. Pontzen, and M. Lochner, “Machine learning cosmological structure formation,” *Monthly Notices of the Royal Astronomical Society*, vol. 479, pp. 3405–3414, 06 2018.
- [197] R. Rosen, “Namespaces and cgroups, the basis of linux containers,” *Seville, Spain, Feb*, 2016.
- [198] D. Fiala, F. Mueller, and K. B. Ferreira, “Flipsphere: A software-based dram error detection and correction library for hpc,” in *2016 IEEE/ACM 20th International Symposium on Distributed Simulation and Real Time Applications (DS-RT)*, pp. 19–28, IEEE, 2016.
- [199] F. Piscaglia and F. Ghioldi, “Gpu acceleration of cfd simulations in openfoam,” *Aerospace*, vol. 10, no. 9, p. 792, 2023.
- [200] X. Wu, Q. Sun, Z. Pu, T. Zheng, W. Ma, W. Yan, Y. Xia, Z. Wu, M. Huo, X. Li, *et al.*, “Enhancing gpu-acceleration in the python-based simulations of chemistry frameworks,” *Wiley Interdisciplinary Reviews: Computational Molecular Science*, vol. 15, no. 2, p. e70008, 2025.
- [201] R. Bird, N. Tan, S. V. Luedtke, S. L. Harrell, M. Taufer, and B. Albright, “Vpic 2.0: Next generation particle-in-cell simulations,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 952–963, 2021.
- [202] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- [203] R. Acciarri, C. Adams, C. Andreopoulos, J. Asaadi, M. Babicz, C. Backhouse, W. Badgett, L. Bagby, D. Barker, V. Basque, M. C. Q. Bazetto, M. Betancourt, A. Bhandari, A. Bhat, *et al.*, “Cosmic ray background removal with deep neural networks in sbnd,” *Frontiers in Artificial Intelligence*, vol. Volume 4 - 2021, 2021.
- [204] P. Villalobos, A. Ho, J. Sevilla, T. Besiroglu, L. Heim, and M. Hobbhahn, “Position: Will we run out of data? limits of llm scaling based on human-generated data,” in *Forty-first International Conference on Machine Learning*, 2024.
- [205] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [206] R. Kettimuthu, Z. Liu, D. Wheeler, I. Foster, K. Heitmann, and F. Cappello, “Transferring a petabyte in a day,” *Future Generation Computer Systems*, vol. 88, pp. 191–198, 2018.
- [207] S. Abdulah, A. H. Baker, G. Bosilca, Q. Cao, S. Castruccio, M. G. Genton, D. E. Keyes, Z. Khalid, H. Ltaief, Y. Song, *et al.*, “Boosting earth system model outputs and saving petabytes in their storage using exascale climate emulators,” in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–12, IEEE, 2024.

- [208] Y. Lu, I. Yamazaki, F. Ino, Y. Matsushita, S. Tomov, and J. Dongarra, “Reducing the amount of out-of-core data access for gpu-accelerated randomized svd,” *Concurrency and Computation: Practice and Experience*, vol. 32, no. 19, p. e5754, 2020.
- [209] R. Li, D. Fu, C. Shi, Z. Huang, and G. Lu, “Efficient llms training and inference: An introduction,” *IEEE Access*, 2024.
- [210] A. Noohi, M. Derispour, and A. Barbalace, “Rmai: Rethinking memory for ai (inference) in-kernel remote shared memory as a software alternative to cxl,” in *Proceedings of the 5th Workshop on Machine Learning and Systems*, pp. 122–131, 2025.
- [211] J. Sun, M. Sun, Z. Zhang, J. Xie, Z. Shi, Z. Yang, J. Zhang, F. Wu, and Z. Wang, “Helios: An efficient out-of-core gnn training system on terabyte-scale graphs with in-memory performance,” *arXiv preprint arXiv:2310.00837*, 2023.
- [212] J. Zhang, D. Donofrio, J. Shalf, M. T. Kandemir, and M. Jung, “Nvmmu: A non-volatile memory management unit for heterogeneous gpu-ssd architectures,” in *2015 International Conference on Parallel Architecture and Compilation (PACT)*, pp. 13–24, IEEE, 2015.
- [213] D. D. Sharma, “Compute express link white paper,” *Compute Express Link Consortium*, 2019.
- [214] National Center for Supercomputing Applications (NCSA), *DeltaAI*, 2024.
- [215] Z. Qureshi, V. S. Mailthody, I. Gelado, S. Min, A. Masood, J. Park, J. Xiong, C. J. Newburn, D. Vainbrand, I.-H. Chung, *et al.*, “Gpu-initiated on-demand high-throughput storage access in the bam system architecture,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pp. 325–339, 2023.
- [216] S. Qiu, W. Liu, Y. Hu, J. Yan, Z. Shen, X. Yao, R. Chen, G. Zhang, and Y. Zhang, “{GeminiFS}: A companion file system for {GPUs},” in *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pp. 221–236, 2025.
- [217] W. F. Godoy, N. Podhorszki, R. Wang, C. Atkins, G. Eisenhauer, J. Gu, P. Davis, J. Choi, K. Germaschewski, K. Huck, *et al.*, “Adios 2: The adaptable input output system. a framework for high-performance data management,” *SoftwareX*, vol. 12, p. 100561, 2020.
- [218] A. Parquet, “Apache parquet,” 2025.
- [219] Y. Liu, Y. Lu, Z. Chen, and M. Zhao, “Pacon: Improving scalability and efficiency of metadata service through partial consistency,” in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 986–996, IEEE, 2020.
- [220] J. Xing, J. Xiong, N. Sun, and J. Ma, “Adaptive and scalable metadata management to support a trillion files,” in *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis*, pp. 1–11, 2009.

- [221] M. Silberstein, B. Ford, I. Keidar, and E. Witchel, “Gpufs: Integrating a file system with gpus,” in *Proceedings of the eighteenth international conference on Architectural Support for Programming Languages and Operating Systems*, pp. 485–498, 2013.
- [222] P. Maucher, L. Kittner, N. Rath, G. Lucka, L. Werling, Y. Khalil, T. Gröninger, and F. Bellosa, “Full-scale file system acceleration on gpu.” Tagungsband des FG-BS Frühjahrstreffens 2024, 2024.
- [223] P. Markthub, M. E. Belviranlı, S. Lee, J. S. Vetter, and S. Matsuoka, “Dragon: breaking gpu memory capacity limits with direct nvm access,” in *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 414–426, IEEE, 2018.
- [224] S. Shahar, S. Bergman, and M. Silberstein, “Activepointers: a case for software address translation on gpus,” *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 596–608, 2016.
- [225] C.-H. Chang, J. Han, A. Sivasubramaniam, V. Sharma Mailthody, Z. Qureshi, and W.-M. Hwu, “Gmt: Gpu orchestrated memory tiering for the big data era,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pp. 464–478, 2024.
- [226] J. Kraus, “An introduction to cuda-aware mpi,” 2013. Accessed: 2025-04-02.
- [227] NVIDIA, “Nvidia collective communications library (nccl),” 2024. Accessed: 2025-04-02.
- [228] Weka, “Gpudirect storage: How it works and more,” 2024. Accessed: 2025-04-02.
- [229] BeegFS, “Gpudirect storage support,” 2024. Accessed: 2025-04-02.
- [230] NVIDIA, “Introducing gpudirect storage,” *NVIDIA Developer Blog*, pp. <https://developer.nvidia.com/blog/gpudirect-storage/>, 2019. Accessed [date].
- [231] M. Arif, A. Maurya, and M. M. Rafique, “Accelerating performance of gpu-based workloads using cxl,” in *Proceedings of the 13th Workshop on AI and Scientific Computing at Scale using Flexible Computing*, pp. 27–31, 2023.
- [232] S. Sano, Y. Bando, K. Hiwada, H. Kajihara, T. Suzuki, Y. Nakanishi, D. Taki, A. Kaneko, and T. Shiozawa, “Gpu graph processing on cxl-based microsecond-latency external memory,” in *Proceedings of the SC’23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, pp. 962–972, 2023.
- [233] R. Abdullah, H. Lee, H. Zhou, and A. Awad, “Salus: Efficient security support for cxl-expanded gpu memory,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 1–15, IEEE, 2024.
- [234] IBM, “Installing gpudirect storage for ibm storage scale,” 2024. Accessed: 2025-04-02.

- [235] C. R. Trott, D. Lebrun-Grandié, D. Arndt, J. Ciesko, V. Dang, N. Ellingwood, R. Gayatri, E. Harvey, D. S. Hollman, D. Ibanez, *et al.*, “Kokkos 3: Programming model extensions for the exascale era,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 805–817, 2021.
- [236] NVIDIA, “Nvshmem,” 2024. Accessed: 2025-04-02.
- [237] S. Kazemkhani, A. Pandya, D. Cornelisse, B. Shacklett, and E. Vinitzky, “Gpu-drive: Data-driven, multi-agent driving simulation at 1 million fps,” *arXiv preprint arXiv:2408.01584*, 2024.
- [238] G. G. Trabes, A. Inostrosa-Psijas, V. Gil-Costa, and G. A. Wainer, “Flattened parallel devs simulations on gpu architectures,” in *2024 Annual Modeling and Simulation Conference (ANNSIM)*, pp. 1–13, IEEE, 2024.
- [239] J. Pan and T. G. Rogers, “Crisp: Concurrent rendering and compute simulation platform for gpus,” in *2024 IEEE International Symposium on Workload Characterization (IISWC)*, pp. 1–14, IEEE, 2024.
- [240] P. Corbett, D. Feitelson, S. Fineberg, Y. Hsu, B. Nitzberg, J.-P. Prost, M. Snirt, B. Traversat, and P. Wong, “Overview of the mpi-io parallel i/o interface,” *Input/Output in Parallel and Distributed Computer Systems*, pp. 127–146, 1996.
- [241] J. Li, W.-k. Liao, A. Choudhary, R. Ross, R. Thakur, W. Gropp, R. Latham, A. Siegel, B. Gallagher, and M. Zingale, “Parallel netcdf: A high-performance scientific i/o interface,” in *Proceedings of the 2003 ACM/IEEE conference on Supercomputing*, p. 39, 2003.
- [242] W. Frings, F. Wolf, and V. Petkov, “Scalable massively parallel i/o to task-local files,” in *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis*, pp. 1–11, 2009.
- [243] A. K. Y. Yanamala, “Emerging challenges in cloud computing security: A comprehensive review,” *International Journal of Advanced Engineering Technologies and Innovations*, vol. 1, no. 4, pp. 448–479, 2024.
- [244] J. S. McGough and K. Riley, “Pattern formation in the gray-scott model,” *Nonlinear analysis: real world applications*, vol. 5, no. 1, pp. 105–121, 2004.
- [245] J. Hong, S. Cho, G. Park, W. Yang, Y.-H. Gong, and G. Kim, “Bandwidth-effective dram cache for gpu s with storage-class memory,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 139–155, IEEE, 2024.
- [246] W. Gropp, T. Boerner, B. Bode, and G. Bauer, “Delta: Balancing gpu performance with advanced system interfaces,” 2023.
- [247] K. Keahey, J. Anderson, Z. Zhen, P. Riteau, P. Ruth, D. Stanzione, M. Cevik, J. Colleran, H. S. Gunawi, C. Hammock, J. Mambretti, A. Barnes, F. Halbach, A. Rocha, and J. Stubbs, “Lessons learned from the chameleon testbed,” in *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC ’20)*, USENIX Association, July 2020.
- [248] A. Raad, O. Lahav, J. Wickerson, P. Balcer, and B. Dongol, “Intel pmdk transactions: Specification, validation and concurrency (extended version),” 2023.

- [249] Alluxio, Inc., “Alluxio: Data orchestration for the cloud,” 2023. Accessed: 2024-09-22.
- [250] S. Yang, T. Harter, N. Agrawal, S. S. Kowsalya, A. Krishnamurthy, S. Al-Kiswany, R. T. Kaushik, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “Split-level i/o scheduling,” in *Proceedings of the 25th Symposium on Operating Systems Principles*, pp. 474–489, 2015.